Appunti di Basi di Dati

Alessio Novel

Introduzione....3
Modellazione concettuale ER....4
Processo di progettazione di un database....4
Concetti del modello ER....7
Entità e attributi....7
Rappresentazione di entity type....9
Relationship....11
Vincoli di relationship....14
Entità deboli....17
Relationship di grado superiore....18
Aggregation....20
Modellazione concettuale EER....21
Sottoclassi e superclassi....21
Vincoli su specializzazioni e generalizzazioni....23
Union types....28
Definizioni formali del modello EER....28
Modellazione relazionale dei dati....29
Concetti del modello relazionale....29
Caratteristiche delle relazioni....33
Vincoli....34
Vincoli sugli attributi....34
Integrità delle entità....35
Vincoli tra due relazioni....35
Altri tipi di vincolo....39
Algebra relazionale....40
Operazioni unarie....43
SELECT....43
PROJECT....45
Espressioni....47
RENAME....48
Operazioni insiemistiche....49
UNION....49
INTERSECTION....50
SET DIFFERENCE....50
Cartesian Product....52
Operazioni binarie....54
JOIN....54
DIVISION....57
OUTER UNION....58

Riassunto....59
Query tree....60
Il linguaggio SQL....61
DDL....62
DML....68
Da ER a Relazionale....77
Normalizzazione per database relazionali....85
Linee guida per la progettazione informale di database relazionali....85
Dipendenze funzionali....88
Normalizzazione....90
Prima forma normale....90
Seconda forma normale....91
Terza forma normale....92
Forma normale di Boyce e Codd....93
Terza forma normale....98
Chiusure....100
Chiusura di un insieme di dipendenze funzionali....100
Chiusura di un insieme di attributi....102
Copertura minimale....105
Modello di costo e Metodi di storage....107
Memoria centrale e secondaria....107
Modelli di costo....108
Metodi di allocazione dei dati....110
Heap file....110
Sorted file....113
Indexed file....116
B+Tree....120
Hash indexes....123
Creazione indice su DBMS....126
Operazioni di Join....127
Nested loop join....127
Sort-merge join....128
External sorting....129
Hash join....130
Index Nested loop join....132
Tabelle riassuntive costi....133

Introduzione

Iniziamo vedendo a grandi linee il ruolo dei **dati** nel XXI secolo.
I dati sono considerati il **petrolio** dell'era digitale.
Un data scientist spreca l'80% del proprio tempo per "pulire" i dati, che arrivano da varie fonti in vari formati.
I database difficilmente vengono fatti "giusto per", ma hanno sempre uno scopo di qualche tipo, ad esempio:

- Sistemi informativi aziendali
- Applicazioni web e mobile
- Business intelligence
- Machine learning / data mining
- Big data analytics
- Intelligenza artificiale

I principi di **modellazione** che vedremo in questo corso sono uguali per ogni campo
La mole di dati registrati ogni giorno al giorno d'oggi pone sfide scientifiche e tecnologiche nuove e straordinarie
I dati sono importantissimi soprattutto per quanto riguarda le tecnologie che abbiamo oggi, che un tempo non erano possibili non per mancanza di conoscenze, ma per mancanza di dati (ad esempio Google Translate usa il confronto su miliardi di documenti per eseguire la traduzione migliore/più comune)
Il modello relazionale è rigido, invece il modello NOSQL non richiede di progettare a priori lo schema ed è più efficiente.
All'inizio NOSQL voleva dire appunto NO SQL ma poi si è evoluto in NOT ONLY SQL.

Dividere un progetto in presentation, business e data è molto consigliato per il **separation of concerns** (separazione dei problemi) e quindi, ad esempio, possiamo cambiare l'interfaccia (presentation) senza toccare le funzionalità (business)

Il corso si propone di fornire gli strumenti di base per:

- **Progettare basi di dati** per la gestione strutturata dei dati e lo sviluppo di applicazioni e analisi
- **Utilizzare data base management systems (DBMS) relazionali** per gestire i dati
- **Rendere più efficiente** l'uso dei dati mediante la creazione di indici
- **Integrare basi di dati in applicazioni** (ad esempio sistemi informativi, applicazioni web, applicazioni mobile, etc.)

Per raggiungere questi obiettivi, il corso si articolerà nei seguenti moduli principali:

- **La progettazione concettuale: il modello E-R**
- **Modello relazionale / algebra relazionale / linguaggio SQL**
- **Metodi per un uso efficiente delle basi di dati** (normalizzazione, indici, costo delle query)
- Uso delle basi di dati per la **costruzione di sistemi informativi** e **applicazioni (web)**

Modellazione concettuale ER

Processo di progettazione di un database

Ci sono due attività principali:

- Progettazione del database
- Progettazione dell'applicazione

In questa parte ci concentreremo sulla progettazione **concettuale** del database (per progettare lo schema concettuale di un database per un'applicazione).
La progettazione dell'applicazione si concentra sui programmi e le interfacce che accedono al database (in genere è considerata parte dell'ingegneria del software).

Nella seguente immagine vediamo una panoramica delle fasi di sviluppo di database, il "minimondo" è la base della progettazione, perché bisogna contestualizzare e capire a cosa servirà il database.

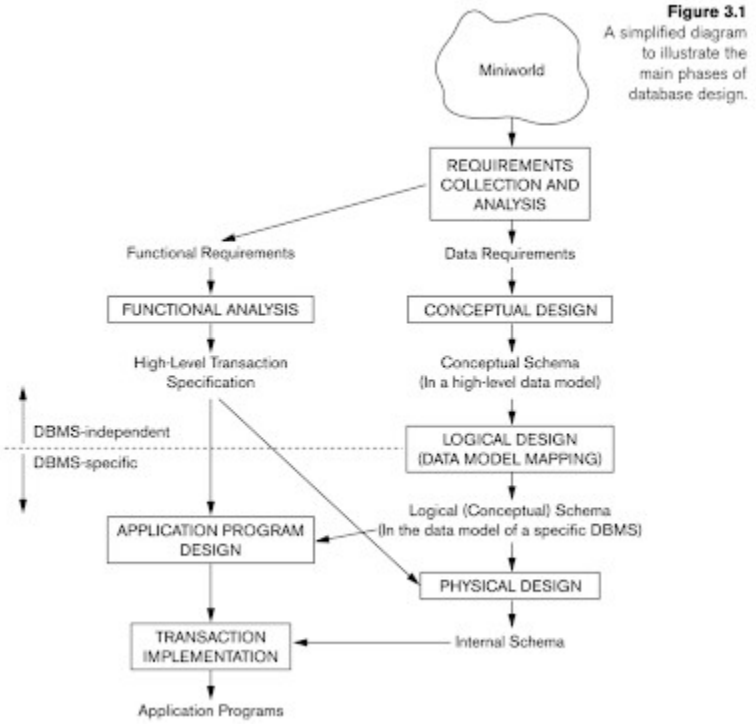
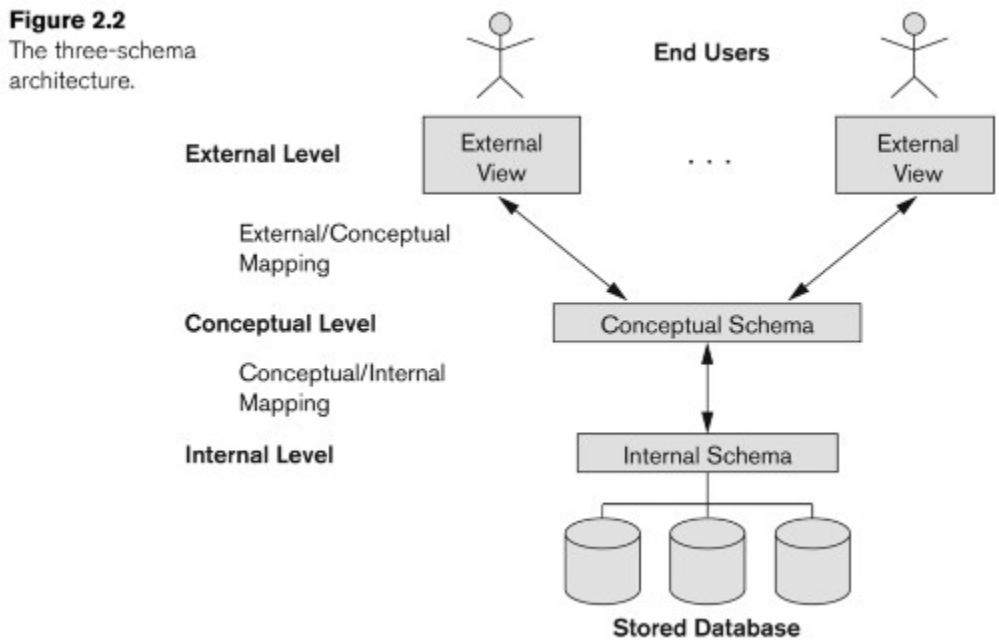


Figure 3.1
A simplified diagram to illustrate the main phases of database design.



Esistono due principali metodologie per la progettazione concettuale di un database:

- Entity Relationship o ER (questo capitolo)
- Enhanced Entity Relationship EER (vedremo più avanti)

Partiamo da un esempio:

Example COMPANY Database

- We need to create a database schema design based on the following (simplified) **requirements** of the COMPANY Database:
  - The company is organized into DEPARTMENTs. Each department has a name, number and an employee who *manages* the department. We keep track of the start date of the department manager. A department may have several locations.
  - Each department *controls* a number of PROJECTs. Each project has a unique name, unique number and is executed at a single location.

- The database will store each EMPLOYEE's social security number, address, salary, sex, and birthdate.
  - Each employee *works for* one department but may *work on* several projects.
  - The DB will keep track of the number of hours per week that an employee currently works on each project.
  - It is required to keep track of the *direct supervisor* of each employee.

- Each employee may *have* a number of DEPENDENTs.
  - For each dependent, the DB keeps a record of name, sex, birthdate, and relationship to the employee.

L'obiettivo è ottenere questo come risultato finale. Vedremo cosa significa ogni concetto e simbolo e come arrivare a questo risultato da zero.

Il nostro target: COMPANY ER Schema

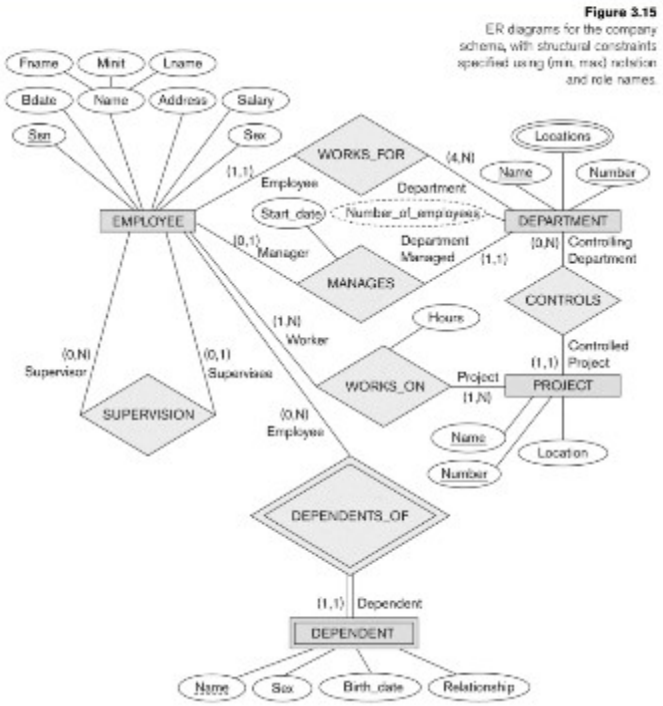


Figure 3.15
ER diagrams for the company schema, with structural constraints specified using (min, max) notation and role names.

Concetti del modello ER

Entità e attributi

Alla base del modello Entity-Relationship, come suggerisce il nome, ci sono le **entità**, che sono delle specifiche cose/oggetti appartenenti al miniworld che vengono rappresentate nel database. (ad esempio l'EMPLOYEE Alessio Novel oppure il research DEPARTMENT)
Gli **attributi** sono le proprietà usate per descrivere un entità, e sono praticamente i suoi dati **rilevanti** (rilevanti al fine dell'applicazione) (EMPLOYEE ha come attributi Ssn, FName, BDate etc).
Ovviamente ogni specifica entità avrà un **valore** per ogni attributo (ad esempio uno specifico EMPLOYEE avrà $name = Alessio \;\; Novel$ etc), e ogni attributo ha un suo **insieme di valori** associati (value set o data type), ovvero che tipo di valore può assumere quell'attributo (integer, string di 15 caratteri etc). I value set si possono vedere come dei data types in un linguaggio di programmazione.

Esistono 3 tipi di attributo:

- **Semplice**, ogni entità ha un singolo valore atomico per quell'attributo
- **Composto**, l'attributo può essere la composizione di varie componenti, ad esempio Address(Apt#, House#, Street, City, State, ZipCode, Country)
  La composizione può formare una gerarchia dove certe componenti sono a loro volta degli attributi composti
- **Multi-valore**, l'entità può avere più valori per quell'attributo (ad esempio colore dell'auto, che può avere più colori, denotato come $\{Colore\}$)

Notiamo che attributi composti e multivalore possono essere combinati in vari modi per formare strutture complesse (anche se è raro farlo) ad esempio {PreviousDegrees (College, Year, Degree, Field)} così uno studente può avere più lauree (ognuna con 4 attributi della sottocomponente).

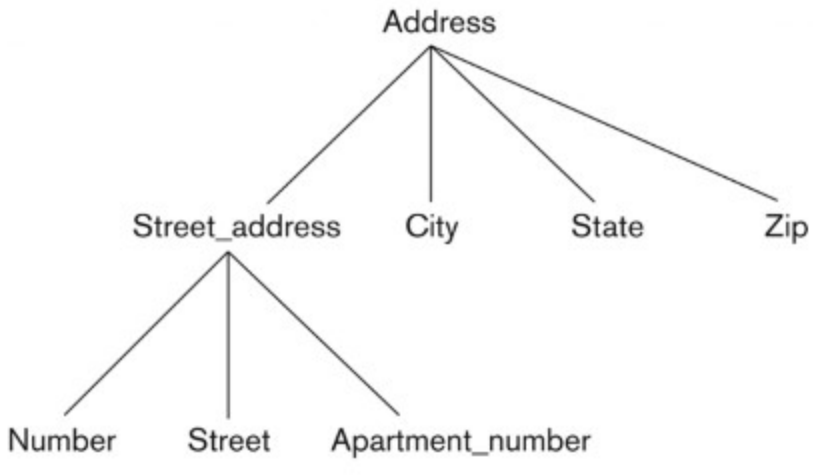


Figure 3.4
A hierarchy of composite attributes.

Le entità con gli stessi attributi base sono raggruppate in un **entity type**, che è come se fosse il nome della classe in Java (nell'esempio EMPLOYEE è un entity type).

Un attributo di un entity type per il quale ogni entità deve avere un **unico** valore (nel senso che deve essere unico in confronto a tutti gli altri valori delle altre entità) è detto **attributo chiave** dell'entity type (ad esempio l'attributo SSN nell'entity type EMPLOYEE). Una chiave può essere un attributo composto (la targa di un'auto composta da numero e stato) e un'entity type può possedere più di una chiave (una persona ha il codice della carta di identità e il codice del passaporto). Nel modello ER ogni chiave è **sottolineata** (attenzione che questo vale solo nel modello concettuale, e che nello schema relazionale solo la **primary key** viene sottolineata).
Nel modello E-R si rappresentano tutte le chiavi candidate (ovvero più attributi che potranno poi diventare chiave nel modello finale)
Ecco un esempio di quanto detto finora:

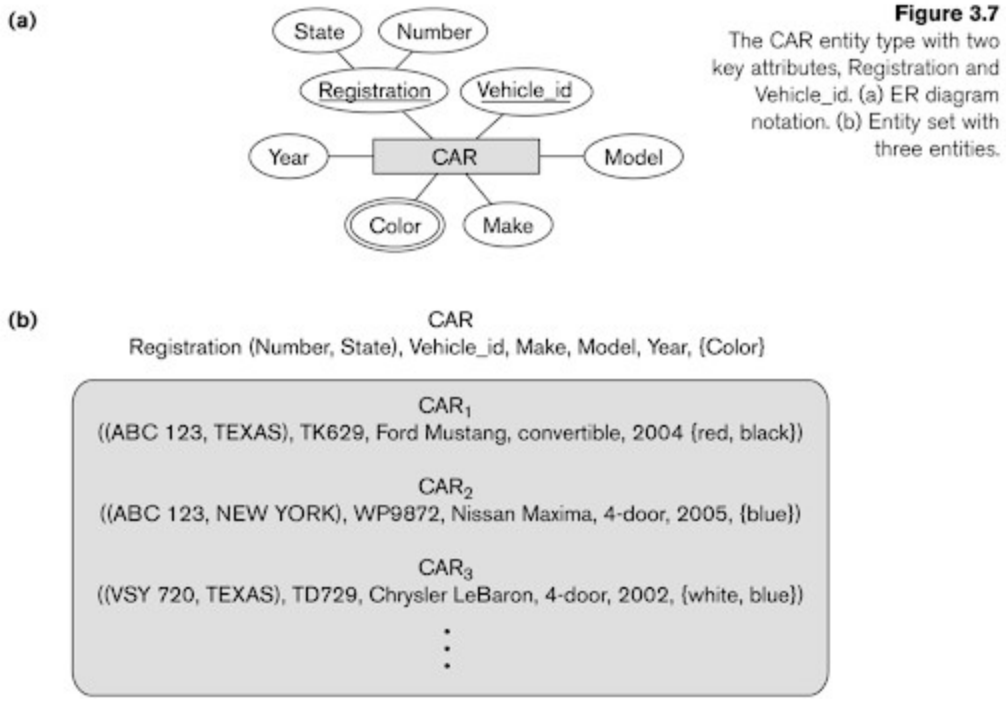


Figure 3.7
The CAR entity type with two key attributes, Registration and Vehicle_id. (a) ER diagram notation. (b) Entity set with three entities.

Ogni entity type ha una collezione di entità salvate nel database, questo insieme viene chiamato **entity set** oppure **entity collection** (nell'esempio precedente ci sono 3 entità CAR nell'entity set CAR). Lo stesso nome (CAR) è usato per riferirsi sia all'entity type che all'entity set.
L'entity set è la **situazione corrente/attuale** delle entità di un certo entity type salvate sul database.

Ogni attributo semplice è associato con un value set (come abbiamo già detto precedentemente).
Matematicamente un attributo A per un entity type E per il quale il value set è V è definito come una funzione (totale):

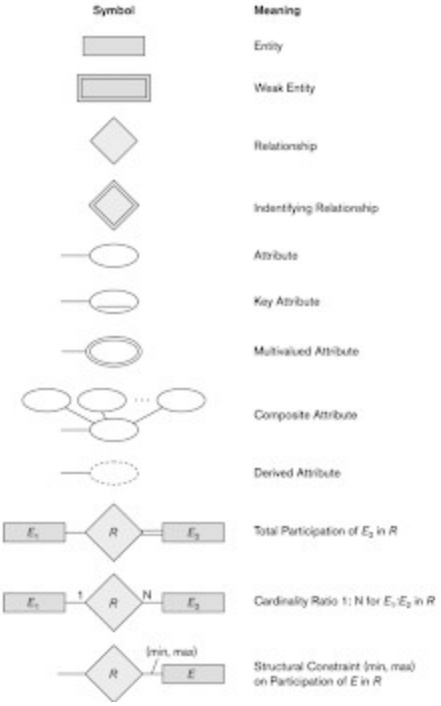
$$A : E \rightarrow P(V)$$

Dove $P(V)$ indica l'insieme potenza di V (ovvero tutti i possibili sottoinsiemi di V), comprendiamo quindi sia attributi semplici che multivalore.
Con la notazione $A(e)$ ci riferiamo al valore dell'attributo A per l'entità e.

Rappresentazione di entity type

Nei diagrammi E-R l’entity type è rappresentata come una **scatola rettangolare** e gli attributi sono disegnati come **ovali**, ogni attributo è collegato al suo entity type.
Le componenti di un attributo composto sono rappresentate dalla **connessione** all’ovale che rappresenta l’attributo composto. Ogni chiave è **sottolineata**. Gli attributi multi valore sono disegnati con un **doppio ovale**.
Ecco la notazione ER completa:

Figure 3.14
Summary of the notation for ER diagrams.



Adesso riprendiamo l'esempio COMPANY e vediamo che entità e attributi possiamo trovare:

Initial Conceptual Design of Entity Types for the COMPANY Database Schema

- Based on the requirements, we can identify four initial entity types in the COMPANY database:
  - DEPARTMENT
  - PROJECT
  - EMPLOYEE
  - DEPENDENT
- Their initial conceptual design is shown on the following slide
- The initial attributes shown are derived from the requirements description

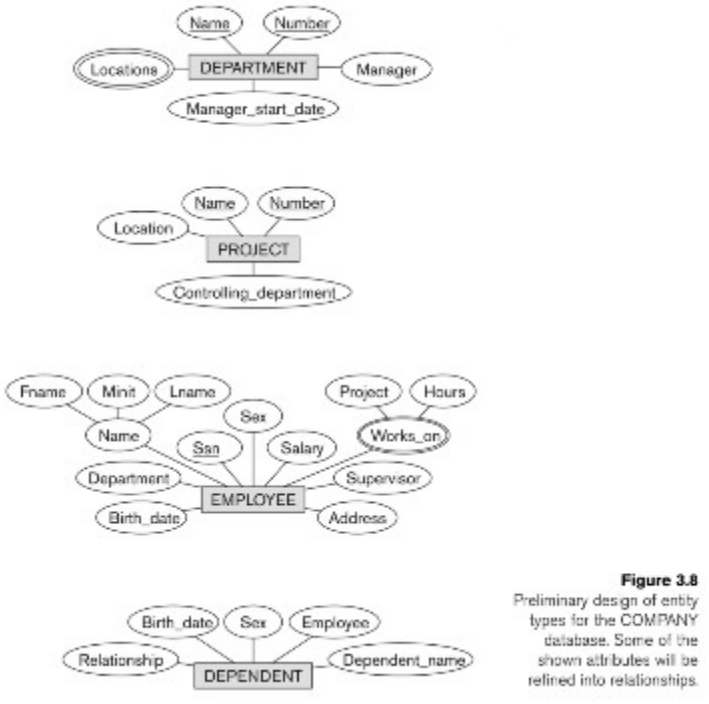


Figure 3.8
Preliminary design of entity types for the COMPANY database. Some of the shown attributes will be refined into relationships.

Relationship

C’è un problema però, ora abbiamo varie entità ma non c’è nessuna correlazione tra loro (che è richiesta dai requisiti), quindi sono poco utili. Introduciamo quindi le **relationship**.
Una relationship si riferisce a due o più entità distinte con uno specifico significato (l’EMPLOYEE Alessio Novel **lavora** al PROJECT progettox)
Le relationship dello stesso tipo sono raggruppate in un **relationship type** (ad esempio “LAVORA_A”, “POSSIEDE_UN”)
Il **grado** di una relationship si riferisce al numero di entity type che partecipano alla relationship.

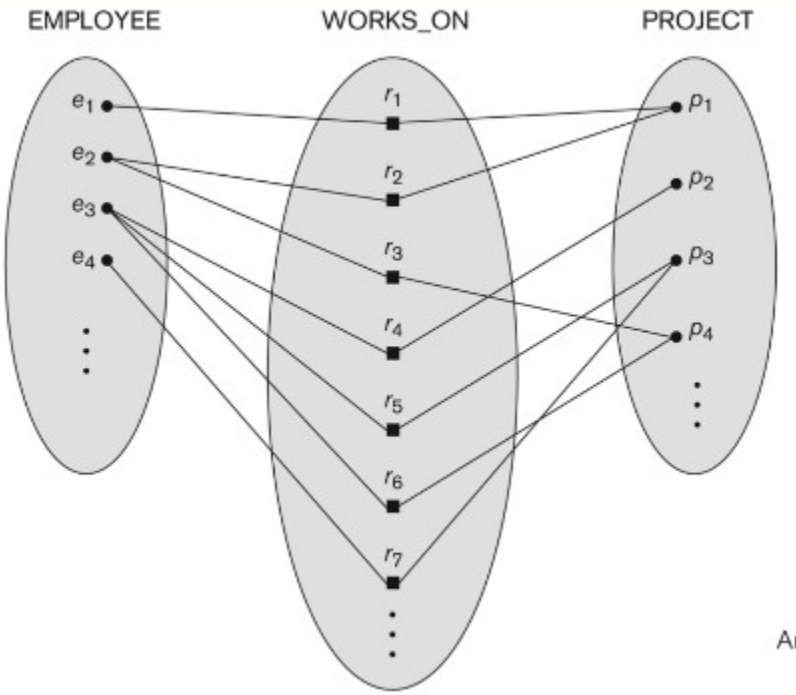


Figure 3.13
An M:N relationship, WORKS_ON.

La **relationship type** è la descrizione dello schema della relationship, identifica il nome della relationship e le entity type che ne fanno parte e identifica anche i **vincoli** della relationship.
Il **relationship set** invece è l’insieme di tutte le istanze di una relationship presenti nel database, si può dire che è lo stato attuale di uno relationship type (la figura precedente mostra un relationship set). Ogni istanza nel relationship set si riferisce a delle specifiche entità partecipanti, una per ogni entity type partecipante.

Nel modello ER abbiamo dei box a forma di diamante per rappresentare le relationship type, che sono collegate con le entity type che fanno parte della relationship e il “verso” di lettura deve essere **da sinistra a destra**, oppure **dall’alto verso il basso**

Aggiungiamo le relationship al nostro esempio dell’azienda:

Refining the COMPANY database schema by introducing relationships

- By examining the requirements, six relationship types are identified
- All are *binary* relationships(degree 2)
- Listed below with their participating entity types:
  - WORKS_FOR (between EMPLOYEE, DEPARTMENT)
  - MANAGES (also between EMPLOYEE, DEPARTMENT)
  - CONTROLS (between DEPARTMENT, PROJECT)
  - WORKS_ON (between EMPLOYEE, PROJECT)
  - SUPERVISION (between EMPLOYEE (as subordinate), EMPLOYEE (as supervisor))
  - DEPENDENTS_OF (between EMPLOYEE, DEPENDENT)

ER DIAGRAM – Relationship Types are:

WORKS_FOR, MANAGES, WORKS_ON, CONTROLS, SUPERVISION, DEPENDENTS_OF

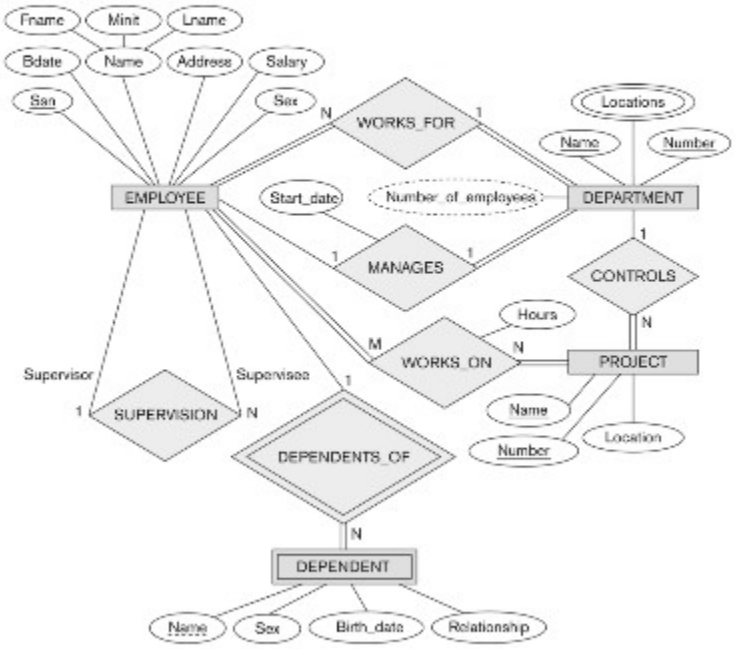


Figure 3.2
An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter.

Ora analizzando la nostra progettazione iniziale delle entità vediamo che è sparito qualche attributo, questo proprio perchè vengono ridefiniti dalle relationship:

Initial Design of Entity Types:
EMPLOYEE, DEPARTMENT, PROJECT, DEPENDENT

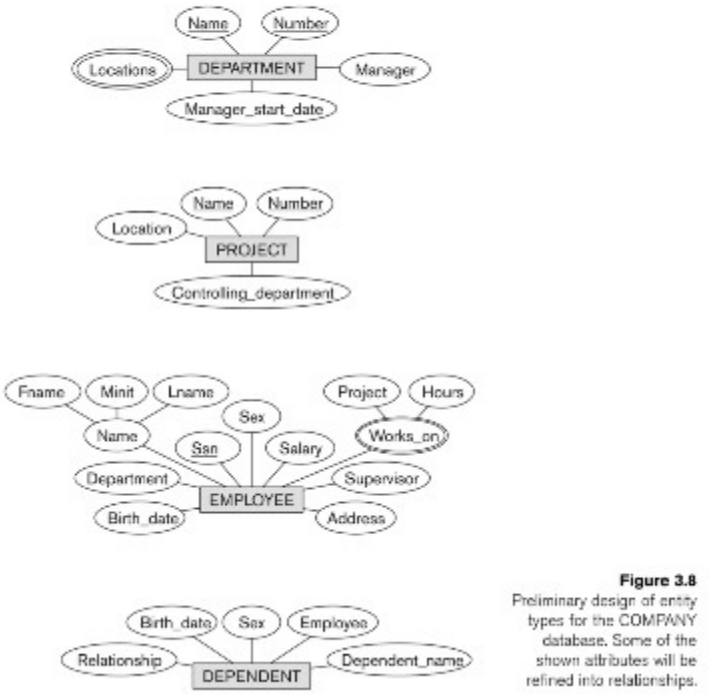


Figure 3.8
Preliminary design of entity types for the COMPANY database. Some of the shown attributes will be refined into relationships.

Discussion on Relationship Types

- In the refined design, some attributes from the initial entity types are refined into relationships:
  - Manager of DEPARTMENT -> MANAGES
  - Works_on of EMPLOYEE -> WORKS_ON
  - Department of EMPLOYEE -> WORKS_FOR
  - etc

In generale può succedere che tra due entity type ci siano più di una relationship type (con significati differenti), e possono anche esistere delle relationship type che comprendono due entità appartenenti allo stesso entity type (self-referencing)

Quindi un modello ER ha tre principali concetti:

- Entità (con le loro entity types e entity sets)
- Attributi (semplice, composto, multivalore)

- Relationship (con le loro relationship types e relationship sets)

Vincoli di relationship

Le relationship possono avere 3 diversi rapporti di cardinalità:

- One-to-one (1:1)
  Ad esempio “persona possiede carta d’identità”
- One-to-many (1:N) oppure Many-to-one (N:1)
  Ad esempio “docente affiliato a dipartimento”
- Many-to-many (M:N)
  Ad esempio “studente frequenta corso”

Una relationship di grado 2 (quindi binaria) tra due entità $E1$ e $E2$ è formalmente rappresentata come:

$$R(E1, E2) \subseteq (E1 \times E2)$$

ovvero da un sottoinsieme delle coppie $<e1, e2>$ appartenenti al prodotto cartesiano dei due entity set $E1$ ed $E2$, dove $e1 \in E1$ e $e2 \in E2$

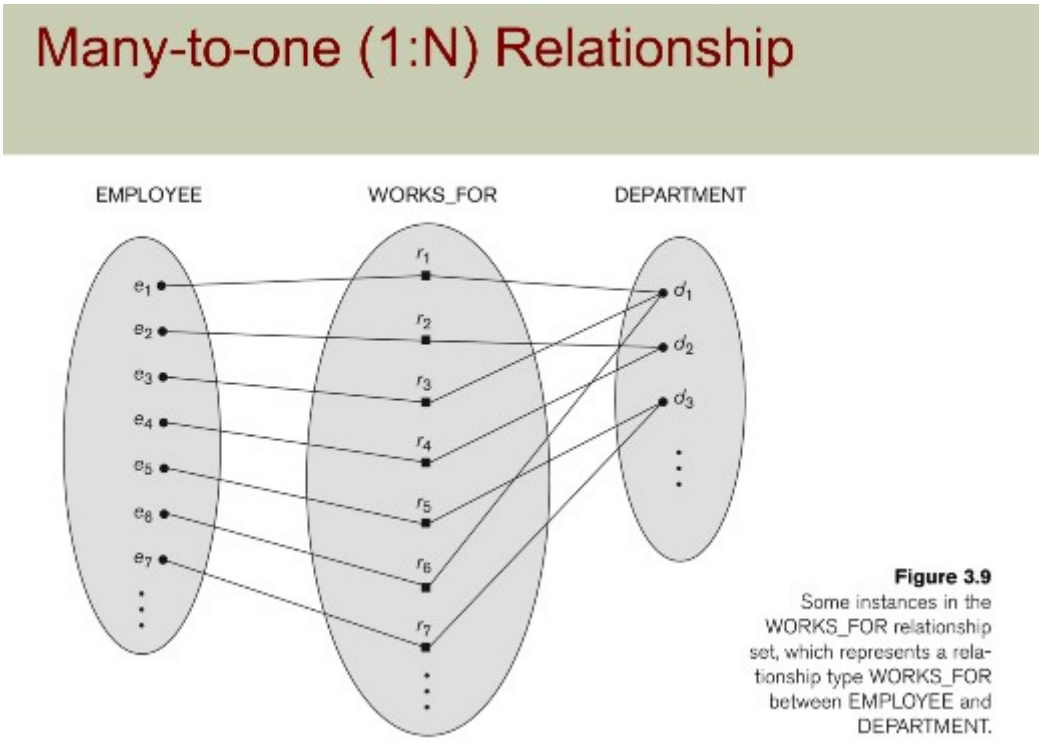


Figure 3.9
Some instances in the WORKS_FOR relationship set, which represents a relationship type WORKS_FOR between EMPLOYEE and DEPARTMENT.

Many-to-many (M:N) Relationship

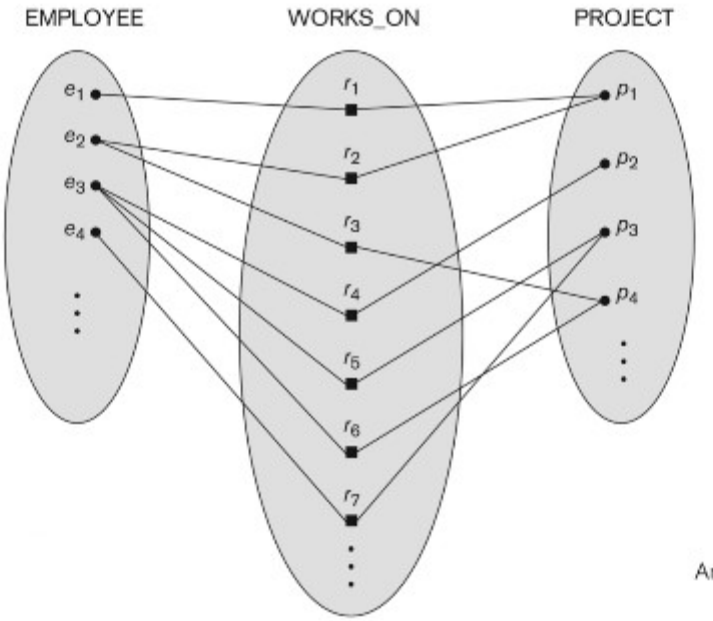


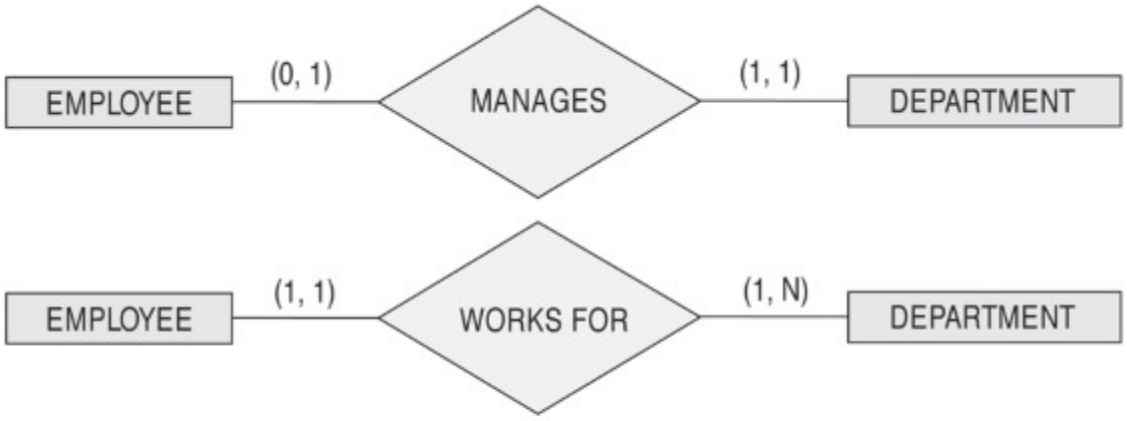
Figure 3.13
An M:N relationship, WORKS_ON.

Esistono dei vincoli in alcune relationship, ad esempio i **vincoli di dipendenza di esistenza** (o anche vincoli di partecipazione), ovvero che un'entità **deve** per forza avere almeno una relationship con un'altra entità per esistere. Indica il numero minimo di istanze di relationship a cui un'entità deve partecipare:

- Se è zero (**partecipazione parziale**)
  Allora la partecipazione è opzionale, non c'è una dipendenza di esistenza
- Se è uno o superiore (**partecipazione totale**)
  Allora la partecipazione è obbligatoria, e c'è la dipendenza di esistenza

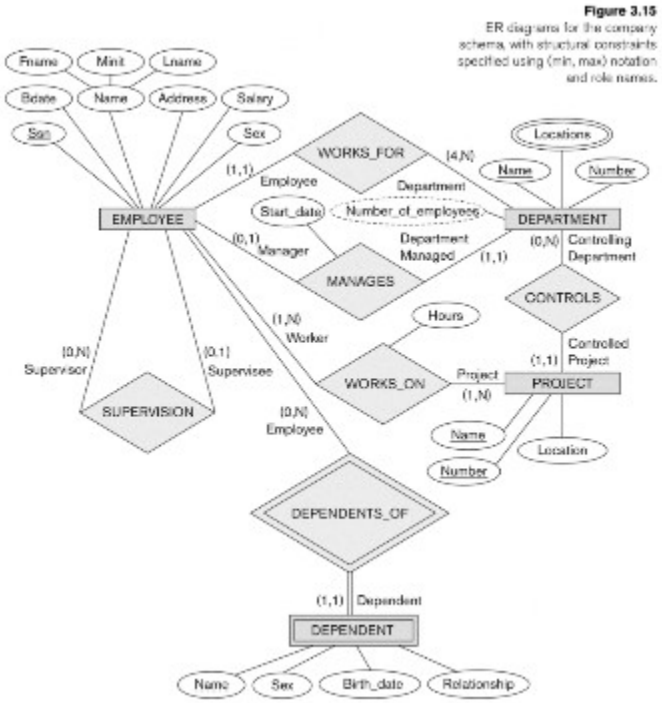
Per la partecipazione totale bisogna usare una linea doppia nel modello E-R verso l'entity type partecipante totalmente, invece per la partecipazione parziale si usa una linea singola.

Esiste un'altra notazione per descrivere i vincoli di relationship, ed è quella del minimo e del massimo delle partecipazioni alla relationship type di un'entity type (il default è $(0, N)$).
Ovviamente deve valere $min \leq max$, $min \geq 0$ e $max \geq 1$
Per esempio un dipartimento ha esattamente un dipendente manager, e un dipendente può essere il manager di al massimo un dipartimento.



Read the min,max numbers next to the entity type and looking **away from** the entity type

Vediamo il nostro esempio con questa notazione:



Una relationship self-referencing deve distinguere i **ruoli** della relationship (nel caso di una relationship di supervisione avremo il supervisore e il supervisionato).
Nelle relationship self-referencing in un E-R diagram bisogna inserire i nomi dei ruoli dei partecipanti per capire in che senso vale la cardinalità

Recursive Relationship Type is: SUPERVISION (participation role names are shown)

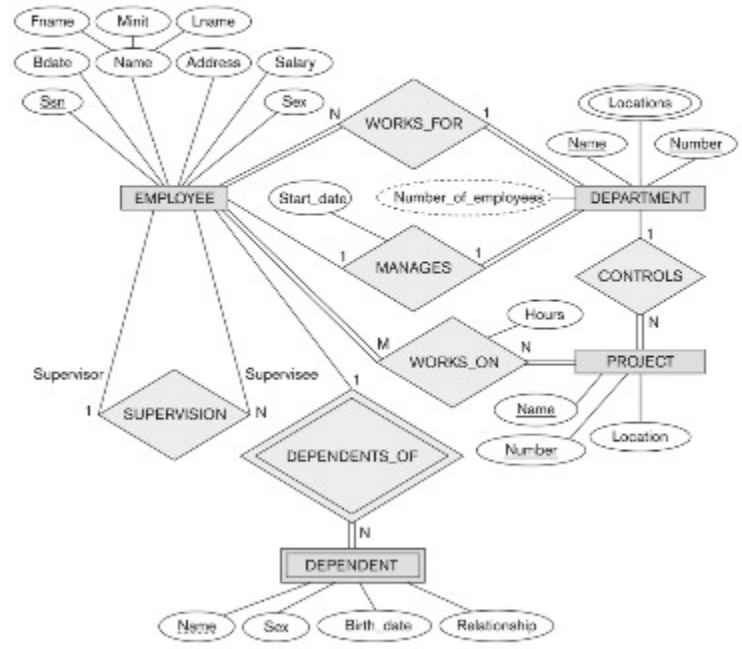
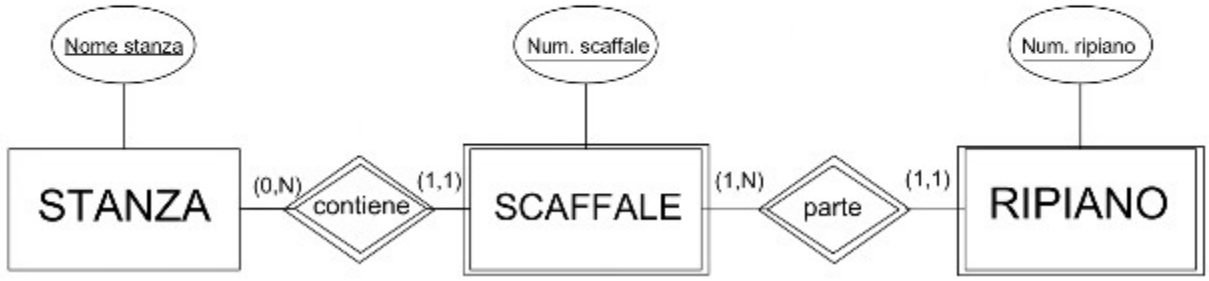


Figure 3.2
An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter.

Entità deboli

Esistono alcune entity type che non hanno attributi chiave, ma che la loro identificazione dipende da altre entity type, dette **entità deboli**.
Una entità debole dovrà sempre avere una **identifying relationship type** con il suo **identifier entity type** per poter formare una chiave “composta”.
L’entità quindi è identificata da un **attributo** dell'entità debole (**chiave parziale**) e dalla specifica **entità** che gli è associata tramite l’identifying relationship type.
Le entità deboli vengono rappresentate con un box con linea doppia, e l’identifying relationship type avrà anche una linea doppia per far capire in che verso vale la “debolezza”.

- In una biblioteca, lo **scaffale** all'interno di una **stanza** (e il **ripiano** su uno **scaffale**)



Le relationship **non** hanno una chiave, ma possono avere attributi (è comune che siano nelle relationship $(M:N)$, visto che nelle altre relationship possono essere trasferiti sulle entità, anche se concettualmente è giusto tenerli come attributi di relationship)

Relationship di grado superiore

Le relationship type di grado 2 sono dette binarie.
Le relationship type di grado 3 sono dette ternarie.
Le relationship type di grado n sono dette n-arie.
In genere una relationship n-aria non è equivalente a n relationship binarie (altrimenti si preferiscono sempre le n relationship binarie)

Example of a ternary relationship

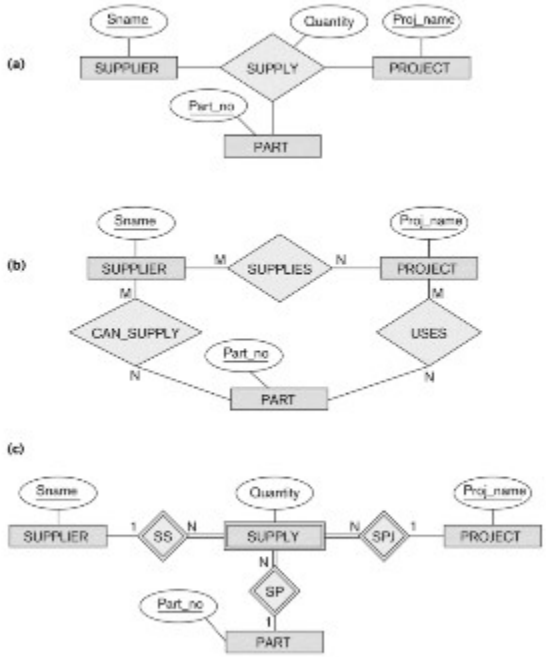
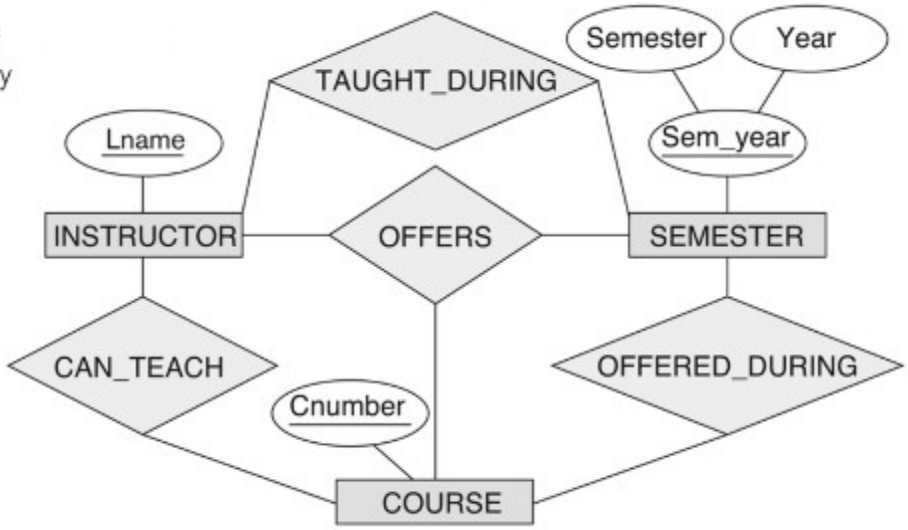


Figure 3.17
Ternary relationship types. (a) The SUPPLY relationship. (b) Three binary relationships not equivalent to SUPPLY. (c) SUPPLY represented as a weak entity type.

In genere una relationship ternaria può essere rappresentata da 3 relationship binarie, ma questo non è sempre vero perché il significato di una relationship ternaria è diverso di quello di 3 relationship binarie.
Potrebbe servire includere in uno schema di database sia la relationship ternaria che le 3 relationship binarie, perché appunto ciascuna relationship ha un significato differente (figura a e figura b).
In certe situazioni si può far diventare la relationship un’entità debole, sempre se il modello permette ad un'entità debole di avere più identifying relationship.
Ragionando sull’esempio precedente la figura a e la figura b hanno significati diversi, perché esistono delle situazioni in cui esistano le 3 relationship binarie tra tre entità, ma però non esiste quella ternaria, quindi l’esistenza di 3 relationship binarie non implica logicamente per forza l’esistenza della relationship ternaria.
Invece, se una relationship binaria può essere “derivata” da una relationship di grado superiore, allora è ridondante.

Figure 3.18
Another example of ternary versus binary relationship types.

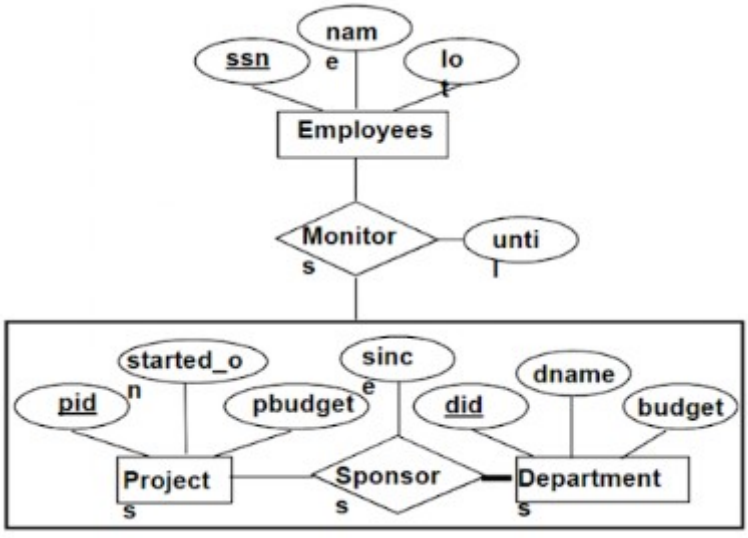


Per esempio in questa figura abbiamo che TAUGHT_DURING è ridondante perché può essere derivata da OFFERS (basandosi sul significato delle relationship).

Specificare i vincoli è complicato e ambiguo quando consideriamo relationship di grado superiore.

Aggregation

Quando abbiamo bisogno di creare una relationship su una relationship già esistente, possiamo usare un aggregation che a livello concettuale equivale a far diventare una relationship set una entity set, così da poterla far partecipare ad un'altra relationship.



Modellazione concettuale EER

La modellazione EER (ovvero Enhanced Entity-Relationship oppure Extended Entity-Relationship) include tutti i concetti della modellazione ER base, e in più aggiunge:

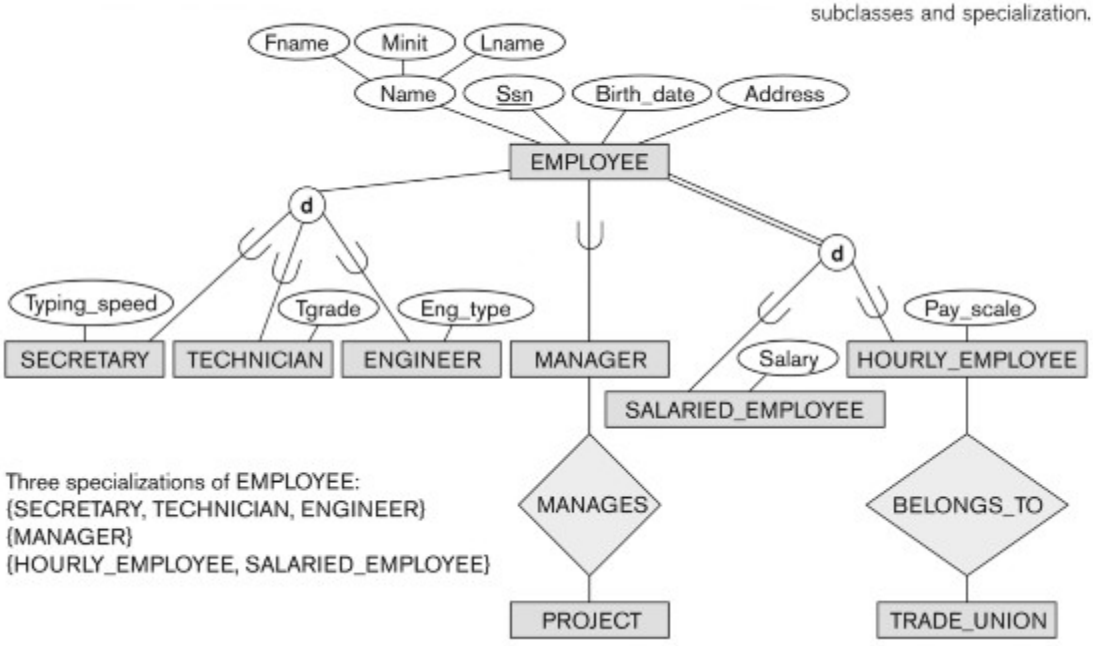
- Sottoclassi e superclassi
- Ereditarietà (attributi e relationship)
- Specializzazioni e generalizzazioni (con dei vincoli)
- Categorie (UNION types)

Questi concetti vengono introdotti per permettere di essere più completi e accurati nella modellazione, ad esempio con concetti Object Oriented (come l'ereditarietà appunto).

Sottoclassi e superclassi

Un entity type può avere ulteriori sottogruppi **significativi** di sue entità, che in alcuni casi devono essere definiti (per aggiunta di attributi o altre caratteristiche).
Ad esempio le entità di EMPLOYEE possono essere sottoragruppate in SECRETARY, ENGINEER, TECHNICIAN oppure in un altra divisione come SALARIED_EMPLOYEE, HOURLY_EMPLOYEE.
I diagrammi EER estendono quelli ER per poter rappresentare questi sottogruppi, chiamati **sottoclassi** (o sottotipi)

Figure 4.1
EER diagram notation to represent subclasses and specialization.



Ognuno di questi sottogruppi è un sottoinsieme delle entità di EMPLOYEE, infatti EMPLOYEE è detta **superclasse** per ognuna di queste **sottoclassi**. Le relationship di questo tipo vengono chiamate **relationship superclasse/sottoclasse** (ad esempio EMPLOYEE/SECRETARY).
Richiamando il fatto dei concetti Object-Oriented, questo tipo di relationship viene anche detta **IS-A relationship** (ad esempio SECRETARY IS-A EMPLOYEE)

Un'entità appartenente ad una sottoclasse rappresenta la stessa entità del mondo reale di alcuni membri della superclasse, infatti un membro di una sottoclasse è la stessa entità della superclasse solo con ruolo diverso.
In un database non può esistere un'entità che è membra solo della sottoclasse, deve essere per forza membra anche delle superclasse.
Un membro di una superclasse può appartenere a quante sottoclassi vuole e non è necessario che appartenga per forza ad almeno una sottoclasse (in assenza di particolari vincoli).
Ad esempio un dipendente salariato che fa l'ingegnere appartiene a due sottoclassi (ENGINEER e SALARIED_EMPLOYEE).

Un'entità che è membra di una sottoclasse **eredita** tutti gli attributi della superclasse e tutte le relationship della superclasse (SECRETARY eredita Fname, Ssn etc da EMPLOYEE)

La **specializzazione** è il processo che definisce un insieme di sottoclassi di una superclasse, e si basa su alcune **caratteristiche distintive** delle entità della superclasse (ad esempio SECRETARY, TECHNICIAN e ENGINEER sono specializzazioni di EMPLOYEE basate sul tipo di lavoro). Abbiamo già visto che ci possono essere diverse specializzazioni per una singola superclasse.

Gli attributi di una sottoclasse sono chiamati **attributi locali** (o specifici) (come ad esempio TypingSpeed in SECRETARY). Le sottoclassi possono anche partecipare in specifiche **relationship types** (ad esempio la relationship BELONGS_TO di HOURLY_EMPLOYEE)

La **generalizzazione** è il processo inverso della specializzazione, infatti è il processo che definisce una superclasse partendo da delle entity type che hanno delle **caratteristiche in comune** (e che andranno poi a formare le sottoclassi) (ad esempio CAR e TRUCK generalizzati in VEHICLE)

Generalization (2)

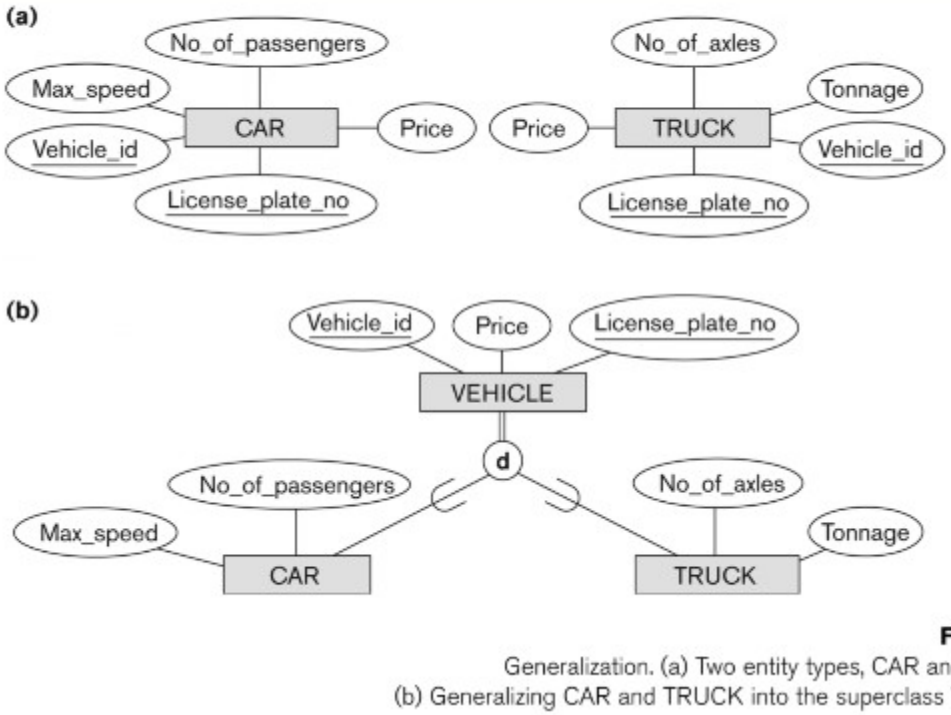


Figure 4.3
Generalization. (a) Two entity types, CAR and TRUCK.
(b) Generalizing CAR and TRUCK into the superclass VEHICLE.

Esistono delle notazioni schematiche nei diagrammi che vengono usate per distinguere tra generalizzazione e specializzazione (una freccia verso la superclasse per denotare una generalizzazione, una freccia verso la sottoclasse per denotare una specializzazione) ma noi non useremo questa notazione perchè è abbastanza soggettiva la scelta in base alla situazione situazione, quindi non avremo nessuna freccia e non faremo distinzione tra le due pratiche.

Una sottoclasse o superclasse rappresenta un gruppo di entità, in particolare rappresenta proprio un **entity type**, infatti in un diagramma EER vengono rappresentate con i rettangoli (proprio come gli entity type).
Chiameremo ogni entity type (e i rispettivi type set) **classi** (sia che siano entity type, che superclassi che sottoclassi).

Vincoli su specializzazioni e generalizzazioni

Ci sono tre tipologie di specializzazioni/generalizzazioni:

1. **Predicate-defined** subclasses (definite da una condizione)
2. **Attribute-defined** specialization (definite da un attributo)
3. **User-defined**

Se possiamo determinare esattamente le entità che faranno parte di ogni sottoclasse in base ad una condizione, le sottoclassi vengono chiamate **predicate-defined**. La condizione quindi è un vincolo che determina i membri di una sottoclasse. In questo caso, per denotarla, scriveremo la

condizione affianco alla linea che congiunge la sottoclasse con la superclasse (ad esempio job-type, age)

Se tutte le sottoclassi in una specializzazione hanno la condizione di appartenenza su uno stesso attributo (appartenente alla superclasse ovviamente), la specializzazione è detta **attribute-defined**. L'attributo su cui si basa la condizione è detto **defining attribute** (attributo di definizione) (ad esempio JobType è il defining attribute della specializzazione JobType)

Se nessuna condizione determina l'appartenenza, la sottoclasse è detta **user-defined**. L'appartenenza è definita dagli utenti del database che la devono aggiungere alla sottoclasse manualmente, ed è specificata individualmente per ogni entità appartenente alla superclasse.

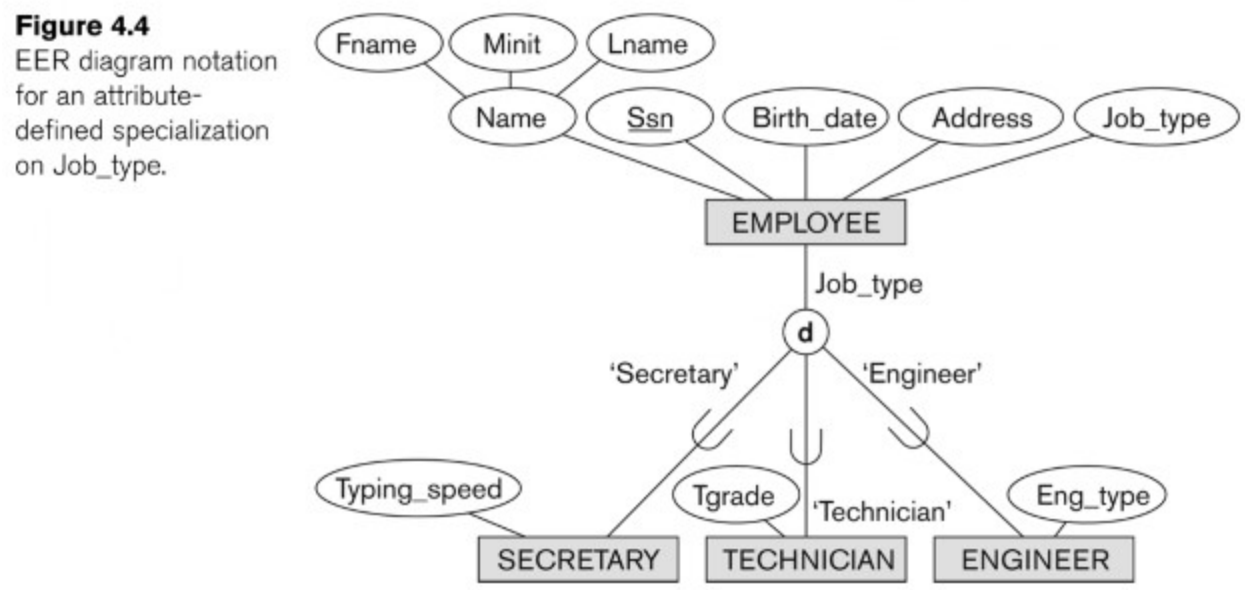


Figure 4.4
EER diagram notation for an attribute-defined specialization on Job_type.

Ci sono poi due vincoli fondamentali che possono essere applicati a una specializzazione/generalizzazione:

- **Disjointness constraint**
- **Completeness constraint**

Quando abbiamo un **vincolo di disgiunzione** (o Disjointness constraint), tutte le sottoclassi di una specializzazione devono essere **disgiunte**, ovvero un'entità può essere membra al massimo di una sottoclasse di specializzazione. Questo nei diagrammi EER viene specificato da una **d**.
Se la specializzazione non è disgiunta si dice **sovrapposta** (overlapping), in questo caso un'entità può appartenere a più sottoclassi della specializzazione. Questo nei diagrammi EER viene specificato da una **o**.

Quando abbiamo un **vincolo di completezza** o esaustività (Completeness or Exhaustiveness constraint):

- **totale**
  Ogni entità della superclasse deve appartenere ad almeno una sottoclasse della specializzazione/generalizzazione (come nei diagrammi ER viene rappresentato da una doppia linea)
- **parziale**
  Le entità possono non fare parte di nessuna sottoclasse (come nei diagrammi ER viene rappresentato da una linea singola)

Di conseguenza possiamo ricavare quattro tipi di specializzazioni/generalizzazioni:
- Disgiunta totale
- Disgiunta parziale
- Sovrapposta totale
- Sovrapposta parziale

Solitamente le generalizzazioni sono disgiunte totali in quanto la superclasse deriva da sottoclassi separate inizialmente.

Example of disjoint partial Specialization

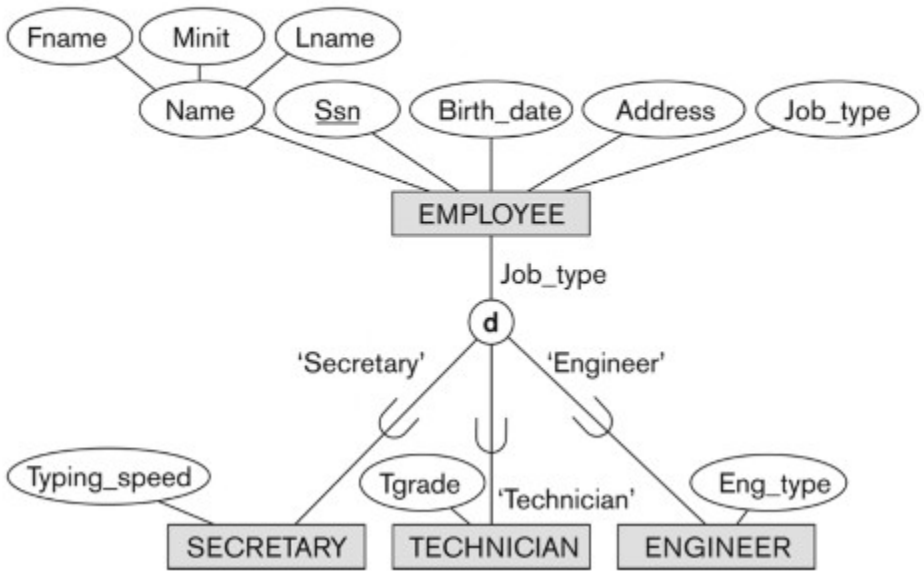


Figure 4.4
EER diagram notation for an attribute-defined specialization on Job_type.

Example of overlapping total Specialization

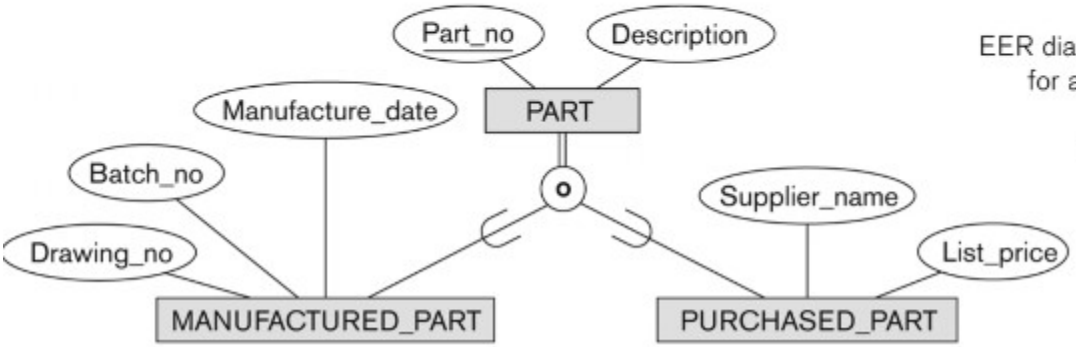


Figure 4.5
EER diagram notation for an overlapping (nondisjoint) specialization.

Una sottoclasse può a sua volta avere ulteriori sottoclassi (diventando la superclasse per quelle sottoclassi), e questo va a formare una gerarchia oppure un reticolo. Ovviamente in questa situazione una sottoclasse non va a ereditare solo attributi solo della sua superclasse, ma anche della superclasse della superclasse e così via.
Se usiamo delle specializzazioni, ovvero partiamo da una entity type e definiamo le sottoclassi per ogni classe, abbiamo un approccio **top down**.
Se invece usiamo delle generalizzazioni, ovvero partiamo da tante entity type e uniamo quelle con cose in comune per ogni classe, abbiamo un approccio **bottom up**.
Nella pratica è comune utilizzare un approccio ibrido.

Una **gerarchia** (hierarchy) ha il vincolo che ogni sottoclasse ha al massimo una superclasse (chiamata anche **ereditarietà singola**) e va a formare una struttura ad **albero**.

In un **reticolo** (lattice) una sottoclasse può essere una sottoclasse per più superclassi (chiamata anche **ereditarietà multipla**), e questo va a formare una un **grafo**.
Una sottoclasse con più di una superclasse è detta **sottoclasse condivisa** (shared)

Shared Subclass “Engineering_Manager”

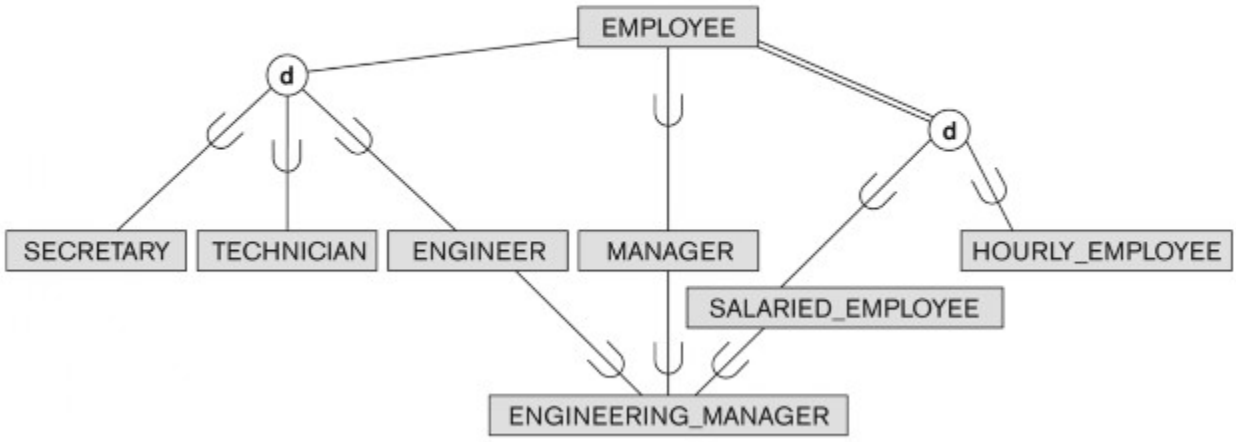


Figure 4.6
A specialization lattice with shared subclass ENGINEERING_MANAGER.

Specialization / Generalization Lattice Example (UNIVERSITY)

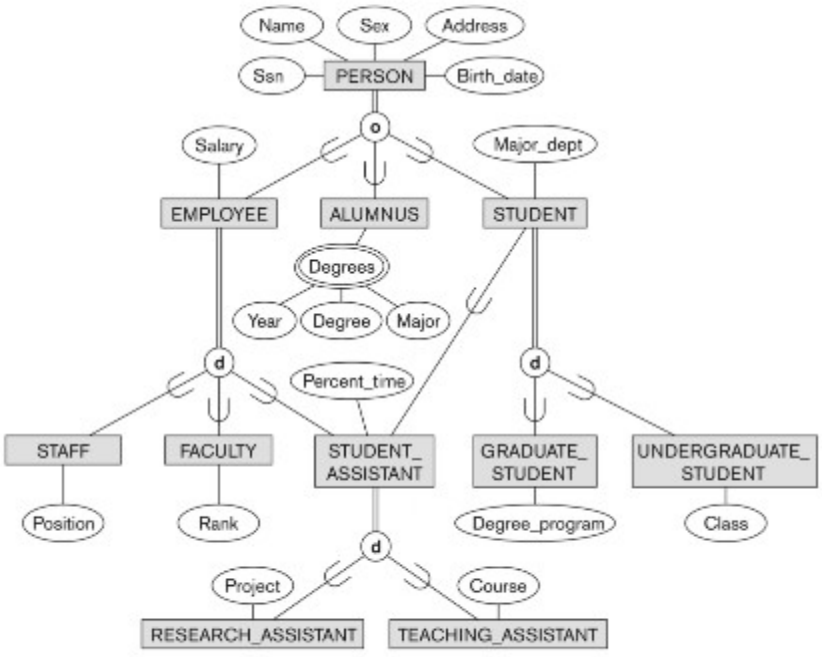


Figure 4.7
A specialization lattice with multiple inheritance for a UNIVERSITY database.

Union types

Tutte le **relationship** sottoclassi/superclassi che abbiamo visto finora hanno una **singola** superclasse (le sottoclassi condivise sono diverse in quanto non c'è una singola relationship che si divide ma son relationship diverse).
In certe situazioni dobbiamo progettare una classe che ha una relationship con **più di una superclasse**.
In questo caso la sottoclasse viene chiamata **categoria** oppure **UNION TYPE** (ad esempio il proprietario di un auto, che sarà una categoria chiamata OWNER, viene creata come l'unione di COMPANY, BANK e PERSON, la categoria OWNER rappresenta un sottoinsieme dell'unione delle tre superclassi).
Un membro di una categoria deve esistere in **almeno una** (tipicamente solo una) delle sue superclassi.
La differenza con le sottoclassi condivise è che una sottoclasse condivisa è un intersezione delle superclassi, quindi il membro **deve esistere in tutte** le superclassi. Invece in una categoria non c'è questo vincolo.

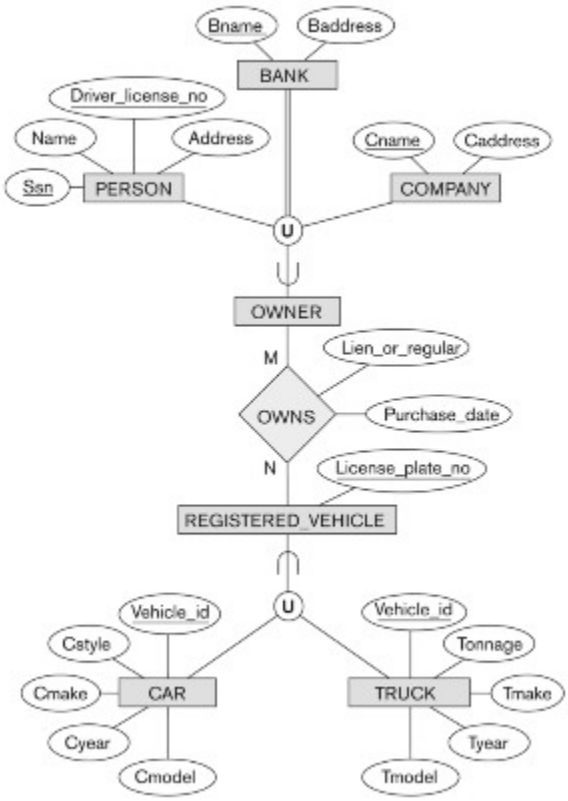


Figure 4.8
Two categories (union types): OWNER and REGISTERED_VEHICLE.

Definizioni formali del modello EER

Una **classe** C è un tipo di entità (può essere un entity type, una sottoclasse, una superclasse o una categoria) con un corrispettivo entity set. (La definizione di relationship type nel modello EER dovrebbe avere il concetto di entity type rimpiazzato dal concetto di classe)
Una **sottoclasse** S è una classe che:

- Eredita tutti gli attributi e le relationship da un'altra classe C

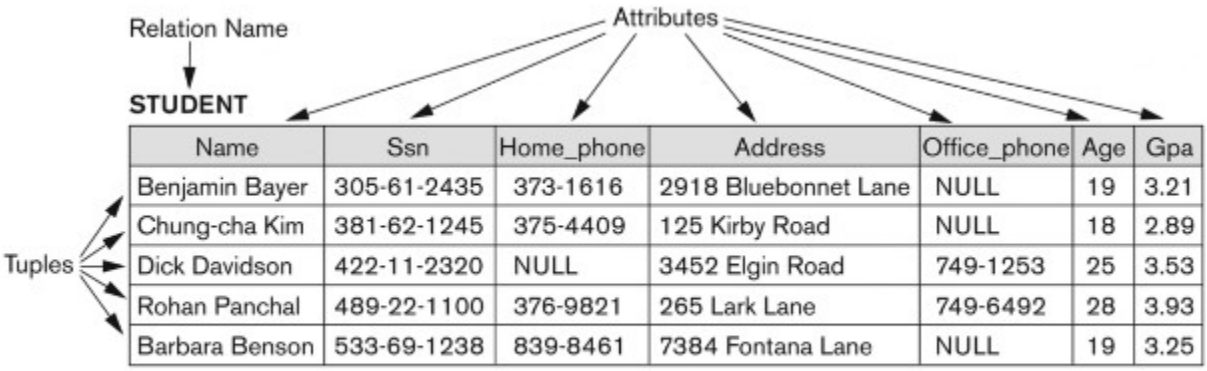
- Il suo entity set è un sottoinsieme dell'entity set di C (quindi $S \subseteq C$)
- C è detta superclasse di S
- Esiste una relationship superclasse/sottoclasse tra C ed S

Una **specializzazione** $Z = \{S_1, \ldots, S_n\}$, è un insieme di sottoclassi con la stessa superclasse G, di conseguenza G / S_i è una relationship superclasse/sottoclasse per ogni $i = 1, \ldots, n$.
G è detta **generalizzazione** delle sottoclassi $\{S_1, \ldots, S_n\}$.
Z è **totale** se l'unione di tutte le sottoclassi forma G, altrimenti è detta **parziale**.
Z è **disgiunta** se abbiamo che le intersezioni delle sottoclassi sono sempre vuote (ovvero $S_i \cap S_j = \varnothing \quad \forall\, i \neq j$, altrimenti è detta **sovrapposta**
Una sottoclasse S di C è detta **predicate defined** se l'appartenenza alla sottoclasse è dipendente da un predicato p sugli attributi di C ($S = C[p]$ dove $C[p]$ è l'insieme delle entità di C che rispettano p).
La sottoclasse è detta **attribute defined** se il predicato (la condizione) si basa su il confrontare lo stesso attributo di tutte le entità con una costante per determinare l'appartenenza ad una sottoclasse.
Una sottoclasse non definita da un predicato è detta **user defined**
Una classe T che è sottoinsieme dell'unione di n superclassi è detta **categoria**. Può avere dei predicati per determinare quali entità fanno parte di T

Modellazione relazionale dei dati

Concetti del modello relazionale

Una **relazione** è un concetto matematico basato sull'idea degli insiemi. Informalmente una relazione è come una **tabella di valori**. Una relazione tipicamente contiene un insieme di righe.
La relazione nel modello relazionale è completamente diversa dalle relationship del modello concettuale, in quanto le relazioni del modello relazionale sono tabelle e rappresentano entrambe (entità e relationship) del modello concettuale.
Gli elementi di ogni riga rappresentano qualcosa che corrisponde a un'**entità** o una **relationship** nel mondo reale (nel modello formale le righe vengono chiamate **tuple**).
Ogni colonna ha un'**intestazione** che indica il significato degli elementi che sono in quella colonna (nel modello formale le intestazioni vengono chiamati **nomi degli attributi** o semplicemente **attributi**).
Ecco un esempio di relazione:



Name	Ssn	Home_phone	Address	Office_phone	Age	Gpa
Benjamin Bayer	305-61-2435	373-1616	2918 Bluebonnet Lane	NULL	19	3.21
Chung-cha Kim	381-62-1245	375-4409	125 Kirby Road	NULL	18	2.89
Dick Davidson	422-11-2320	NULL	3452 Elgin Road	749-1253	25	3.53
Rohan Panchal	489-22-1100	376-9821	265 Lark Lane	749-6492	28	3.93
Barbara Benson	533-69-1238	839-8461	7384 Fontana Lane	NULL	19	3.25

Figure 5.1
The attributes and tuples of a relation STUDENT.

La **chiave** di una relazione è il valore di una colonna o più colonne (di un insieme di attributi) che identificano univocamente quella riga (quella tupla) (nell'esempio il valore di Ssn è la chiave di STUDENT).
In certe situazioni (soprattutto quando non è possibile trovare/avere una chiave) si usano gli identificatori di riga (ovvero numeri sequenziali da 1 a n) come chiavi per identificare la riga, queste chiavi vengono chiamate **chiavi artificiali** oppure **surrogate key**.
Lo **schema** (o la descrizione) di una relazione è denotato da $R(A_1,\ldots,A_n)$, dove R è il **nome** della relazione e A_i sono gli **attributi** della relazione.

- Example:
  CUSTOMER (Cust-id, Cust-name, Address, Phone#)
  - CUSTOMER is the relation name
  - Defined over the four attributes: Cust-id, Cust-name, Address, Phone#

Ogni attributo ha un **dominio** (ovvero l'insieme dei valori "validi") (ad esempio il dominio di Cust-name sono numeri di 6 cifre).
Ci sono due misure per una relazione:

- **Cardinalità**
  Ovvero il numero di righe della relazione
- **Grado** (o arietà)
  Ovvero il numero delle colonne della relazione

Solitamente la cardinalità è più grande rispetto al grado.

Name	Ssn	Home_phone	Address	Office_phone	Age	Gpa
Benjamin Bayer	305-61-2435	373-1616	2918 Bluebonnet Lane	NULL	19	3.21
Chung-cha Kim	381-62-1245	375-4409	125 Kirby Road	NULL	18	2.89
Dick Davidson	422-11-2320	NULL	3452 Elgin Road	749-1253	25	3.53
Rohan Panchal	489-22-1100	376-9821	265 Lark Lane	749-6492	28	3.93
Barbara Benson	533-69-1238	839-8461	7384 Fontana Lane	NULL	19	3.25

- Cardinalità = 5, grado = 6

Una **tupla** (ovvero una riga) è un insieme ordinato di valori, racchiusi tra parentesi angolari <…>. Ogni valore viene fuori da un dominio appropriato.

- A row in the CUSTOMER relation is a 4-tuple and would consist of four values, for example:
  - <632895, "John Smith", "101 Main St. Atlanta, GA 30332", "(404) 894-2000">
  - This is called a 4-tuple as it has 4 values
  - A tuple (row) in the CUSTOMER relation.

Una relazione è un insieme di queste tuple (righe).
Il **dominio** ha una definizione logica (ad esempio “NUMERO_DI_TELEFONO_USA” avrà come dominio l’insieme dei numeri a 10 cifre validi come numeri di telefono in USA).
Il dominio ha anche un **tipo** (data type) o un **formato** definito per esso (la data è GG/MM/AAAA, “NUMERO_DI_TELEFONO_USA” è (ddd)ddd-dddd dove d è una cifra decimale).
Il nome di un attributo si usa per “capire” il ruolo del dominio in una relazione, ovvero per interpretare il significato dei dati corrispondenti a quell’attributo (ad esempio il dominio Data usato per “Data_acquisto” e “Data_consegna”, che prendono due significati diversi grazie al nome).

Lo **stato di una relazione** è un sottoinsieme del prodotto cartesiano dei domini degli attributi (dove ogni dominio contiene l’insieme di tutti i possibili valori che l’attributo può avere) e lo indicheremo con $r(R)$ se R è il nome della relazione.

- Formally,
  - Given R(A1, A2, ..., An)
  - r(R) ⊂ dom (A1) X dom (A2) X ....X dom(An)
- R(A1, A2, …, An) is the **schema** of the relation
- R is the **name** of the relation
- A1, A2, …, An are the **attributes** of the relation
- r(R): a specific **state** (or "value" or “population”) of relation R – this is a *set of tuples* (rows)
  - r(R) = {t1, t2, …, tn} where each ti is an n-tuple
  - ti = <v1, v2, …, vn> where each vj is an *element-of* dom(Aj)

Vediamo un esempio:

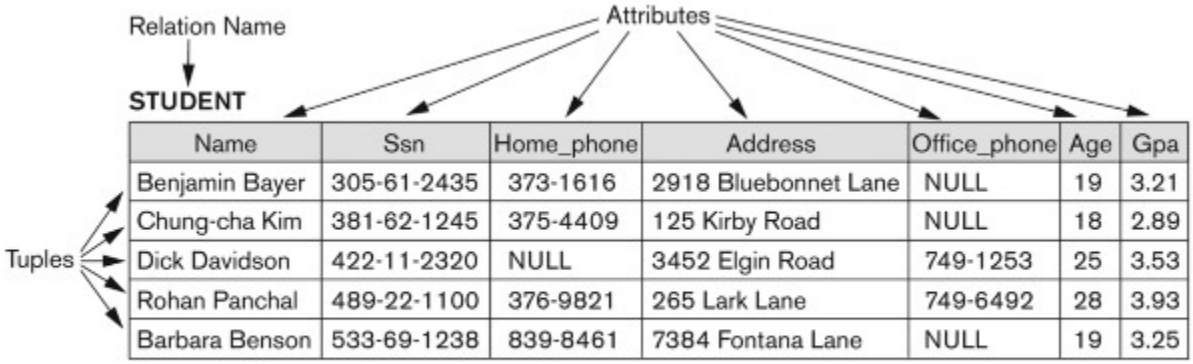
- Let R(A1, A2) be a relation schema:
  - Let dom(A1) = {0,1}
  - Let dom(A2) = {a,b,c}
- Then: dom(A1) X dom(A2) is all possible combinations:
  {<0,a> , <0,b> , <0,c>, <1,a>, <1,b>, <1,c> }

- The relation state r(R) ⊂ dom(A1) X dom(A2)
- For example: r(R) could be {<0,a> , <0,b> , <1,c> }
  - this is one possible state (or “population” or “extension”) r of the relation R, defined over A1 and A2.
  - It has three 2-tuples: <0,a> , <0,b> , <1,c>

Definition Summary

Informal Terms		Formal Terms
Table		Relation
Column Header		Attribute
All possible Column Values		Domain
Row		Tuple
Table Definition		Schema of a Relation
Populated Table		State of the Relation

Ecco un esempio di relazione:



Name	Ssn	Home_phone	Address	Office_phone	Age	Gpa
Benjamin Bayer	305-61-2435	373-1616	2918 Bluebonnet Lane	NULL	19	3.21
Chung-cha Kim	381-62-1245	375-4409	125 Kirby Road	NULL	18	2.89
Dick Davidson	422-11-2320	NULL	3452 Elgin Road	749-1253	25	3.53
Rohan Panchal	489-22-1100	376-9821	265 Lark Lane	749-6492	28	3.93
Barbara Benson	533-69-1238	839-8461	7384 Fontana Lane	NULL	19	3.25

Figure 5.1
The attributes and tuples of a relation STUDENT.

Caratteristiche delle relazioni

Le **tuple non** sono considerate **ordinate** (nel senso che potrebbero esserlo ma quasi sempre non lo sono), anche se appaiono in forma tabellare.
L'**ordine** degli **attributi non conta**, ovvero possiamo invertire le colonne (a patto di invertire anche i nomi degli attributi). Ovviamente l'ordine conta una volta definita la relazione con un certo ordine (anche se la maggior parte delle volte ci si riferisce alle colonne per nome e non per indice).

Tutti i **valori** di una tupla sono considerati **atomici** (ovvero indivisibili). Ogni valore di una tupla deve appartenere al dominio dell'attributo per quella colonna. Un valore speciale **null** viene usato quando non si sa il valore, oppure non è disponibile oppure inapplicabile ad una certa tupla.
Per riferirci ai componenti di una tupla useremo la seguente notazione $t[A_i]$ oppure $t.A_i$, e significa che voglio accedere all'attributo A_i della tupla t, posso riferirmi anche ad una sotto tupla con la notazione $t[A_i, A_j, \ldots, A_k]$, ovvero selezionando un sottoinsieme di attributi.

Vincoli

I vincoli determinano quali valori sono concessi e quali no in un database, e esistono tre tipi principali:

- **Inherent or Implicit Constraints** (impliciti)
  Sono basati dal modello dei dati (ad esempio il modello relazionale non permette una lista come valore di un attributo)
- **Schema-based or Explicit Constraints**
  Sono rappresentati in base alle notazioni dei modelli (ad esempio la cardinalità massima in un ER)
- **Application based or semantic Constraints**
  Sono oltre al potere espressivo del modello, ovvero non si possono rappresentare, e quindi devono essere specificati ed implementati dalle applicazioni e programmi che utilizzano il modello.

I vincoli sono delle condizioni che devono valere in **tutti gli stati validi** delle relazioni.
Ora concentriamoci su i vincoli espliciti (Schema-based), che possono essere di tre tipi principali:

- **Vincoli di chiave**
- **Vincoli di integrità delle entità**
- **Vincoli di integrità referenziale**

Un altro vincolo esplicito è il **vincolo di dominio** (ogni valore deve appartenere al dominio di quell'attributo, oppure null, se è permesso per quell'attributo).

Vincoli sugli attributi

Una **superchiave** di una relazione R è un insieme di attributi S_K di R tale che nessuna coppia di tuple in qualsiasi stato valido della relazione può avere S_K uguale (per ogni $t_1, t_2 \in r(R)$ vale $t_1[S_K] \neq t_2[S_K]$).
Una **chiave** è una superchiave "minima", ovvero che una chiave è una superchiave K alla quale quando le viene tolto qualsiasi attributo non è più una superchiave.
Tutte le chiavi vengono dette **chiavi candidate**, una tra esse viene scelta arbitrariamente e diventa **chiave primaria**

- Example: Consider the CAR relation schema:
  - CAR(State, Reg#, SerialNo, Make, Model, Year)
  - CAR has two keys:
    - Key1 = {State, Reg#}
    - Key2 = {SerialNo}
  - Both are also superkeys of CAR
  - {SerialNo, Make} is a superkey but *not* a key.

Una chiave è sempre una superchiave, ma il viceversa non vale.
Qualsiasi insieme di attributi che include una chiave è una superchiave.

Se un'entità ha varie **chiavi candidate**, ne viene scelta una arbitrariamente per formare la **chiave primaria** (l'attributo di chiave primaria viene sottolineato nel modello relazionale), solitamente si prende la chiave che ha meno attributi, ma è soggettivo e dipende dalle situazioni.
Il valore della chiave primaria viene usato per identificare univocamente una tupla in una relazione e viene anche usata per riferirsi alla tupla dall'esterno (da un altra tupla ad esempio).

Integrità delle entità

Gli attributi che vanno a formare la chiave primaria PK di ogni relazione R dello schema S **non possono** avere come valore null in nessuna tupla di $r(R)$. Questo perché i valori della PK vengono usati per identificare le tuple. Quindi $t[PK] \neq null$ per ogni tupla t in $r(R)$.
Se PK è formata da più attributi, nessuno di questi può avere null come valore.
(altri attributi non facenti parte della PK potrebbero comunque avere vincoli sul non consentire il valore null)

Vincoli tra due relazioni

I vincoli che abbiamo visto finora (ovvero vincoli di chiave e di integrità delle entità) sono relativi a una singola relazione.
Il vincolo di **referential integrity** richiede di introdurre la nozione di **schema di base di dati relazionale**, visto che questo vincolo è specificato **tra due relazioni** e serve a mantenere la consistenza tra le tuple delle relazioni coinvolte.

Lo **schema relazionale** è un insieme S di relazioni che appartengono allo stesso database.
S è il nome dello schema del database e vale $S = \{R_1, \ldots, R_n\}$ con l'aggiunta dei vincoli di integrità dello schema. $R_1, \ldots, R_n$ sono i nomi degli schemi delle singole relazioni.

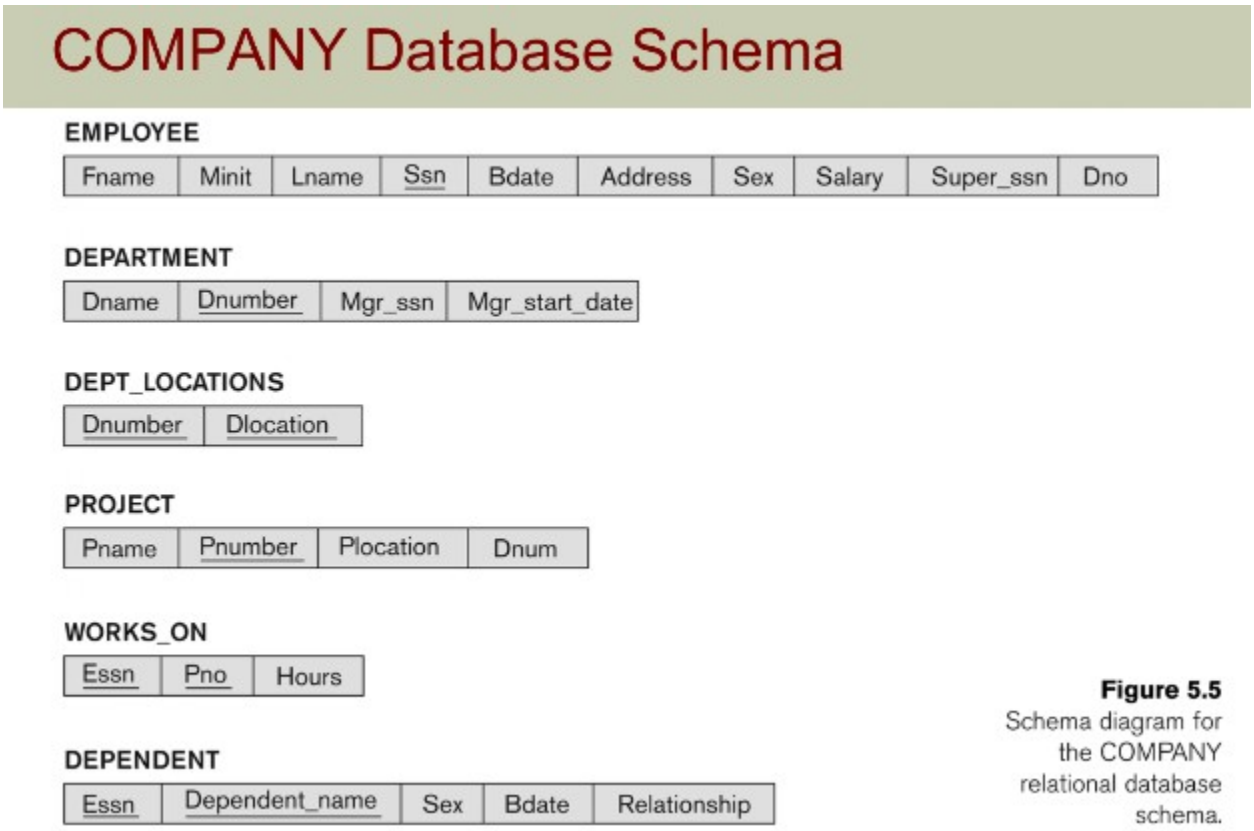


Figure 5.5
Schema diagram for the COMPANY relational database schema.

Uno **stato di uno schema relazionale** DB di S è un insieme di stati di relazioni tale che $DB = \{r_1, \ldots, r_n\}$ tale che ogni r_i è lo stato della relazione R_i e che ogni r_i rispetti i vincoli d'integrità dello schema.
Lo stato di uno schema relazionale in un determinato momento viene anche detto **snapshot** o **istanza**.
Uno stato di uno schema relazionale che non rispetta i vincoli d'integrità è detto **stato non valido**.

Ogni relazione avrà varie tuple nello stato corrente dello schema relazionale.
Lo stato dello schema relazionale è l'unione di tutti gli stati delle relazioni individuali.
Ci sono delle **operazioni base** che si possono fare per cambiare lo stato del database:

- INSERT a new tuple in a relation
- DELETE an existing tuple from a relation
- MODIFY an attribute of an existing tuple

Populated database state for COMPANY

Figure 5.6
One possible database state for the COMPANY relational database schema.

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

Un vincolo di integrità referenziale è usato per specificare una relationship tra due tuple in due differenti relazioni (la "**referencing relation**" e la "**referenced relation**").
Le tuple della referencing relation (relazione referenziante) R1 hanno degli attributi FK (**Foreign Key attributes**) che fanno riferimento alla chiave primaria PK della referenced relation (relazione referenziata) R2.
Una tupla t1 in R1 si dice che **fa riferimento** a una tupla t2 in R2 se $t1[FK] = t2[PK]$.
Nello schema relazionale un vincolo di integrità referenziale si denota come una freccia da $R1.FK$ a $R2.PK$

Una FK di una tupla di una referencing relation R1 può avere come valori:

- Il valore di PK di una tupla della referenced relation R2
- Il valore null

Se è permesso il secondo caso ovviamente la FK non farà parte della PK della referencing relation (quindi non sarà permesso nel caso di entità deboli). La FK può fare riferimento alla stessa relazione di appartenenza della PK.

Ecco un esempio di FK:

Populated database state for COMPANY

Figure 5.6
One possible database state for the COMPANY relational database schema.

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT

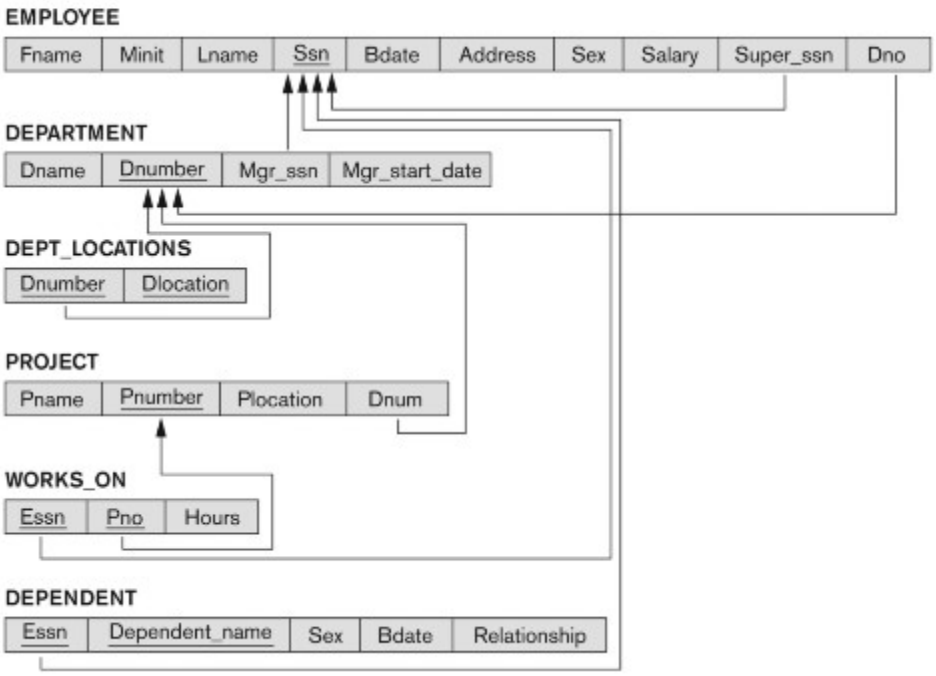
Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

Ogni schema di relazione può venir rappresentato come una riga di nomi degli attributi. Il nome della relazione viene scritta sopra i nomi degli attributi. La chiave primaria viene sottolineata. Le foreign key vengono rappresentate con delle frecce che partono dagli attributi che fanno parte della foreign key della referencing relation e arrivano alla referenced relation (possiamo anche farle arrivare agli attributi che fanno parte della primary key della referenced relation per essere più precisi).

Ecco lo schema relazionale con i vincoli di integrità referenziale del nostro esempio dell’azienda:

Referential Integrity Constraints for COMPANY database

Figure 5.7
Referential integrity constraints displayed on the COMPANY relational database schema.



Altri tipi di vincolo

Altri tipi di vincoli sono quelli di semantica, ovvero basati sulla semantica dell'applicazione e non possono essere espressi dal modello relazionale (come il massimo di numero di ore per settimana di un dipendente).
Dei linguaggi di specifica dei vincoli permettono di esprimere questi vincoli (in SQL abbiamo CREATE TRIGGER, inserire delle operazioni da fare quando viene fatta un insert/delete/modify, e CREATE ASSERTION, dei controlli in fase di creazione di tabella).
Gli altri vincoli che abbiamo visto (chiavi, possibilità di valori null, chiavi candidate, foreign keys, vincoli di referenziazione) vengono espressi dal comando CREATE TABLE in SQL.
Le operazioni di aggiornamento che si possono fare su una relazione sono:

- INSERT a tuple
- DELETE a tuple
- MODIFY a tuple

I vincoli di integrità devono essere rispettati da queste operazioni di aggiornamento. Gli aggiornamenti potrebbero dover venire **propagati** per eseguire altri aggiornamenti in maniera automatica (necessari per mantenere vincoli di integrità).
Nel caso di una violazione di un vincolo d'integrità, diverse azioni possono essere esegurite:

- Cancellare l'operazione di aggiornamento che ha causato la violazione (RESTRICT o REJECT option)

- Eseguire l'operazione di aggiornamento ma notificare l'utente della violazione
- Far partire ulteriori operazioni così che la violazione venga corretta (CASCADE, SET NULL o SET DEFAULT option)
- Eseguire una procedura di correzione dell'errore automatica specificata dall'utente

Un'operazione di **insert** può violare:
- Vincoli di dominio
  Quando un valore di un attributo della nuova tupla non fa parte del dominio di quell'attributo
- Vincoli di chiave
  Quando il valore della chiave della nuova tupla esiste già nella relazione per un'altra tupla
- Integrità referenziale
  Quando una FK della nuova tupla si riferisce ad una PK che non appartiene a nessuna tupla nella referenced relation
- Integrità delle entità
  Se la PK è null nella nuova tupla

Un'operazione di **delete** può violare solo l'integrità referenziale:
- Se la PK della tupla eliminate è "riferita" da delle FK in altre relazioni nel DB
  In questo caso si possono eseguire 3 operazioni (l'operazione da fare viene scelta in fase di progettazione del database):
  - RESTRICT
    "Rifiutare" l'operazione
  - CASCADE
    Propagare il nuovo valore della PK alle FK che si riferiscono a quella tupla (questa viene usata nelle operazioni di update, non di delete)
  - SET NULL (o SET DEFAULT)
    Settare a null le FK (non si può fare nel caso in cui la FK faccia parte della PK)

Un'operazione di **update** può violare:
- Aggiornamento della PK (simile a una DELETE seguita da una INSERT)
  Bisogna specificare le opzioni come nella DELETE
- Aggiornamento della FK
  Si potrebbero violare vincoli di integrità referenziale
- Aggiornamento di un attributo normale
  Violare il dominio per quel attributo, oppure metterlo a null quando c'è un vincolo NOT NULL

Algebra relazionale

L'algebra relazionale è l'insieme base delle operazioni per il un modello relazionale.
Queste operazioni permettono all'utente di eseguire delle specifiche richieste di recupero (chiamate anche **query**).

Il risultato di una di queste operazioni è una **nuova relazione**, che viene costruita partendo da una (o più) relazioni date in input all'operazione. Questo rende l'algebra relazionale **chiusa** (in quanto tutti gli oggetti delle operazioni, ovvero input e risultato, sono relazioni).
Le nuove relazioni che vengono create da un'operazione possono essere manipolate ulteriormente da altre operazioni dell'algebra relazionale. Questa sequenza di operazioni che si forma viene chiamata **espressione** di algebra relazionale.

L'algebra relazionale consiste in più gruppi di operazioni:

- Operazioni di algebra relazionale **unarie**
  - SELECT (σ: sigma)
  - PROJECT (π: pi)
  - RENAME (ρ: rho)
- Operazioni di algebra relazionale **derivate dalla teoria degli insiemi**
  - UNION ($\cup$)
  - INTERSECTION ($\cap$)
  - DIFFERENCE or MINUS (-)
  - CARTESIAN PRODUCT ($\times$)
- Operazioni di algebra relazionale **binarie**
  - JOIN
  - DIVISION
- Operazioni di algebra relazionale **addizionali**
  - OUTER JOIN
  - OUTER UNION
  - AGGREGATE FUNCTIONS (un resoconto delle informazioni)
    - SUM
    - COUNT
    - AVG
    - MIN
    - MAX
    - …

Tutti gli esempi che useremo per spiegare le varie operazioni fanno riferimento al seguente schema relazionale:

- All examples discussed below refer to the COMPANY database shown here.

Figure 5.7
Referential integrity constraints displayed on the COMPANY relational database schema.

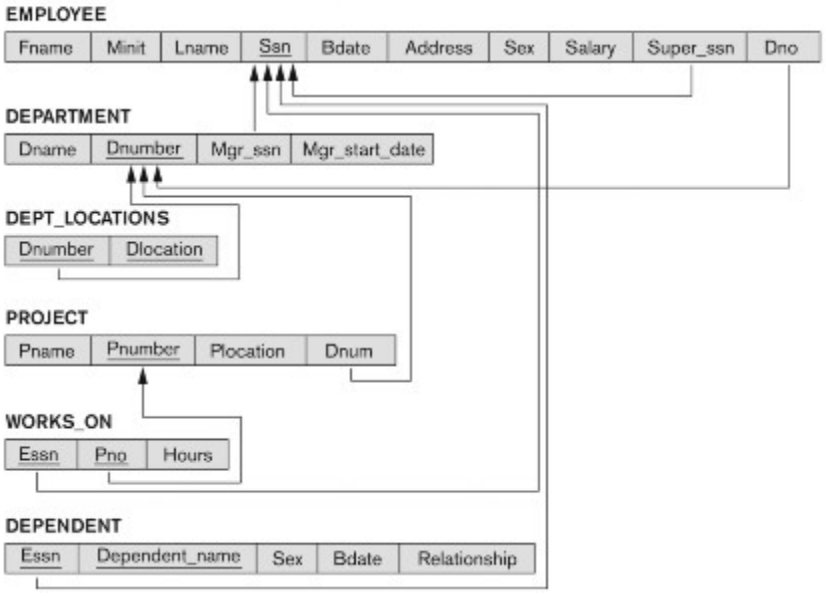


Figure 5.6
One possible database state for the COMPANY relational database schema.

EMPLOYEE

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	Dnumber	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

WORKS_ON

Essn	Pno	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	30.0
987654321	20	15.0
888665555	20	NULL

PROJECT

Pname	Pnumber	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT

Essn	Dependent_name	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

Operazioni unarie

SELECT

L'operazione SELECT, denotata da sigma σ, è usata per selezionare un **sottoinsieme di tuple** da una relazione sulla base di una **condizione di selezione**. La selezione di condizione agisce come un **filtro**, la relazione risultato comprenderà solo le tuple che soddisfano la condizione. Le altre tuple vengono "scartate".

- Examples:
  - Select the EMPLOYEE tuples whose department number is 4:

$$\sigma_{DNO = 4}(EMPLOYEE)$$

  - Select the employee tuples whose salary is greater than $30,000:

$$\sigma_{SALARY > 30,000}(EMPLOYEE)$$

Elenchiamo di seguito le proprietà dell'operazione SELECT:

- L'operazione SELECT $\sigma_{<select\ \ condition>}(R)$ produce una relazione S che ha lo stesso schema (stessi attributi) di R
- L'operazione SELECT è commutativa:
  $\sigma_{<condition1>}(\sigma_{<condition2>}(R)) = \sigma_{<condition2>}(\sigma_{<condition1>}(R))$
  Di conseguenza una "cascata" di operazioni SELECT può essere applicata in qualsiasi ordine
- Una cascata di operazioni SELECT può essere rimpiazzata da una singola selezione con la congiunzione di tutte le condizioni (ovvero con l'AND logico)
  $\sigma_{<condition1>}(\sigma_{<condition2>}(\sigma_{<condition3>}(R))) = \sigma_{<condition1> \wedge <condition2> \wedge <condition3>}(R)$
- Il numero di tuple di un risultato di un'operazione SELECT è **minore o uguale** al numero di tuple della relazione in input R

Selezione: esempi (1)

Esami

Matricola	CodCorso	Voto	Lode
29323	483	28	NO
39654	729	30	SÌ
29323	913	26	NO
35467	913	30	NO
31283	729	30	NO

$\sigma_{(Voto = 30)\ AND\ (Lode = NO)}(Esami)$

Matricola	CodCorso	Voto	Lode
35467	913	30	NO
31283	729	30	NO

$\sigma_{(CodCorso = 729)\ OR\ (Voto = 30)}(Esami)$

Matricola	CodCorso	Voto	Lode
39654	729	30	SÌ
35467	913	30	NO
31283	729	30	NO

Selezione: esempi (2)

Partite

Giornata	Casa	Ospite	GolCasa	GolOspite
4	Venezia	Bologna	0	1
5	Brescia	Atalanta	3	3
5	Inter	Bologna	1	0
5	Lazio	Parma	0	0

$\sigma_{(Giornata = 5)\ AND\ (GolCasa = GolOspite)}(Partite)$

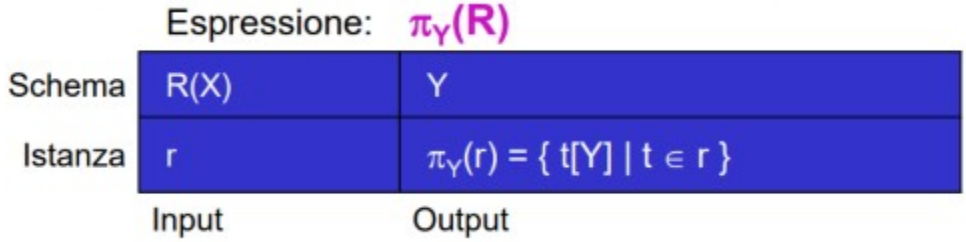
Giornata	Casa	Ospite	GolCasa	GolOspite
5	Brescia	Atalanta	3	3
5	Lazio	Parma	0	0

$\sigma_{(Ospite = Bologna)\ AND\ (GolCasa < GolOspite)}(Partite)$

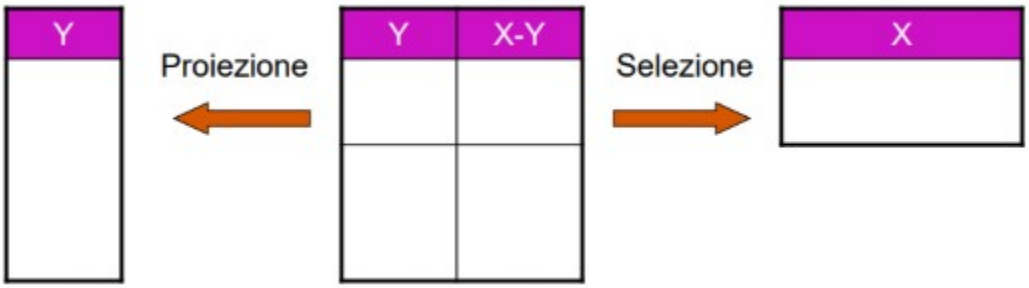
Giornata	Casa	Ospite	GolCasa	GolOspite
4	Venezia	Bologna	0	1

PROJECT

L'operazione PROJECT (ovvero **proiezione**), denotata da pi π, è ortogonale all'operazione di selezione, in quanto permette di selezionare un sottoinsieme Y degli **attributi di una relazione**. In sintesi mantiene certe colonne di una relazione e "scarta" le altre.



Possiamo vedere la selezione come una partizione orizzontale (in quanto seleziona righe) e la proiezione come una partizione verticale (in quanto seleziona colonne).



- Example: To list each employee's first and last name and salary, the following is used:

$$\pi_{LNAME,\ FNAME,SALARY}(EMPLOYEE)$$

Elenchiamo di seguito le proprietà dell'operazione PROJECT:

- Il formato di un'operazione PROJECT è: $\pi_{<lista\ \ di\ \ attributi>}(R)$
  Dove la lista degli attributi sono gli attributi di R che ci interessano nella proiezione.
- L'operazione PROJECT rimuove **tutte le tuple duplicate**, questo perché il risultato è una relazione, ovvero un **insieme** di tuple, e un insieme non ammette duplicati. Ovviamente se nella lista di attributi includiamo anche la chiave (quindi la lista di attributi è una superchiave) non verrà eliminata nessuna tupla (per le proprietà della chiave di una relazione).
- Anche in questo caso il numero di tuple della relazione risultato di una PROJECT sarà minore o uguale al numero di tuple dell'input R (nel caso in cui negli attributi ci sia la **chiave**, ovvero la lista degli attributi è una superchiave, il numero di tuple sarà sicuramente **uguale**)
- L'operazione PROJECT **non è commutativa**.
  Però nel caso in cui la seconda lista di attributi sia un sottoinsieme della prima, il risultato è lo stesso che applicare la PROJECT solo con la seconda lista di attributi.

Viene chiamata proiezione perché nel caso in cui la lista degli attributi non sia una superchiave, certe tuple potrebbero "collassare" in una singola tupla.

Proiezione: esempi (1)

Corsi

CodCorso	Titolo	Docente	Anno
483	Analisi	Biondi	1
729	Analisi	Neri	1
913	Sistemi Informativi	Castani	2

$\pi_{CodCorso,Docente}(Corsi)$

CodCorso	Docente
483	Biondi
729	Neri
913	Castani

$\pi_{CodCorso,Anno}(Corsi)$

CodCorso	Anno
483	1
729	1
913	2

Proiezione: esempi (2)

Corsi

CodCorso	Titolo	Docente	Anno
483	Analisi	Biondi	1
729	Analisi	Neri	1
913	Sistemi Informativi	Castani	2

$\pi_{Titolo}(Corsi)$

Titolo
Analisi
Sistemi Informativi

$\pi_{Docente}(Corsi)$

Docente
Biondi
Neri
Castani

Espressioni

In certe situazioni potremmo voler applicare più operazioni di algebra relazionale, una dopo l'altra, e possiamo "gestirle" in due modi:

- Possiamo scrivere le operazioni come un'unica **espressione di algebra relazionale**, annidando le operazioni
- Possiamo applicare un'operazione alla volta e creare dei **risultati intermedi**, a cui **dobbiamo** dare dei nomi per poterceli "salvare".

Bisogna fare sempre **prima la selezione della proiezione**, altrimenti rischiamo di selezionare in base ad un attributo che non esiste più.
Ad esempio:

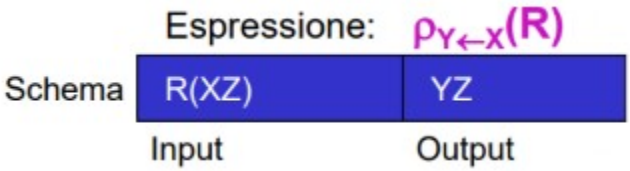
- To retrieve the first name, last name, and salary of all employees who work in department number 5, we must apply a select and a project operation
- We can write a *single relational algebra expression* as follows:
  - $\pi_{\text{FNAME, LNAME, SALARY}}(\sigma_{\text{DNO=5}}(\text{EMPLOYEE}))$
- OR We can explicitly show the *sequence of operations*, giving a name to each intermediate relation:
  - DEP5_EMPS $\leftarrow \sigma_{\text{DNO=5}}(\text{EMPLOYEE})$
  - RESULT $\leftarrow \pi_{\text{FNAME, LNAME, SALARY}}$ (DEP5_EMPS)

RENAME

L'operazione RENAME (di ridenominazione), denotato da rho ρ, modifica lo schema di una relazione **cambiando il nome della tabella e/o i nomi di uno o più attributi**.
Quindi facendo:

$$\rho_{Y \leftarrow X}(r) \text{ con } r \text{ su } R(XZ) \text{ cambia lo schema in } R(YZ)$$

Questa operazione lascia invariati i valori delle tuple. Nel caso in cui si cambi il nome di più attributi bisogna fare attenzione all'ordine, perché la ridenominazione viene fatta in base a quello.
Questa operazione è utile se una relazione richiede usi diversi (ad esempio in base all'uso possiamo rinominare gli attributi come ci è più comodo) è in certe situazioni è necessario (come vedremo nell'operazione JOIN).



L'operatore RENAME può essere utilizzato nei seguenti modi:

- $\rho_{S(B_1,\ldots,B_n)}(R)$ per cambiare **entrambi**.
  Per cambiare il nome della relazione da R in S e i nomi degli attributi in $B_1,\ldots,B_n$
- $\rho_S(R)$ per cambiare **il nome della relazione**.
  Per cambiare il nome della relazione da R in S
- $\rho_{(B_1,\ldots,B_n)}(R)$ per cambiare **il nome degli attributi**.
  Per cambiare i nomi degli attributi in $B_1,\ldots,B_n$



Ecco qualche esempio:

Redditi

CF	Imponibile
BNCGRG78F21A	10000

$\rho_{CodiceFiscale \leftarrow CF}(Redditi)$

CodiceFiscale	Imponibile
BNCGRG78F21A	10000

VoliNoSmoking

Numero	Giorno
SC278	28/07/2001
SC315	30/07/2001

$\rho_{Codice,Data \leftarrow Numero,Giorno}(VoliNoSmoking)$

Codice	Data
SC278	28/07/2001
SC315	30/07/2001

Figure 8.2 Results of a sequence of operations. (a) $\pi_{Fname, Lname, Salary}(\sigma_{Dno=5}(EMPLOYEE))$.
(b) Using intermediate relations and renaming of attributes.

(a)

Fname	Lname	Salary
John	Smith	30000
Franklin	Wong	40000
Ramesh	Narayan	38000
Joyce	English	25000

(b)
TEMP

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston,TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston,TX	M	40000	888665555	5
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble,TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5

R

First_name	Last_name	Salary
John	Smith	30000
Franklin	Wong	40000
Ramesh	Narayan	38000
Joyce	English	25000

Operazioni insiemistiche

UNION

L'operazione UNION è un'operazione binaria, denotata da $\cup$.
Il risultato di $R \cup S$ è una relazione che include tutte le tuple che sono o in R o in S o in entrambe R e S, con l'eliminazione delle tuple nel caso siano duplicate.
Ovviamente le due relazioni R e S devono essere di **tipo compatibile** (o anche **UNION compatibile**), ovvero se $R(A_1, \ldots, A_n)$ e $S(B_1, \ldots, B_m)$:

- Devono avere lo stesso numero di attributi (ovvero $n = m$)
- Ogni paio di attributi corrispondenti devono essere tipo compatibili (ovvero avere lo stesso dominio / dominio compatibile). Quindi $\forall i \in (1, \ldots, n)$ vale $dom(A_i) = dom(B_i)$

- Example:
  - To retrieve the social security numbers of all employees who either *work in department 5* (RESULT1 below) or *directly supervise an employee who works in department 5* (RESULT2 below)
  - We can use the UNION operation as follows:

$$DEP5_EMPS \leftarrow \sigma_{DNO=5}(EMPLOYEE)$$
$$RESULT1 \leftarrow \pi_{SSN}(DEP5_EMPS)$$
$$RESULT2(SSN) \leftarrow \pi_{SUPERSSN}(DEP5_EMPS)$$
$$RESULT \leftarrow RESULT1 \cup RESULT2$$

  - The union operation produces the tuples that are in either RESULT1 or RESULT2 or both

RESULT1

Ssn
123456789
333445555
666884444
453453453

RESULT2

Ssn
333445555
888665555

RESULT

Ssn
123456789
333445555
666884444
453453453
888665555

La compatibilità di tipo tra gli operandi è richiesta per l'operazione UNION (è richiesta anche per l'operazione INTERSECTION e DIFFERENCE come vedremo dopo)
Il risultato di un'operazione UNION $R_1 \cup R_2$ (anche INTERSECTION e DIFFERENCE) prende il nome degli attributi dal **primo** operando dell'operazione R_1 (per convenzione)

INTERSECTION

L'operazione INTERSECTION è un'operazione binaria, denotata da $\cap$.
Il risultato di $R \cap S$ è una relazione che include tutte le tuple che sono sia in R che in S.
Anche qui i due operatori devono essere tipi compatibili e la relazione risultato avrà gli stessi nomi degli attributi del primo operatore (R in questo caso).

SET DIFFERENCE

L'operazione DIFFERENCE (chiamata anche "meno") è un'operazione binaria, denotata da -.
Il risultato di $R - S$ è una relazione che include tutte le tuple che sono in R, ma che non sono in S .
Anche qui i due operatori devono essere tipi compatibili e la relazione risultato avrà gli stessi nomi degli attributi del primo operatore (R in questo caso).

Ecco un esempio che comprende tutte e tre queste operazioni insiemistiche:

Figure 8.4 The set operations UNION, INTERSECTION, and MINUS. (a) Two union-compatible relations. (b) STUDENT ∪ INSTRUCTOR. (c) STUDENT ∩ INSTRUCTOR. (d) STUDENT − INSTRUCTOR. (e) INSTRUCTOR − STUDENT.

(a) STUDENT

Fn	Ln
Susan	Yao
Ramesh	Shah
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert

INSTRUCTOR

Fname	Lname
John	Smith
Ricardo	Browne
Susan	Yao
Francis	Johnson
Ramesh	Shah

(b)

Fn	Ln
Susan	Yao
Ramesh	Shah
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert
John	Smith
Ricardo	Browne
Francis	Johnson

(c)

Fn	Ln
Susan	Yao
Ramesh	Shah

(d)

Fn	Ln
Johnny	Kohler
Barbara	Jones
Amy	Ford
Jimmy	Wang
Ernest	Gilbert

(e)

Fname	Lname
John	Smith
Ricardo	Browne
Francis	Johnson

Notiamo che UNION e INTERSECTION sono commutative, ovvero vale:

$$R \cup S = S \cup R \text{ e } R \cap S = S \cap R$$

Queste due operazioni possono essere trattate come operazioni n-arie visto che entrambe godono della proprietà associativa:

$$R \cup (S \cup T) = (R \cup S) \cup T \text{ e } R \cap (S \cap T) = (R \cap S) \cap T$$

Invece per definizione DIFFERENCE non è commutativa.

Unione e differenza: esempi

VoliCharter

Codice	Data
XY123	21/07/2001
SC278	28/07/2001
XX338	18/08/2001

VoliNoSmoking

Codice	Data
SC278	28/07/2001
SC315	30/07/2001

VoliCharter ∪ VoliNoSmoking

Codice	Data
XY123	21/07/2001
SC278	28/07/2001
XX338	18/08/2001
SC315	30/07/2001

VoliCharter - VoliNoSmoking

Codice	Data
XY123	21/07/2001
XX338	18/08/2001

VoliNoSmoking - VoliCharter

Codice	Data
SC315	30/07/2001

Cartesian Product

L'operazione CARTESIAN PRODUCT (chiamata anche "CROSS PRODUCT") è un'operazione binaria, denotata da $\times$.

Il risultato di $R \times S$ è una relazione che combina tutte le tuple della prima relazione R con tutte le tuple della seconda relazione S in modo combinatorio.

Quindi se facciamo $R(A_1, \ldots, A_n) \times S(B_1, \ldots, B_m)$ il risultato sarà una relazione di $n+m$ attributi $Q(A_1, \ldots, A_n, B_1, \ldots, B_m)$ con gli attributi in questo ordine.

Se R ha n_R tuple (quindi $|R| = n_R$) e S ha n_S, allora $R \times S = Q$ avrà $n_R * n_S$.

In questo caso i due operandi **non** devono essere per forza di tipo compatibile.

Ecco un esempio:

Corsi

Codice	NomeCorso	Semestre	MatrDocente
M2170	Informatica 1	1	D102
M4880	Sistemi digitali	2	D104
F1401	Elettronica	1	D104
F0410	Basi di dati	2	D102

Docenti

MatrDocente	NomeDoc	Dipartimento
D102	Verdi	Informatica
D105	Neri	Informatica
D104	Bianchi	Elettronica

R

Corsi. Codice	Corsi. NomeCorso	Corsi. Semestre	Corsi. MatrDocente	Docenti. MatrDocente	Docenti. NomeDoc	Docenti. Dipartimento
M2170	Informatica 1	1	D102	D102	Verdi	Informatica
M2170	Informatica 1	1	D102	D105	Neri	Informatica
M2170	Informatica 1	1	D102	D104	Bianchi	Elettronica
M4880	Sistemi digitali	2	D104	D102	Verdi	Informatica
M4880	Sistemi digitali	2	D104	D105	Neri	Informatica
M4880	Sistemi digitali	2	D104	D104	Bianchi	Elettronica
F1401	*Elettronica*	*1*	*D104*	*D102*	*Verdi*	*Informatica*
F1401	*Elettronica*	*1*	*D104*	*D105*	*Neri*	*Informatica*
F1401	*Elettronica*	*1*	*D104*	*D104*	*Bianchi*	*Elettronica*
F0410	*Basi di dati*	*2*	*D102*	*D102*	*Verdi*	*Informatica*
F0410	*Basi di dati*	*2*	*D102*	*D105*	*Neri*	*Informatica*
F0410	*Basi di dati*	*2*	*D102*	*D104*	*Bianchi*	*Elettronica*

In genere il prodotto cartesiano non ha molto senso come operazione (non ha senso avere delle tuple combinate così “casualmente”), però può prendere senso se seguito da altre operazioni. Ad esempio possiamo aggiungere una SELECT per mettere in uguaglianza la foreign key con la primary key dell’altra tabella ad esempio (nell’esempio precedente ha senso fare una SELECT con condizione $Corsi.MatrDocente = Docenti.MatrDocente$).

Operazioni binarie

JOIN

Il prodotto cartesiano seguito da una SELECT è una operazione abbastanza comune per identificare tuple correlate.
L'operazione di JOIN, denotata da , combina queste due operazioni in una sola, questa operazione è molto importante in qualsiasi database con più di una relazione, perché permette di combinare tuple "semanticamente legate" prese da varie relazioni.
La forma generale di un JOIN tra due relazioni $R(A_1,\ldots,A_n)$ e $S(B_1,\ldots,B_m)$ è:

$$R \quad {}_{<join\ condition>} S$$

Se ad esempio facciamo:

$$R \quad {}_{R.A_i=S.B_j} S$$

Il risultato sarà una relazione $Q(A_1,\ldots,A_n,B_1,\ldots,B_m)$ di grado $n+m$, però includerà solo le tuple per cui vale la condizione $r[A_i]=s[B_j]$. Quindi $n_Q \leq n_R * n_S$

- Example: Suppose that we want to retrieve the name of the manager of each department.
  - To get the manager's name, we need to combine each DEPARTMENT tuple with the EMPLOYEE tuple whose SSN value matches the MGRSSN value in the department tuple.
  - We do this by using the join ⋈ operation.
  - DEPT_MGR ← DEPARTMENT $\bowtie_{MGRSSN=SSN}$ EMPLOYEE
- MGRSSN=SSN is the join condition
  - Combines each department record with the employee who manages the department
  - The join condition can also be specified as DEPARTMENT.MGRSSN= EMPLOYEE.SSN

Corsi. Codice	Corsi. NomeCorso	Corsi. Semestre	Corsi. MatrDocente	Docenti. MatrDocente	Docenti. NomeDoc	Docenti. Dipartimento
M2170	*Informatica 1*	*1*	*D102*	*D102*	*Verdi*	*Informatica*
M2170	Informatica 1	1	D102	D105	Neri	Informatica
M2170	Informatica 1	1	D102	D104	Bianchi	Elettronica
M4880	Sistemi digitali	2	D104	D102	Verdi	Informatica
M4880	Sistemi digitali	2	D104	D105	Neri	Informatica
M4880	*Sistemi digitali*	*2*	*D104*	*D104*	*Bianchi*	*Elettronica*
F1401	Elettronica	1	D104	D102	Verdi	Informatica
F1401	Elettronica	1	D104	D105	Neri	Informatica
F1401	*Elettronica*	*1*	*D104*	*D104*	*Bianchi*	*Elettronica*
F0410	*Basi di dati*	*2*	*D102*	*D102*	*Verdi*	*Informatica*
F0410	Basi di dati	2	D102	D105	Neri	Informatica
F0410	Basi di dati	2	D102	D104	Bianchi	Elettronica

R

Corsi. Codice	Corsi. NomeCorso	Corsi. Semestre	Corsi. MatrDocente	Docenti. MatrDocente	Docenti. NomeDoc	Docenti. Dipartimento
M2170	Informatica 1	1	D102	D102	Verdi	Informatica
M4880	Sistemi digitali	2	D104	D104	Bianchi	Elettronica
F1401	Elettronica	1	D104	D104	Bianchi	Elettronica
F0410	Basi di dati	2	D102	D102	Verdi	Informatica

Notiamo che (giustamente) gli elementi della prima relazione che **non hanno collegamenti** con elementi della seconda non compaiono nel risultato del JOIN (vale anche per gli elementi della seconda che non hanno collegamenti con elementi della prima).

Questo JOIN che abbiamo visto (ovvero il caso generale) è detto **Theta-join**, e la condizione di join viene chiamata theta (quindi ad esempio $R.A_i = S.B_j$ è theta).
Theta può essere **qualsiasi operazione booleana** basata sugli attributi di R e S (quindi possiamo creare delle espressioni booleane con AND, OR e NOT).
Il Theta.join è **commutativo** e **associativo**.

Esiste poi **EQUIJOIN**, che consiste in sole operazioni di comparazione.
L'EQUIJOIN è un Theta-join dove viene usato solo l'operatore = (e non maggiore, minore o etc).
Nel risultato di un EQUIJOIN abbiamo sempre una o più coppie di attributi i cui valori sono uguali per ogni tupla. L'esempio di JOIN che abbiamo visto prima è un EQUIJOIN.

Un'altra variazione di JOIN è il **NATURAL JOIN**, denotato da *, che è fatto apposta per evitare la seconda ripetizione dell'attributo della condizione dell'EQUIJOIN, eliminando quella colonna (questo perché non ha senso tenere una coppia di attributi se il valore è identico).

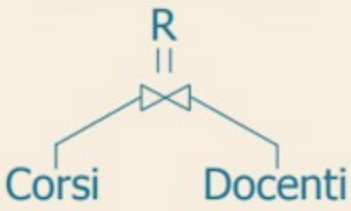
La condizione per poter applicare il NATURAL JOIN è che la coppia di attributi del JOIN (o le coppie) abbiano **lo stesso nome dell'attributo** (se non è così basta applicare un RENAME)

- Example: To apply a natural join on the DNUMBER attributes of DEPARTMENT and DEPT_LOCATIONS, it is sufficient to write:
  - DEPT_LOCS ← DEPARTMENT * DEPT_LOCATIONS
- Only attribute with the same name is DNUMBER
- An implicit join condition is created based on this attribute:
  DEPARTMENT.DNUMBER=DEPT_LOCATIONS.DNUMBER

- Another example: Q ← R(A,B,C,D) * S(C,D,E)
  - The implicit join condition includes *each pair* of attributes with the same name, "AND"ed together:
    - R.C=S.C AND R.D=S.D
  - Result keeps only one attribute of each such pair:
    - Q(A,B,C,D,E)

Trovare le informazioni sui corsi e sui docenti che li tengono

R = Corsi ⋈ Docenti



R

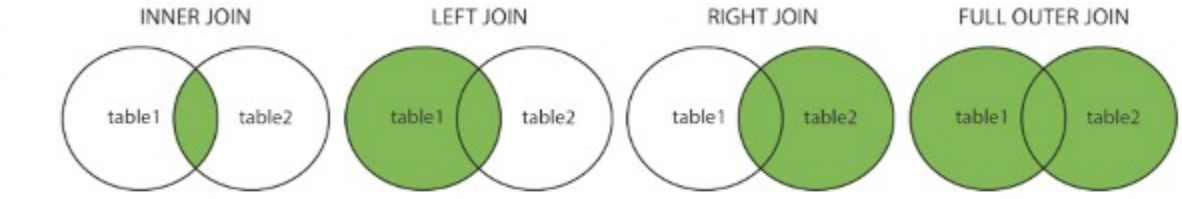
Corsi. Codice	Corsi. NomeCorso	Corsi. Semestre	MatrDocente	Docenti. NomeDoc	Docenti. Diparimento
M2170	Informatica 1	1	D102	Verdi	Informatica
M4880	Sistemi digitali	2	D104	Bianchi	Elettronica
F1401	Elettronica	1	D104	Bianchi	Elettronica
F0410	Basi di dati	2	D102	Verdi	Informatica

NATURAL JOIN e EQUIJOIN sono detti **inner join**, questo perché mostra solo gli elementi interni alla JOIN, ovvero solo le tuple che hanno un collegamento con tuple dell'altra tabella.

Ci sono poi gli **outer join**, dove le tuple che non hanno "collegamenti" con l'altra relazione (ovvero non semanticamente collegate al predicato di join) vengono comunque selezionate, ma hanno i campi rimanenti settati a null (invece nel natural join e nell'equijoin queste tuple venivano scartate). Ne esistono di tre tipi:

- **LEFT OUTER JOIN**, denotato da ⟕ o da <
  Vengono inserite e "completate" tutte le tuple della prima relazione operando

- **RIGHT OUTER JOIN**, denotato da ⟖ o da >
  Vengono inserite e “completate” tutte le tuple della seconda relazione operando
- **FULL OUTER JOIN**, denotato da ⟗ o da < >
  Vengono inserite e “completate” tutte le tuple di entrambe le relazioni



Ecco alcuni esempi:

Ricercatori

Nome	CodProgetto
Rossi	HK27
Bianchi	HK27
Verdi	HK28

Progetti

CodProgetto	Responsabile
HK27	Bianchi
HAL2000	Neri

Ricercatori =▷◁ Progetti

Nome	CodProgetto	Responsabile
Rossi	HK27	Bianchi
Bianchi	HK27	Bianchi
Verdi	HK28	NULL

Ricercatori ▷◁= Progetti

Nome	CodProgetto	Responsabile
Rossi	HK27	Bianchi
Bianchi	HK27	Bianchi
NULL	HAL2000	Neri

Ricercatori =▷◁= Progetti

Nome	CodProgetto	Responsabile
Rossi	HK27	Bianchi
Bianchi	HK27	Bianchi
Verdi	HK28	NULL
NULL	HAL2000	Neri

DIVISION

L’operazione DIVISION è un’operazione binaria, denotata da ÷.
Consideriamo $R(Z) \div S(X)$, dove $X \subset Z$. Definiamo $Y = Z - X$ (e quindi $Z = X \cup Y$).
Quindi Y è l’insieme di attributi di R che non fanno parte di S.
Il risultato della divisione è una relazione $T(Y)$ che include una tupla t se esistono delle tuple t_R che appaiono in R tali che $t_R[Y] = t$ e che $t_R[X] = t_s$ per ogni tupla t_s in S.
In sintesi una tupla t appare nel risultato T se i valori di t appaiono in R in combinazione con ogni elemento di S

Anche qui i due operatori devono essere tipi compatibili e la relazione risultato avrà gli stessi nomi degli attributi del primo operatore (R in questo caso).

Figure 8.8 The DIVISION operation. (a) Dividing SSN_PNOS by SMITH_PNOS. (b) T ← R ÷ S.

(a)

SSN_PNOS

Essn	Pno
123456789	1
123456789	2
666884444	3
453453453	1
453453453	2
333445555	2
333445555	3
333445555	10
333445555	20
999887777	30
999887777	10
987987987	10
987987987	30
987654321	30
987654321	20
888665555	20

SMITH_PNOS

Pno
1
2

SSNS

Ssn
123456789
453453453

(b)

R

A	B
a1	b1
a2	b1
a3	b1
a4	b1
a1	b2
a3	b2
a2	b3
a3	b3
a4	b3
a1	b4
a2	b4
a3	b4

S

A
a1
a2
a3

T

B
b1
b4

OUTER UNION

L'operazione OUTER UNION è stata sviluppata per eseguire l'unione di due relazioni che non sono di tipo compatibile.
Questa operazione prende in input due relazioni che sono **di tipo parzialmente compatibile**, ovvero che solo un sottoinsieme degli attributi delle due relazioni e di tipo compatibile.
Gli attributi che sono tipo compatibile vengono rappresentati una sola volta nel risultato, e gli attributi che non sono tipo compatibile vengono comunque mantenuti nel risultato (sia quelli della prima relazione che quelli della seconda).
Quindi se facciamo l'OUTER UNION di $R(X, Y)$ e $S(X, Z)$ otterremo $T(X, Y, Z)$.

- Example: An outer union can be applied to two relations whose schemas are STUDENT(Name, SSN, Department, Advisor) and INSTRUCTOR(Name, SSN, Department, Rank).
  - Tuples from the two relations are matched based on having the same combination of values of the shared attributes— Name, SSN, Department.
  - If a student is also an instructor, both Advisor and Rank will have a value; otherwise, one of these two attributes will be null.
  - The result relation STUDENT_OR_INSTRUCTOR will have the following attributes:

STUDENT_OR_INSTRUCTOR (Name, SSN, Department, Advisor, Rank)

Riassunto

L’insieme di operazioni composto da SELECT, PROJECT, RENAME, UNION, DIFFERENCE e CARTESIAN PRODUCT è chiamato **insieme completo** perchè tutte le altre operazioni di algebra relazionale possono venir espresse come una combinazione di queste operazioni.

- For example:
  - $R \cap S = (R \cup S) - ((R - S) \cup (S - R))$
  - $R \bowtie_{\text{<join condition>}} S = \sigma_{\text{<join condition>}} (R \times S)$

Table 8.1 Operations of Relational Algebra

OPERATION	PURPOSE	NOTATION
SELECT	Selects all tuples that satisfy the selection condition from a relation R.	$\sigma_{\text{<selection condition>}}(R)$
PROJECT	Produces a new relation with only some of the attributes of R, and removes duplicate tuples.	$\pi_{\text{<attribute list>}}(R)$
THETA JOIN	Produces all combinations of tuples from R_1 and R_2 that satisfy the join condition.	$R_1 \bowtie_{\text{<join condition>}} R_2$
EQUIJOIN	Produces all the combinations of tuples from R_1 and R_2 that satisfy a join condition with only equality comparisons.	$R_1 \bowtie_{\text{<join condition>}} R_2$, OR $R_1 \bowtie_{\text{(<join attributes 1>), (<join attributes 2>)}} R_2$
NATURAL JOIN	Same as EQUIJOIN except that the join attributes of R_2 are not included in the resulting relation; if the join attributes have the same names, they do not have to be specified at all.	$R_1 *_{\text{<join condition>}} R_2$, OR $R_1 *_{\text{(<join attributes 1>), (<join attributes 2>)}} R_2$ OR $R_1 * R_2$

OPERATION	PURPOSE	NOTATION
UNION	Produces a relation that includes all the tuples in R_1 or R_2 or both R_1 and R_2; R_1 and R_2 must be union compatible.	$R_1 \cup R_2$
INTERSECTION	Produces a relation that includes all the tuples in both R_1 and R_2; R_1 and R_2 must be union compatible.	$R_1 \cap R_2$
DIFFERENCE	Produces a relation that includes all the tuples in R_1 that are not in R_2; R_1 and R_2 must be union compatible.	$R_1 - R_2$
CARTESIAN PRODUCT	Produces a relation that has the attributes of R_1 and R_2 and includes as tuples all possible combinations of tuples from R_1 and R_2.	$R_1 \times R_2$
DIVISION	Produces a relation $R(X)$ that includes all tuples $t[X]$ in $R_1(Z)$ that appear in R_1 in combination with every tuple from $R_2(Y)$, where $Z = X \cup Y$.	$R_1(Z) \div R_2(Y)$

Ecco degli esempi:

- **Q1: Retrieve the name and address of all employees who work for the 'Research' department.**

  $RESEARCH_DEPT \leftarrow \sigma_{DNAME='Research'}(DEPARTMENT)$

  $RESEARCH_EMPS \leftarrow (RESEARCH_DEPT \bowtie_{DNUMBER=DNOEMPLOYEE} EMPLOYEE)$

  $RESULT \leftarrow \pi_{FNAME, LNAME, ADDRESS}(RESEARCH_EMPS)$

- **Q6: Retrieve the names of employees who have no dependents.**

  $ALL_EMPS \leftarrow \pi_{SSN}(EMPLOYEE)$

  $EMPS_WITH_DEPS(SSN) \leftarrow \pi_{ESSN}(DEPENDENT)$

  $EMPS_WITHOUT_DEPS \leftarrow (ALL_EMPS - EMPS_WITH_DEPS)$

  $RESULT \leftarrow \pi_{LNAME, FNAME}(EMPS_WITHOUT_DEPS * EMPLOYEE)$

As a single expression, these queries become:

- **Q1: Retrieve the name and address of all employees who work for the 'Research' department.**

$$\pi_{Fname, Lname, Address}(\sigma_{Dname='Research'}(DEPARTMENT \bowtie_{Dnumber=Dno}(EMPLOYEE))$$

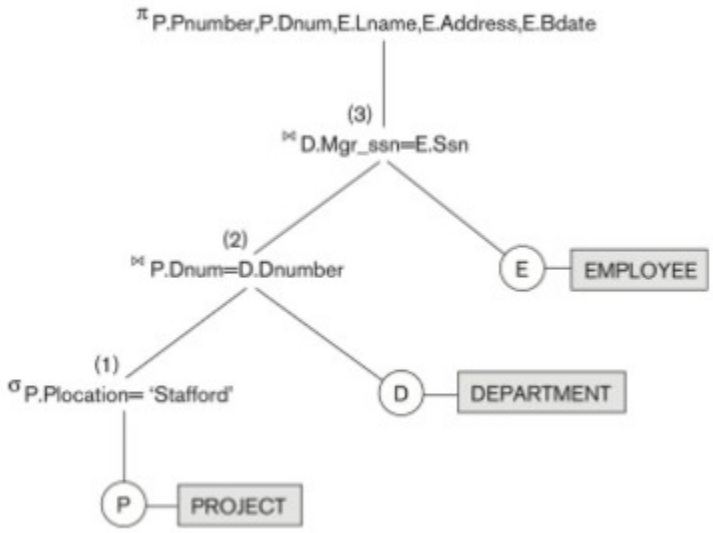
- **Q6: Retrieve the names of employees who have no dependents.**

$$\pi_{Lname, Fname}((\pi_{Ssn}(EMPLOYEE) - \rho_{Ssn}(\pi_{Essn}(DEPENDENT))) * EMPLOYEE)$$

Query tree

È una notazione ad albero che si utilizza per **rappresentare una query composta**. Grazie a questo diagramma si riesce a visualizzare la complessità di una query, l'ordine di esecuzione, la generazione di risultato intermedi e si è in grado di **ottimizzare** facilmente una query.
I **nodi interni** sono le **operazioni** (ad esempio SELECT, PROJECT, etc) e le **foglie** sono le **relazioni base**.
L'ottimizzazione delle query algebriche consiste nella riscrittura della query o nella modifica dell'albero delle query in un albero equivalente.
Ecco un esempio:

Figure 8.9 Query tree corresponding to the relational algebra expression for Q2.



Il linguaggio SQL

Il modello relazionale definisce i **concetti generali** e i **vincoli** per modellare e strutturare i dati di una certa applicazione o dominio di interesse.
I linguaggi per DBMS sono linguaggi basati sulle proprietà algebrico/logiche del modello relazionale.
Il linguaggio SQL è un linguaggio **dichiarativo** per poter manipolare un DBMS e i suoi dati.
SQL è basato sul **modello relazionale** (con qualche differenza, ad esempio che si parla di tabelle, non di relazioni)
Questo linguaggio è composto da due componenti principali:

- **Data Definition Language** (DDL)
  Contiene i costrutti necessari per la creazione e modifica dello **schema** della base di dati
- **Data Manipulation Language** (DML)
  Contiene i costrutti necessari per le interrogazioni e quelli di inserimento, eliminazione e modifica dei **dati**.

È importante sapere che il linguaggio SQL dice al DBMS **cosa** fare, e non come (che viene fatto implicitamente)

Uno **schema SQL** è **identificato da un nome** ed è un collettore che include:

- Tabelle
- Vincoli
- Views
- Domini
- Altri costrutti

Ogni **dichiarazione SQL** (query) termina con un punto e virgola

DDL

Prima di tutto bisogna creare lo **schema** (ovvero il collettore) di una base di dati con il comando

$$CREATE\ \ SCHEMA(DATABASE)\ \ <nomeSchema>\ \ [authorization\ \ nomeProprietario]$$

Poi possiamo usare i seguenti comandi per vedere tutti gli schemi o per eliminarne uno:

$$SHOW\ \ SCHEMA(DATABASE)$$

$$DROP\ \ SCHEMA(DATABASE)\ \ <nomeSchema>$$

Il comando

$$CREATE\ \ TABLE\ \ <nomeSchema.nomeTabella>\ \ (<listaAttributi>)$$

specifica una **nuova relazione**, bisogna specificare il **nome della tabella** e gli **attributi** (con i loro domini e vincoli). Si può specificare lo schema in cui vogliamo inserire la tabella oppure no.
Si può anche creare una tabella partendo da un'altra già esistente (la nuova tabella verrà riempita con i valori già presenti nella vecchia tabella) con il comando:

$$CREATE\ \ TABLE\ \ <nuovaTabella>\ AS\ \ SELECT\ \ (<listaAttributi>)\ ¿<vecchiaTabella>$$

```
CREATE TABLE D5EMPS LIKE EMPLOYEE
        (SELECT  E.*
         FROM    EMPLOYEE AS E
         WHERE   E.Dno=5)
WITH DATA;
```

In SQL si possono creare delle vere e proprie tabelle, con il comando appena visto, e in questo caso le relazioni e le loro tuple vengono effettivamente create e salvate come un file nel DBMS.
Si possono però creare anche delle **view**, ovvero delle relazioni virtuali, che non corrispondono a nessun file fisico presente effettivamente sul DBMS. Per fare una view basta sostituire *TABLE* con *VIEW* e possiamo anche aggiungere una clausola WHERE.
Una view è una singola tabella, derivata da altre tabelle, considerata come una tabella virtuale (ovvero che non esiste veramente) che non è per forza popolata.

```
V1:   CREATE VIEW   WORKS_ON1
      AS SELECT     Fname, Lname, Pname, Hours
          FROM      EMPLOYEE, PROJECT, WORKS_ON
          WHERE     Ssn=Essn AND Pno=Pnumber;

V2:   CREATE VIEW   DEPT_INFO(Dept_name, No_of_emps, Total_sal)
      AS SELECT     Dname, COUNT (*), SUM (Salary)
          FROM      DEPARTMENT, EMPLOYEE
          WHERE     Dnumber=Dno
          GROUP BY  Dname;
```

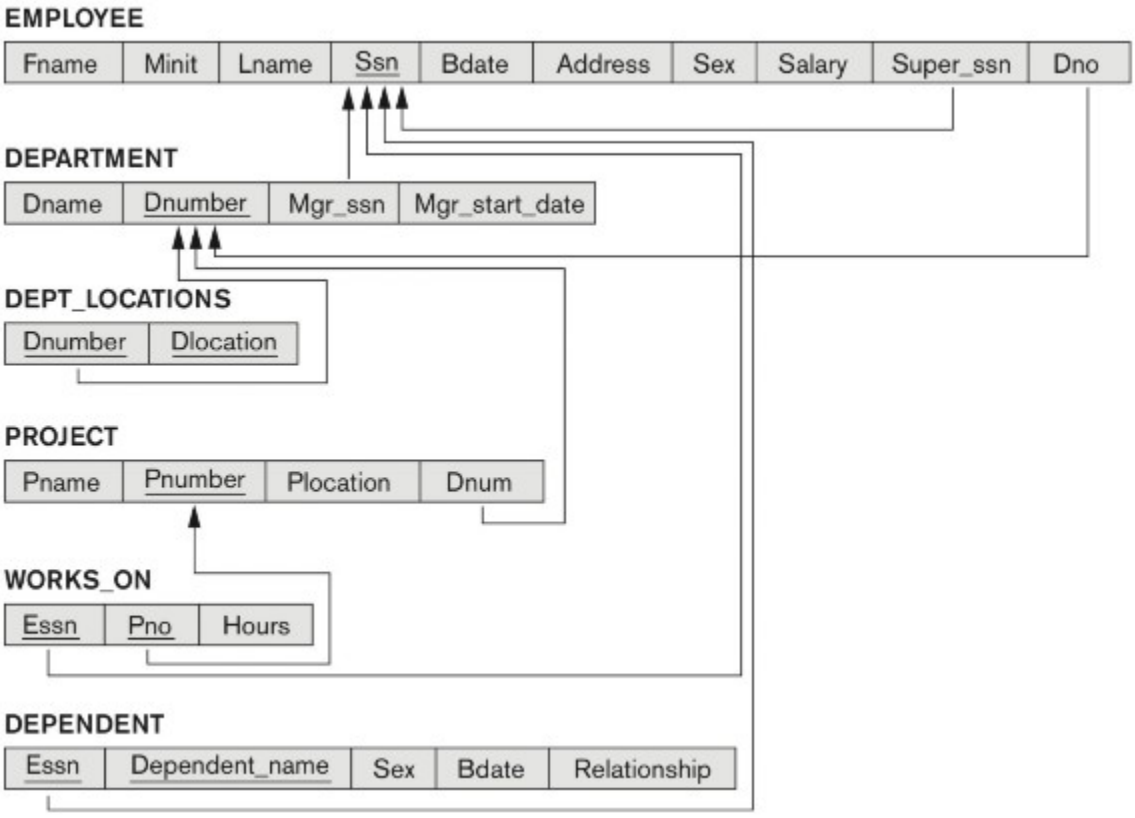
Le tabelle si possono eliminare (eliminando anche tutti i dati al loro interno) con il comando:

$$DROP\ \ TABLE\ \ <nomeTabella>$$

Esiste anche un comando per eliminare solo le tuple di una tabella ma non la tabella stessa:

$$TRUNCATE\ \ TABLE\ \ <nomeTabella>$$

Ecco un esempio di creazioni di tabelle partendo da uno schema relazionale.




```
CREATE TABLE EMPLOYEE
        ( Fname                 VARCHAR(15)             NOT NULL,
          Minit                 CHAR,
          Lname                 VARCHAR(15)             NOT NULL,
          Ssn                   CHAR(9)                 NOT NULL,
          Bdate                 DATE,
          Address               VARCHAR(30),
          Sex                   CHAR,
          Salary                DECIMAL(10,2),
          Super_ssn             CHAR(9),
          Dno                   INT                     NOT NULL,
        PRIMARY KEY (Ssn),
CREATE TABLE DEPARTMENT
        ( Dname                 VARCHAR(15)             NOT NULL,
          Dnumber               INT                     NOT NULL,
          Mgr_ssn               CHAR(9)                 NOT NULL,
          Mgr_start_date        DATE,
        PRIMARY KEY (Dnumber),
        UNIQUE (Dname),
        FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn) );
CREATE TABLE DEPT_LOCATIONS
        ( Dnumber               INT                     NOT NULL,
          Dlocation             VARCHAR(15)             NOT NULL,
        PRIMARY KEY (Dnumber, Dlocation),
        FOREIGN KEY (Dnumber) REFERENCES DEPARTMENT(Dnumber) );
```

continued on next sl

```
CREATE TABLE PROJECT
        ( Pname                 VARCHAR(15)             NOT NULL,
          Pnumber               INT                     NOT NULL,
          Plocation             VARCHAR(15),
          Dnum                  INT                     NOT NULL,
        PRIMARY KEY (Pnumber),
        UNIQUE (Pname),
        FOREIGN KEY (Dnum) REFERENCES DEPARTMENT(Dnumber) );
CREATE TABLE WORKS_ON
        ( Essn                  CHAR(9)                 NOT NULL,
          Pno                   INT                     NOT NULL,
          Hours                 DECIMAL(3,1)            NOT NULL,
        PRIMARY KEY (Essn, Pno),
        FOREIGN KEY (Essn) REFERENCES EMPLOYEE(Ssn),
        FOREIGN KEY (Pno) REFERENCES PROJECT(Pnumber) );
CREATE TABLE DEPENDENT
        ( Essn                  CHAR(9)                 NOT NULL,
          Dependent_name        VARCHAR(15)             NOT NULL,
          Sex                   CHAR,
          Bdate                 DATE,
          Relationship          VARCHAR(8),
        PRIMARY KEY (Essn, Dependent_name),
        FOREIGN KEY (Essn) REFERENCES EMPLOYEE(Ssn) );
```

Una volta creata una tabella possiamo modificarne la struttura con il comando

$$ALTER\ \ TABLE\ \ <nomeTabella>$$

e poi aggiungendo il tipo di modifica che si vuole fare e su cosa:

- Aggiungere una colonna
  $ADD\ \ <nomeColonna>$
- Eliminare una colonna
  $DROP\ \ COLUMN\ \ <nomeColonna>$
- Modificare il datatype di una colonna
  $ALTER\ \ COLUMN\ \ <nomeColonna>\ \ <datatype>$

I **data types** degli attributi possono essere:

- **Numerico**
  - numeric, INTEGER, INT o SMALLINT
  - decimal, FLOAT, REAL o DOUBLE PRECISION
- **Stringa di caratteri**
  - A lunghezza fissa: CHAR(n) o CHARACTER(n)
  - A lunghezza variabile: VARCHAR(n), CHAR VARYING(n), CHARACTER VARYING(n)
- **Stringa di bit**
  - A lunghezza fissa: BIT(n)
  - A lunghezza variabile: BIT VARYING(n)
- **Booleano**
  - I valori saranno TRUE, FALSE o NULL
- **DATE**
  - I componenti sono YEAR, MONTH e DAY nel formato YYYY-MM-DD

    - Il formato si può cambiare
- **TIME**
- **Timestamp**
    - I valori saranno composti da DATE e TIME
- **INTERVAL**
  Intervalli di tempo

I tipi DATE, TIME, Timestamp e INTERVAL possono essere convertiti in **stringhe** per il confronto

I **domini** sono utili quando vogliamo cambiare il datatype di un dominio usato da più attributi, e migliorano la leggibilità dello schema.
Ad esempio:

$$CREATE\ \ DOMAIN\ \ SSN_TYPE\ \ AS\ \ CHAR(9)\ \ <vincoli>;$$

Si possono anche creare tipi definiti dall'utente (supportati per le applicazioni object-oriented) con $CREATE\ \ TYPE$

Ci sono **3 vincoli base** supportati in SQL:

- Vincoli di **chiave**
  Il valore di una primary key non può essere duplicato
- Vincoli di **integrità di entità**
  Il valore di una primary key non può essere null
- Vincoli di **integrità referenziale**
  Il valore di una foreign key deve avere come valore il valore di una primary key già presente, altrimenti null

Certe foreign keys possono causare errori (ad esempio perché si riferiscono a tabelle che non sono ancora state create). I DBA hanno modi per fermare l'applicazione dell'integrità referenziale per aggirare questo problema.

Quando creiamo o modificchiamo una tabella, possiamo aggiungere ad ogni attributo i seguenti vincoli/restrizioni (scrivendoli subito dopo il datatype dell'attributo):

- $DEFAULT\ \ <value>$
  Possiamo specificare un valore di default per un attributo nel caso in cui non venga specificato
- $NOT\ \ NULL$
  Il valore di quell'attributo per ogni tupla non potrà mai essere NULL
- $\vee <condizione>$
  Possiamo applicare una clausola sul valore di un attributo
- $UNIQUE$
  Il valore di quell'attributo per ogni tupla non potrà mai essere uguale ad un altro valore di quell'attributo di un'altra tupla.
  Questi attributi vengono chiamati chiavi candidate nel modello relazionale
  Se vogliamo che siano uniche coppie (o più) di attributi dobbiamo inserire alla fine, dopo tutti gli attributi, la riga $UNIQUE(att1, att2)$

- $PRIMARY\ \ KEY$
  Specifica che quell'attributo è la chiave primaria, e assume automaticamente i vincoli di NOT NULL e UNIQUE
  Nel caso in cui la chiave è composta da più attributi scriviamo
  $PRIMARY\ \ KEY(att1, att2)$ in fondo alla query. (il vincolo PRIMARY KEY deve apparire una sola volta per tabella)
- $FOREIGN\ \ KEY\ \ REFERENCES\ \ <nomeTabella.nomeColonna>$
  Specifica che quell'attributo è una chiave esterna che si riferisce ad un'altra primary key.
  Con REFERENCES nomeTabella(nomeAttributo) specifichiamo a che attributi "punta".
  L'attributo a cui si fa riferimento nella tabella esterna deve essere soggetto al vincolo UNIQUE
  L'operazione di default quando questo vincolo viene violato è rifiutare l'aggiornamento
  Si possono aggiungere le azioni di violazione SET NULL, CASCADE o SET DEFAULT (oppure NO ACTION, non consente l'azione)
  Nel caso in cui il vincolo di integrità referenziale riguardi più di un attributo scriviamo
  $FOREIGN\ \ KEY(att1, att2, etc)\ \ REFERENCES\ \ nomeTabella(nAtt1, nAtt2, etc)$ in fondo alla query
- $AUTO_INCREMENT$
  Quando vogliamo che ogni tupla che inseriamo abbia il valore dell'ultima tupla inserita sommato di uno.
  Così otteniamo un campo che si autoincrementa a ogni inserimento nella tabella.

Si possono dare dei nomi ai vincoli, così da rendere più semplice la modifica successivamente:

$$CONSTRAINT\ \ <nomeVincolo>\ \ <vincolo>$$

Quando diamo dei nomi ai vincoli bisogna usare sempre la modalità di scriverli in fondo alla query.

Ecco l'aggiunta di vincoli con dei nomi al nostro esempio di prima:

```
CREATE TABLE EMPLOYEE
    ( ... ,
      Dno          INT          NOT NULL       DEFAULT 1,
    CONSTRAINT EMPPK
      PRIMARY KEY (Ssn),
    CONSTRAINT EMPSUPERFK
      FOREIGN KEY (Super_ssn) REFERENCES EMPLOYEE(Ssn)
                  ON DELETE SET NULL          ON UPDATE CASCADE,
    CONSTRAINT EMPDEPTFK
      FOREIGN KEY(Dno) REFERENCES DEPARTMENT(Dnumber)
                  ON DELETE SET DEFAULT       ON UPDATE CASCADE);
CREATE TABLE DEPARTMENT
    ( ... ,
      Mgr_ssn CHAR(9)          NOT NULL       DEFAULT '888665555',
      ... ,
    CONSTRAINT DEPTPK
      PRIMARY KEY(Dnumber),
    CONSTRAINT DEPTSK
      UNIQUE (Dname),
    CONSTRAINT DEPTMGRFK
      FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn)
                  ON DELETE SET DEFAULT       ON UPDATE CASCADE);
CREATE TABLE DEPT_LOCATIONS
    ( ... ,
    PRIMARY KEY (Dnumber, Dlocation),
    FOREIGN KEY (Dnumber) REFERENCES DEPARTMENT(Dnumber)
                  ON DELETE CASCADE           ON UPDATE CASCADE);
```

Si possono anche specificare **vincoli** che vengono applicati sulle **singole tuple**, questo si fa aggiungendo un vincolo CHECK al termine della dichiarazione della tabella, così facendo possiamo mettere vincoli su un attributo che dipendono dal valore di un altro attributo.

In SQL esistono altri vincoli che vanno oltre al modello relazionale, come CREATE ASSERTION (specifica altri tipi di vincoli al di fuori dell'ambito dei vincoli del modello relazionale integrato) e CREATE TRIGGER (specifica le azioni automatiche che il sistema di database eseguirà quando si verificano determinati eventi e condizioni)
I trigger sono composti da:

1. Evento (o eventi)
2. Condizioni
3. Azioni

```
R5:
CREATE TRIGGER SALARY_VIOLATION
BEFORE INSERT OR UPDATE OF Salary, Supervisor_ssn ON
EMPLOYEE

FOR EACH ROW
WHEN (NEW.SALARY > ( SELECT Salary FROM EMPLOYEE
                     WHERE Ssn = NEW. Supervisor_Ssn))
INFORM_SUPERVISOR (NEW.Supervisor.Ssn, New.Ssn)
```

DML

L'operazione SELECT è l'operazione base per ottenere informazioni da un database.
A differenza dell'algebra relazionale, SQL permette ad una tabella di avere due o più tuple che sono **uguali in tutti gli attributi** (questo quando facciamo una SELECT senza selezionare la PRIMARY KEY, perchè in quel caso non ci potranno essere tuple uguali per le proprietà della PRIMARY KEY).
La forma più base dello SELECT è:

$$SELECT\ \ column1,\ column2,\ldots\ ¿\ nomeTabella;$$

Dove al posto dei nomi delle colonne possiamo mettere * per ottenerle tutte.
Negli attributi del select possiamo mettere anche espressione aritmetiche (semplici come addizione, sottrazione, moltiplicazoine, divisione) sugli attributi.
Non dobbiamo confondere la selezione dell'algebra relazionale con il SELECT di SQL, in quanto il SELECT permette di fare proiezione e selezione assieme. Infatti la forma vista qui sopra è una proiezione (anche se ammette duplicati, visto che di default è SELECT ALL).
Per ottenere la vera è propria proiezione (quindi senza duplicati) utilizziamo

$$SELECT\ \ DISTINCT\ \ column1,\ column2,\ldots\ ¿\ nomeTabella;$$

(di default è SELECT ALL).
Per introdurre il concetto di selezione aggiungiamo $WHERE\ \ <condizione>$ alla fine della select. Nella clausola WHERE sono ammessi solo risultati booleani, quindi possiamo utilizzare i seguenti operatori.

Operator	Description	Example
=	Equal	Try it
>	Greater than	Try it
<	Less than	Try it
>=	Greater than or equal	Try it
<=	Less than or equal	Try it
<>	Not equal. **Note:** In some versions of SQL this operator may be written as !=	Try it
BETWEEN	Between a certain range	Try it
LIKE	Search for a pattern	Try it
IN	To specify multiple possible values for a column	Try it

e possiamo unire più condizioni con AND, OR e NOT.
Quando in queste condizioni ci riferiamo ad un valore dobbiamo scriverlo tra apici $'valore'$, così da non creare ambiguità e poterlo distinguere dai nomi degli attributi.

Ecco degli esempi:

Bdate	Address
1965-01-09	731Fondren, Houston, TX

Fname	Lname	Address
John	Smith	731 Fondren, Houston, TX
Franklin	Wong	638 Voss, Houston, TX
Ramesh	Narayan	975 Fire Oak, Humble, TX
Joyce	English	5631 Rice, Houston, TX

Query 0. Retrieve the birth date and address of the employee(s) whose name is 'John B. Smith'.

```
Q0:   SELECT   Bdate, Address
      FROM     EMPLOYEE
      WHERE    Fname='John' AND Minit='B' AND Lname='Smith';
```

Query 1. Retrieve the name and address of all employees who work for the 'Research' department.

```
Q1:   SELECT   Fname, Lname, Address
      FROM     EMPLOYEE, DEPARTMENT
      WHERE    Dname='Research' AND Dnumber=Dno;
```

(c)

Pnumber	Dnum	Lname	Address	Bdate
10	4	Wallace	291Berry, Bellaire, TX	1941-06-20
30	4	Wallace	291Berry, Bellaire, TX	1941-06-20

Query 2. For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, address, and birth date.

```
Q2:   SELECT   Pnumber, Dnum, Lname, Address, Bdate
      FROM     PROJECT, DEPARTMENT, EMPLOYEE
      WHERE    Dnum=Dnumber AND Mgr_ssn=Ssn AND
               Plocation='Stafford';
```

Se voglio usare la condizione che un valore sia NULL, oppure non lo sia, devo usare *WHERE <nomeColonna> IS NULL* oppure *WHERE <nomeColonna> IS NOT NULL*
Questo perchè *NULL* può significare tante cose:

- Valore sconosciuto
- Valore non disponibile o trattenuto
- Attributo non applicabile

Di conseguenza ogni valore *NULL* deve essere considerato come diverso dagli altri valori *NULL*.
Quindi SQL usa una logica a tre valori:

- *TRUE*
- *FALSE*
- *UNKNOWN* (tipo "forse")

Di conseguenza la comparazione $NULL = NULL$ non si può fare.

Table 7.1 Logical Connectives in Three-Valued Logic

(a)	AND	TRUE	FALSE	UNKNOWN
	TRUE	TRUE	FALSE	UNKNOWN
	FALSE	FALSE	FALSE	FALSE
	UNKNOWN	UNKNOWN	FALSE	UNKNOWN
(b)	OR	TRUE	FALSE	UNKNOWN
	TRUE	TRUE	TRUE	TRUE
	FALSE	TRUE	FALSE	UNKNOWN
	UNKNOWN	TRUE	UNKNOWN	UNKNOWN
(c)	NOT			
	TRUE	FALSE		
	FALSE	TRUE		
	UNKNOWN	UNKNOWN		

Nel BETWEEN i due estremi sono compresi e si usa così $BETWEEN \quad val1 \ \wedge \ val2$.
Nell' IN possiamo inserire una lista di valori oppure una query annidata (tipicamente una select con risultato compatibile con l'attributo). L'IN può essere visto come una serie di uguaglianze con degli OR e si usa cosi ¿ $(<listaValori>)$ oppure ¿ $(SELECT \ \ldots)$
L'operatore $LIKE$ è usato per pattern matching:

Wildcard Characters in SQL Server

Symbol	Description	Example
%	Represents zero or more characters	bl% finds bl, black, blue, and blob
_	Represents a single character	h_t finds hot, hat, and hit
[]	Represents any single character within the brackets	h[oa]t finds hot and hat, but not hit
^	Represents any character not in the brackets	h[^oa]t finds hit, but not hot and hat
-	Represents any single character within the specified range	c[a-b]t finds cat and cbt

LIKE Operator	Description
WHERE CustomerName LIKE 'a%'	Finds any values that start with "a"
WHERE CustomerName LIKE '%a'	Finds any values that end with "a"
WHERE CustomerName LIKE '%or%'	Finds any values that have "or" in any position
WHERE CustomerName LIKE '_r%'	Finds any values that have "r" in the second position
WHERE CustomerName LIKE 'a_%'	Finds any values that start with "a" and are at least 2 characters in length
WHERE CustomerName LIKE 'a__%'	Finds any values that start with "a" and are at least 3 characters in length
WHERE ContactName LIKE 'a%o'	Finds any values that start with "a" and ends with "o"

Si possono fare anche **query annidate**, ovvero dei blocchi completi di select-from-where tutto all'interno di una WHERE di un'altra query.

```
Q4A:  SELECT   DISTINCT Pnumber
      FROM     PROJECT
      WHERE    Pnumber IN
               ( SELECT     Pnumber
                 FROM       PROJECT, DEPARTMENT, EMPLOYEE
                 WHERE      Dnum=Dnumber AND
                            Mgr_ssn=Ssn AND Lname='Smith' )
               OR
               Pnumber IN
               ( SELECT     Pno
                 FROM       WORKS_ON, EMPLOYEE
                 WHERE      Essn=Ssn AND Lname='Smith' );
```

Ecco alcuni esempi:

Bdate	Address
1965-01-09	731Fondren, Houston, TX

Fname	Lname	Address
John	Smith	731 Fondren, Houston, TX
Franklin	Wong	638 Voss, Houston, TX
Ramesh	Narayan	975 Fire Oak, Humble, TX
Joyce	English	5631 Rice, Houston, TX

Query 0. Retrieve the birth date and address of the employee(s) whose name is 'John B. Smith'.

```
Q0:   SELECT   Bdate, Address
      FROM     EMPLOYEE
      WHERE    Fname='John' AND Minit='B' AND Lname='Smith';
```

Query 1. Retrieve the name and address of all employees who work for the 'Research' department.

```
Q1:   SELECT   Fname, Lname, Address
      FROM     EMPLOYEE, DEPARTMENT
      WHERE    Dname='Research' AND Dnumber=Dno;
```

Lo stesso nome può essere usato per due (o più) attributi in relazioni differenti, ma nel momento in cui facciamo una SELECT (che esegue il **prodotto cartesiano** senza la condizione WHERE) dobbiamo specificare nella clausola WHERE il nome della tabella dell'attributo a cui vogliamo riferirci, per evitare ambiguità: *nomeTabella*.*nomeAttributo*.

```
Q1A:  SELECT   Fname, EMPLOYEE.Name, Address
      FROM     EMPLOYEE, DEPARTMENT
      WHERE    DEPARTMENT.Name='Research' AND
               DEPARTMENT.Dnumber=EMPLOYEE.Dnumber;
```

Se facciamo il prodotto cartesiano sulla stessa tabella dobbiamo fare un **aliasing**, altrimenti è impossibile distinguere le due tabelle: ¿ *<nomeTabella> AS <nuovoNome>*.

Possiamo anche rinominare il nome degli attributi: $\ldots \ AS \ <nuovoNome(att1, att2, \ldots)>$.

Query 8. For each employee, retrieve the employee's first and last name and the first and last name of his or her immediate supervisor.

```
SELECT E.Fname, E.Lname, S.Fname, S.Lname
FROM EMPLOYEE AS E, EMPLOYEE AS S
WHERE E.Super_ssn=S.Ssn;
```

Abbiamo anche qua le operazioni insiemistiche (UNION, EXCEPT, INTERSECT) ma le due tabelle (anche derivate) devono essere compatibili (stesse colonne nello stesso ordine e con gli stessi datatypes):

$$(SELECT \ \ldots \ ¿\ldots) \cup (SELECT \ \ldots \ ¿\ldots)$$

possiamo usare UNION ALL se vogliamo ammettere duplicati.
Se le colonne hanno nome diverso possiamo usare ovviamente l'aliasing per risolvere questo problema.

Durante una $SELECT$ possiamo aggiungere operazioni matematiche sia sugli attributi selezionati che nelle condizioni del WHERE.

Se vogliamo ottenere il massimo o il minimo valore quando scegliamo quali colonne selezionare scriviamo $MAX(nomeColonna)$ oppure $MIN(nomeColonna)$. Nel caso in cui selezioniamo altri attributi verranno mostrati gli attributi solo del massimo (ma non è una buona pratica in quanto nel caso in cui più tuple abbiano il valore massimo sarà casuale).
Nello stesso modo funzionano anche la conta $COUNT()$, la media $AVG()$ e la somma $\sum()$.
Queste funzioni si chiamano funzioni di aggregazione e vengono usate per "riassumere" le informazioni presenti in più tuple in una singola tupla.
È importantissimo sapere che nelle funzioni di aggregazione i valori $NULL$ vengono scartati, quindi se abbiamo 5 query di cui 1 con quell'attributo NULL, allora il COUNT ritornerà 4.
Le funzioni di aggregazione sono usate spesso assieme a GROUP BY, per raggruppare le tuple in specifici insiemi e ottenere le aggregazioni per ognuno di questi insiemi.

Posso usare $ORDER \ \ BY \ \ <nomeAttributo> \ \ mod$ per ordinare le tuple del risultato in base ai valori di un attributo. mod può essere ASC (default) o DESC. Posso mettere più attributi su cui effettuare l'ordinamento, gli altri attributi verranno usati quando il valore del primo attributo è uguale.

```
SELECT      <attribute list>
FROM        <table list>
[ WHERE     <condition> ]
[ ORDER BY <attribute list> ];
```

Un SELECT tra due tabelle (o più) senza filtri fa il prodotto cartesiano (sempre senza l'eliminazione dei duplicati), altrimenti posso mettere una condizione WHERE per effettuare una sorta di join.
Esiste però il comando fatto apposta per il join, dove dobbiamo specificare quali colonne sono "uguali" per la condizione di join.

$$SELECT \dots \; ¿ \; tab1 \;\; JOIN \;\; tab2 \;\; ON \;\; <condizione>$$

Se non specifichiamo gli attributi verrà fatto un EQUIJOIN per tutti gli attributi che hanno lo stesso nome.
Ovviamente abbiamo i vari tipi di join:

- (INNER) JOIN
- LEFT (OUTER) JOIN
- RIGH (OUTER) JOIN
- FULL (OUTER) JOIN

L'inner join si può replicare con una semplice SELECT … FROM … WHERE.

Si può usare anche INNER JOIN invece di fare SELECT WHERE, e poi abbiamo gli altri JOIN come LEFT JOIN

Per raggruppare le tuple che hanno lo stesso valore per un attributo (potrebbe essere utile contarle) possiamo usare $GROUP \;\; BY \;\; <nomeColonna>$.
GROUP BY è spesso utilizzato assieme alle funzioni di aggregazione (SUM, COUNT, AVG, MIN, MAX). L'attributo utilizzato per il group by deve sempre essere nella lista della selezione, e tutti gli altri attributi devono essere funzioni di aggregazione (che verranno applicate al gruppo ristretto). Se seleziono un attributo "normale" mi viene tornato il valore della prima tupla.
Se l'attributo che utilizziamo per il group by ammette valori $NULL$, allora verrà creato un gruppo separato per le tuple con $NULL$ (se presenti)

Se vogliamo usare una condizione su un valore aggregato non possiamo farlo con il WHERE, perchè non permette di usare valori aggregati (perchè si riferisce solo alle singole tuple), quindi l'unico modo per farlo con il WHERE e usarlo con una query annidata.
Per renderlo possibile senza query annidata allora usiamo
$HAVING \;\; <condizione \;\; su \;\; valore \;\; aggregato>$, ma questo operatore può essere usato solo con il GROUP BY. Questo perché HAVING si basa appunto sugli interi gruppi che abbiamo creato tramite GROUP BY, quindi ci permette di filtrare i gruppi di un GROUP BY.
Quindi il WHERE viene applicato ad ogni tupla, il HAVING viene applicato ad ogni gruppo (anche per questo dobbiamo usare HAVING con le funzioni aggregate).

Usiamo l'operatore EXISTS quando vogliamo vedere se esistono delle tuple con determinate caratteristiche associate alla nostra tabella. Verrà ritornato TRUE se ne esiste almeno una e nel caso in cui sia TRUE verrà selezionata la tupla della nostra tabella.

$$SELECT \;\; \dots \;\; \text{FROM} \;\; \dots \;\; WHERE \;\; \exists \;\; (SELECT \;\; \dots);$$

Esiste anche NOT EXISTS.

Esiste anche la funzione UNIQUE(Q) che ritorna TRUE se non ci sono duplicati nel risultato della query Q.

Invece usiamo gli operatori *ANY* (o *SOME*) e *ALL* se vogliamo ottenere rispettivamente se il confronto è vero per almeno una tupla, oppure per tutte le tuple. Quindi si basa su operazioni booleane.
La sintassi è la seguente:

SELECT … ¿ … WHERE nomeAttributo operatore ALL (subquery)

Dove *operatore* può essere $>, <, >=, <=, <>, =$.
Usare *ANY* con = è come usare IN.
Possiamo vedere *ANY* come una sequenza di OR, e *ALL* come una sequenza di AND

Possiamo usare *WITH* per creare una tabella temporanea che sarà utilizzata solamente in una determinata query (è come una view temporanea), ecco un esempio:

- **Q28':**

```
WITH BIGDEPTS (Dno) AS
    ( SELECT  Dno
      FROM    EMPLOYEE
      GROUP BY Dno
      HAVING  COUNT (*) > 5)
SELECT      Dno, COUNT (*)
FROM        EMPLOYEE
WHERE       Salary>40000 AND Dno IN BIGDEPTS
GROUP BY Dno;
```

Esiste anche il costrutto *CASE* che può essere usato come un if, infatti possiamo usare valori diversi in base a certe condizioni, ecco un esempio:

- **U6':**

```
UPDATE    EMPLOYEE
SET       Salary =
CASE      WHEN  Dno = 5 THEN    Salary + 2000
          WHEN  Dno = 4 THEN    Salary + 1500
          WHEN  Dno = 1 THEN    Salary + 3000
```

Case può essere usato in un qualsiasi punto di una query che richiede un valore.

Nel caso di **relationship ricorsive** (ovvero relationship che prendono due elementi della stessa tabella), dobbiamo poter avere il modo di vederle e scorrerle in un database. Ecco un esempio:

- **Q29:**
```
WITH RECURSIVE SUP_EMP (SupSsn, EmpSsn) AS
        SELECT    SupervisorSsn, Ssn
        FROM      EMPLOYEE
          UNION
        SELECT    E.Ssn, S.SupSsn
        FROM      EMPLOYEE AS E, SUP_EMP AS S
        WHERE     E.SupervisorSsn = S.EmpSsn)
        SELECT        *
        FROM      SUP_EMP;
```

Questa query parte con una tabella SUP_EMP vuota, e computa prima il primo livello di supervisori, poi il secondo etc, fino ad arrivare al punto in cui non ci sono più supervisori che possono essere aggiunti e si ferma.

Mettendo assieme tutte le nozioni che abbiamo visto, una query select può avere la seguente forma:

```
SELECT <attribute and function list>
FROM <table list>
[ WHERE <condition> ]
[ GROUP BY <grouping attribute(s)> ]
[ HAVING <group condition> ]
[ ORDER BY <attribute list> ];
```

L'ordine di esecuzione dei comandi in una QUERY è il seguente:

1. FROM e JOIN
   La clausola FROM e le successive JOIN vengono prima eseguite per determinare il working set totale di dati che viene interrogato. Ciò include sottoquery in questa clausola e può causare la creazione di tabelle temporanee sotto il cofano contenenti tutte le colonne e le righe delle tabelle che vengono unite.
2. WHERE
   Una volta ottenuto il working set totale di dati, i vincoli WHERE di primo passaggio vengono applicati alle singole righe e le righe che non soddisfano il vincolo vengono scartate. Ognuno dei vincoli può accedere solo alle colonne direttamente dalle tabelle richieste nella clausola FROM. Gli alias nella parte SELECT della query non sono accessibili nella maggior parte dei database poiché possono includere espressioni dipendenti da parti della query che non sono ancora state eseguite.
3. GROUP BY
   Le righe rimanenti dopo l'applicazione dei vincoli WHERE vengono quindi raggruppate in base ai valori comuni nella colonna specificata nella clausola GROUP BY. Come

risultato del raggruppamento, ci saranno solo tante righe quanti sono i valori univoci in quella colonna. Implicitamente, questo significa che dovresti usarlo solo quando hai funzioni di aggregazione nella tua query.

4. HAVING
   Se la query ha una clausola GROUP BY, i vincoli nella clausola HAVING vengono quindi applicati alle righe raggruppate, scartare le righe raggruppate che non soddisfano il vincolo. Come la clausola WHERE, anche gli alias non sono accessibili da questo passaggio nella maggior parte dei database.
5. SELECT
   Eventuali espressioni nella parte SELECT della query vengono infine calcolate.
6. DISTINCT
   Delle righe rimanenti, le righe con valori duplicati nella colonna contrassegnata come DISTINCT verranno eliminate.
7. ORDER BY
   Se un ordine è specificato dalla clausola ORDER BY, le righe vengono quindi ordinate in base ai dati specificati in ordine crescente o decrescente. Poiché tutte le espressioni nella parte SELECT della query sono state calcolate, è possibile fare riferimento agli alias in questa clausola.
8. LIMIT / OFFSET
   Infine, le righe che non rientrano nell'intervallo specificato da LIMIT e OFFSET vengono scartate, lasciando il set finale di righe da restituire dalla query.

Ora vediamo i comandi che ci permettono di **modificare** il database:

- **INSERT**
  Per inserire una tupla (una riga) o più nel database.
  L'ordine degli attributi deve essere lo stesso dell'ordine usato nel comando *CREATE TABLE* per quella tabella.
  I vincoli vengono controllati automaticamente
  $INSERT\ \ INTO\ \ <nomeTabella>|<listaColonne>|\ \ VALUES\ \ <listaValori>$

  ```
  U1:   INSERT INTO   EMPLOYEE
        VALUES        ( 'Richard', 'K', 'Marini', '653298653', '1962-12-30', '98
                        Oak Forest, Katy, TX', 'M', 37000, '653298653', 4 );
  ```

  Se non specifichiamo quali colonne (e/o in che ordine) allora vengono prese in considerazione tutte le colonne con l'ordine dello schema
  All'interno di VALUES possiamo mettere anche delle tuple estratte da un'altra tabella (quindi possiamo mettere una subquery SELECT)
- **UPDATE**
  Per aggiornare (cambiare i valori di qualche attributo) le tuple che soddisfano una certa condizione
  Bisogna inserire una clausola WHERE per specificare quali tuple modificare.
  Bisogna anche inserire una clausola SET per specificare quali attributi modificare e che valori dare.
  I vincoli di integrità referenziale (e anche tutti gli altri) vengono controllati automaticamente.

$UPDATE \ <nomeTabella> SET \ <col1 = val1, \ldots> \ WHERE \ <condizione>$

```
U5:   UPDATE    PROJECT
      SET       PLOCATION = 'Bellaire',
      DNUM = 5
      WHERE PNUMBER=10
```

Anche nel SET possiamo usare delle subquery.

- **DELETE**
  Per eliminare le tuple che soddisfano una certa condizione dal database
  Bisogna inserire una clausola WHERE per specificare la condizione delle tuple da eliminare.
  I vincoli di integrità referenziale vengono controllati automaticamente.
  Le tuple vengono eliminate da una tabella alla volta (a meno che non sia specificato CASCADE).
  Senza la clausola WHERE vengono eliminate tutte le tuple, lasciando vuota la tabella.
  $DELETE \ ¿ \ <nomeTabella> WHERE \ <condizione>$

```
U4A:   DELETE FROM    EMPLOYEE
       WHERE          Lname='Brown';

U4B:   DELETE FROM    EMPLOYEE
       WHERE          Ssn='123456789';

U4C:   DELETE FROM    EMPLOYEE
       WHERE          Dno=5;

U4D:   DELETE FROM    EMPLOYEE;
```

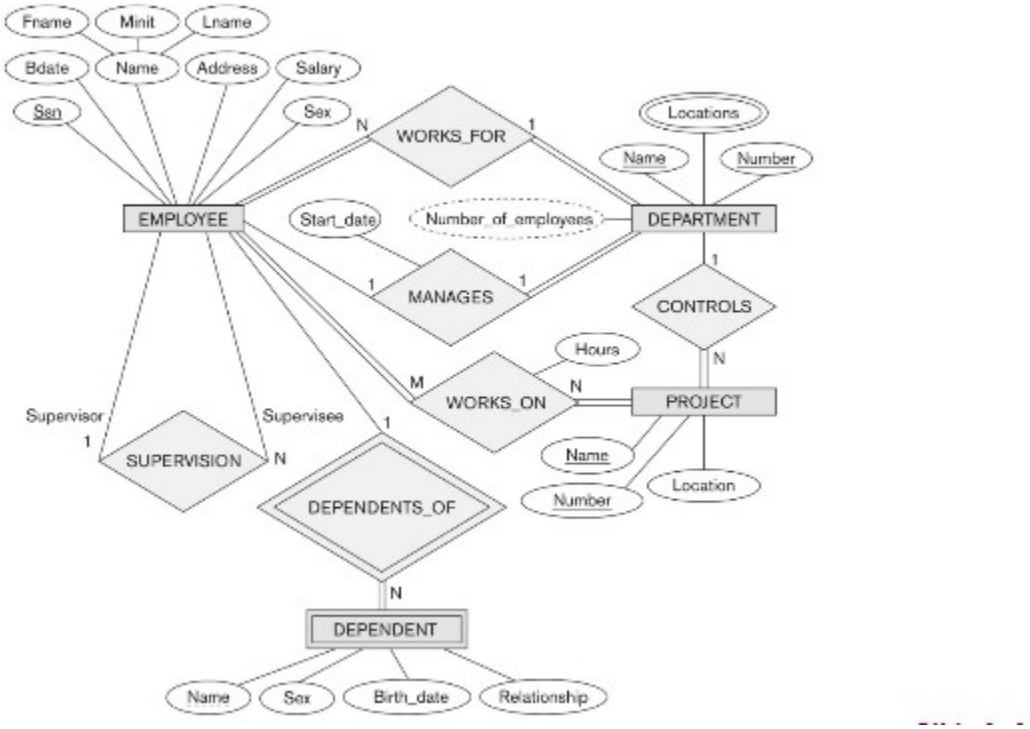
SQL mette a disposizione anche una serie di comandi per specificare i parametri di progettazione del database fisico, la struttura dei file delle relazioni, i percorsi di accesso etc. (ad esempio CREATE INDEX, i triggers).

Da ER a Relazionale

Per vedere come tradurre un modello ER in un modello relazionale, mantenendo invariate tutte le informazioni e tutti i vincoli, dividiamo la traduzioni in varie parti, vedendo per ogni situazione come si può agire:

Step 1. **Mappatura di entità “regolari”**
Step 2. **Mappatura di entità deboli**
Step 3. **Mappatura di relationship binarie 1:1**
Step 4. **Mappatura di relationship binarie 1:N**
Step 5. **Mappatura di relationship binarie N:M**
Step 6. **Mappatura di attributi multivalore**
Step 7. **Mappatura di relationship N-arie**

Come abbiamo detto la cosa più importante è quella di **preservare** ogni informazione (includendo anche gli attributi), mantenendo i vincoli (dove possibile, ad esempio non è possibile il vincolo massimo di cardinalità e la classificazione dei sottotipi).
Un altro obiettivo che dobbiamo porci è quello di **minimizzare i valori null**, ovvero creare un modello relazionale dove gli attributi che avranno valore null sono il meno possibile (vedremo cosa si intende e come raggiungere questo obiettivo).
Come sempre per gli esempi ci appoggeremo sul COMPANY database:



STEP 1
Vediamo la mappatura per una **entità** “regolare” (ovvero non debole).

Per ogni entità “forte” E nello schema ER, dobbiamo creare **una relazione** R che include tutti **gli attributi** semplici di E. Dobbiamo poi scegliere una tra le chiavi candidate di E che diventi la **chiave primaria** di R (se la chiave scelta da E è composta, l’insieme degli attributi semplici che formano la chiave candidata andranno a formare la chiave primaria di R).

Esempio:

- Example: We create the relations EMPLOYEE, DEPARTMENT, and PROJECT in the relational schema corresponding to the regular entities in the ER diagram.
  - SSN, DNUMBER, and PNUMBER are the primary keys for the relations EMPLOYEE, DEPARTMENT, and PROJECT as shown.

STEP 2
Vediamo la mappatura per una **entità debole**.

Per ogni entità debole W nello schema ER con entità identificante E, dobbiamo creare **una relazione** R che include tutti **gli attributi** semplici di W. Dobbiamo anche includere come **chiave esterna** di R la chiave primaria della relazione che corrisponde all'entità identificante E. Dobbiamo poi scegliere una tra le chiavi candidate di W che diventi la **chiave parziale** di R. Per ottenere la **chiave primaria** di R dobbiamo mettere assieme la chiave parziale di R e la chiave esterna della relazione che corrisponde all'entità identificante E

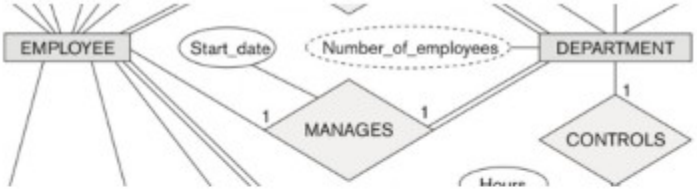
Esempio:

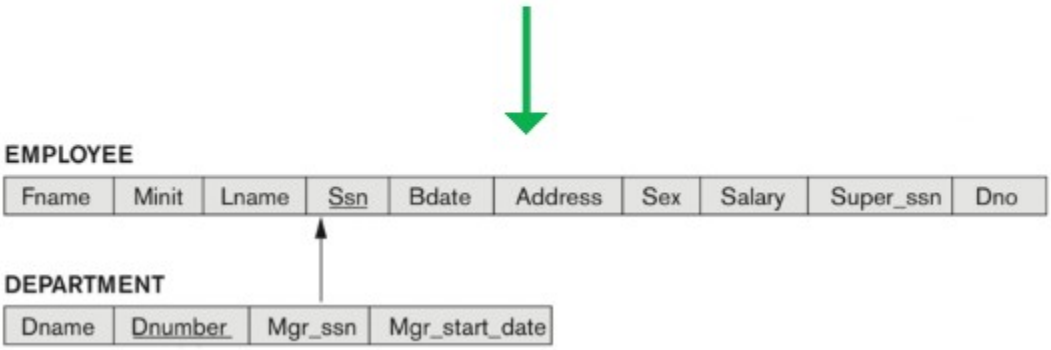
- **Example:** Create the relation DEPENDENT in this step to correspond to the weak entity type DEPENDENT.
  - Include the primary key SSN of the EMPLOYEE relation as a foreign key attribute of DEPENDENT (renamed to ESSN).
  - The primary key of the DEPENDENT relation is the combination {ESSN, DEPENDENT_NAME} because DEPENDENT_NAME is the partial key of DEPENDENT.

STEP 3
Vediamo la mappatura per una **relationship binaria 1:1**.
Per ogni relationship binaria 1:1 R dello schema ER, per prima cosa dobbiamo identificare le relazioni S e T che corrispondono alle entity type che partecipano alla relationship R.
Dopo possiamo proseguire in 3 modi diversi:

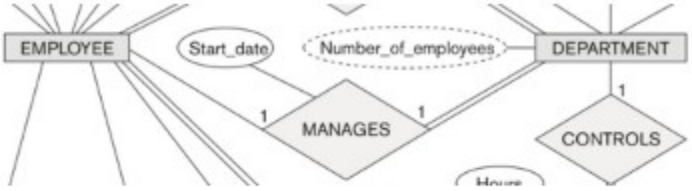
1. Approccio con **chiave esterna** (2 relazioni)
   Scegliamo una delle due relazioni (ad esempio S) e inseriamo in S una chiave esterna, che si riferirà alla chiave primaria di T.
   Se possibile è meglio scegliere una entity type che ha partecipazione totale in R nel ruolo di S, questo per minimizzare i valori null (se ha partecipazione totale dovrà sempre avere un valore in quell'attributo)

- Example: 1:1 relation MANAGES is mapped by choosing the participating entity type DEPARTMENT to serve in the role of S, because its participation in the MANAGES relationship type is total.





2. Approccio tramite **unione delle relazioni** (1 relazione)
   Uniamo le due relazioni in una relazione sola. Questo può essere appropriato quando entrambe le relazioni hanno partecipazione totale.
   Bisogna stare molto attenti alla violazione dei vincoli (in quanto perdiamo l'unicità di una chiave unendola con la chiave dell'altra relazione) e nel caso in cui non ci sia partecipazione totale possiamo avere "metà" tabella riempita con dei null (e quindi sprechiamo in un certo senso spazio).

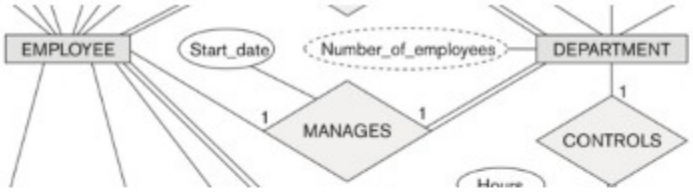


Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Dname	Dnumber	Mgr_ssn	Mgr_start_date

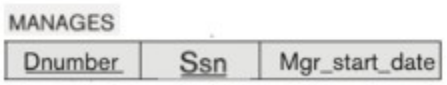
Non appropriato (manca partecipazione totale di EMPLOYEE)
NULL su parte destra per tutti gli impiegati non manager!

3. Approccio tramite **riferimento incrociato** (o relazione della relationship) (3 relazioni)
   Creiamo una terza relazione R che ha lo scopo di collegare tra loro le due primary key

delle relazioni S e T.
Dobbiamo creare una terza relazione (non strettamente necessaria) e questo non è molto conveniente.





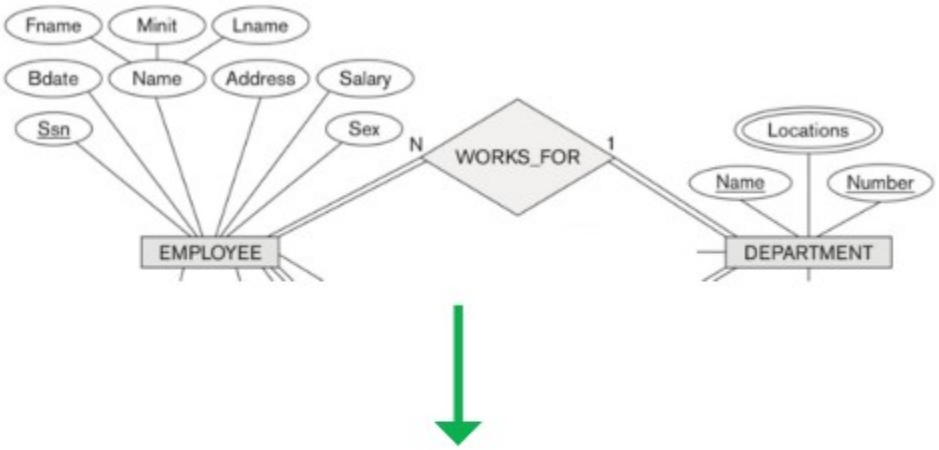


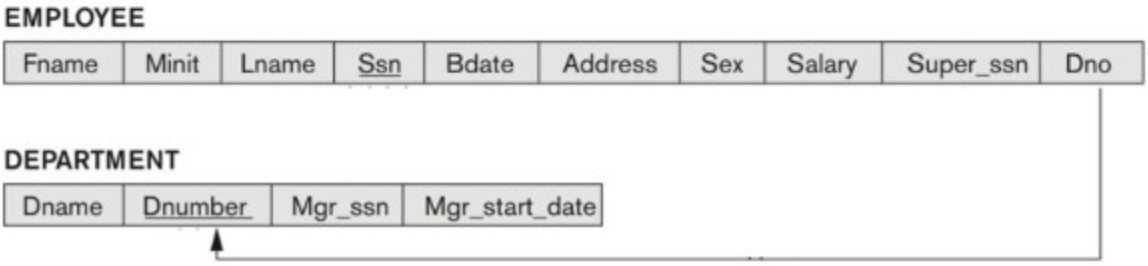
Corretto, ma in questo caso crea una relazione non necessaria

STEP 4
Vediamo la mappatura per una **relationship binaria 1:N**.
Per ogni relationship binaria 1:N R, per prima cosa dobbiamo identificare la relazione S che rappresenta l'entity type partecipante alla relationship dal lato di cardinalità N.
Dobbiamo includere in questa relazione S la foreign key dell'altra relazione T, che rappresenta l'altro entity type partecipante nella relazione.
Ogni attributo della relationship 1:N deve essere incluso in S.
(Si può usare anche qui un riferimento incrociato, con una terza relazione, ma è raro)

Esempio:

- Example: 1:N relationship types WORKS_FOR, CONTROLS, and SUPERVISION in the figure.
  - For WORKS_FOR we include the primary key DNUMBER of the DEPARTMENT relation as foreign key in the EMPLOYEE relation and call it DNO.





STEP 5

Vediamo la mappatura per una **relationship binaria N:M**.

Per ogni relationship binaria N:M R, creiamo una **nuova relazione** S che rappresenta R.

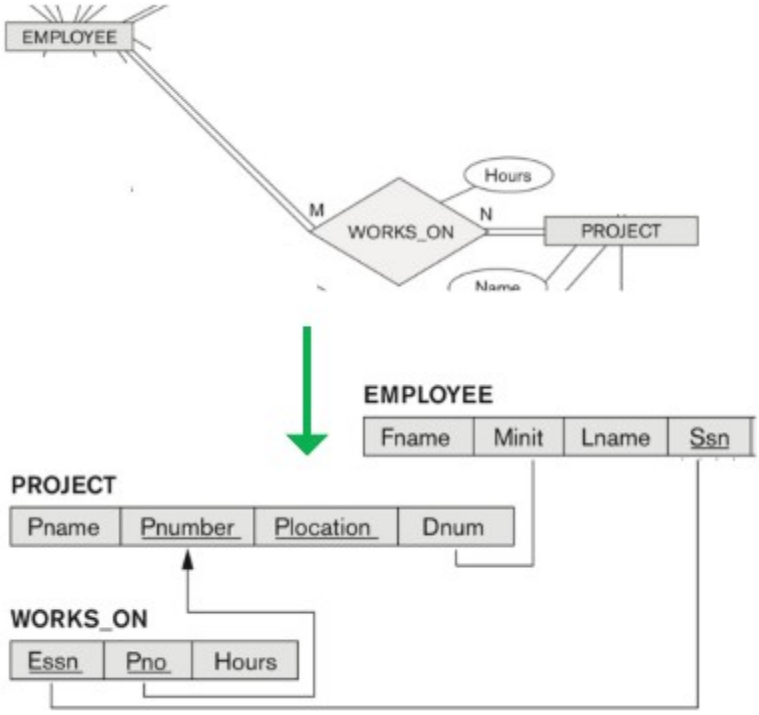
Questa è una relazione di relationship.

Questa nuova relazione includerà come chiavi esterne le chiavi primarie delle relazioni partecipanti, e la loro combinazione andrà a formare la **chiave primaria** di S.

Gli attributi della relazione R saranno inclusi come attributi di S.

Esempio:

- Example: The M:N relationship type WORKS_ON from the ER diagram is mapped by creating a relation WORKS_ON in the relational database schema.
  - The primary keys of the PROJECT and EMPLOYEE relations are included as foreign keys in WORKS_ON and renamed PNO and ESSN, respectively.
  - Attribute HOURS in WORKS_ON represents the HOURS attribute of the relation type. The primary key of the WORKS_ON relation is the combination of the foreign key attributes {ESSN, PNO}.



Step 6

Vediamo la mappatura per **attributi multivalore**.

Per ogni attributo multivalore A, creiamo una nuova relazione R.

La relazione R includerà un attributo che corrisponde ad A, con anche la foreign key alla primary key della relazione K, che è la relazione che rappresenta l'entità che ha come attributo multivalore A.

La chiave primaria di R è la combinazione di A e K

Esempio:

- **Example:** The relation DEPT_LOCATIONS is created.
  - The attribute DLOCATION represents the multivalued attribute LOCATIONS of DEPARTMENT, while DNUMBER-as foreign key-represents the primary key of the DEPARTMENT relation.
  - The primary key of R is the combination of {DNUMBER, DLOCATION}.

STEP 7

Vediamo la mappatura per **relationship N-arie**.

Per ogni relationship N-aria R, dove $N > 2$, creiamo una nuova relazione S che rappresenta R.

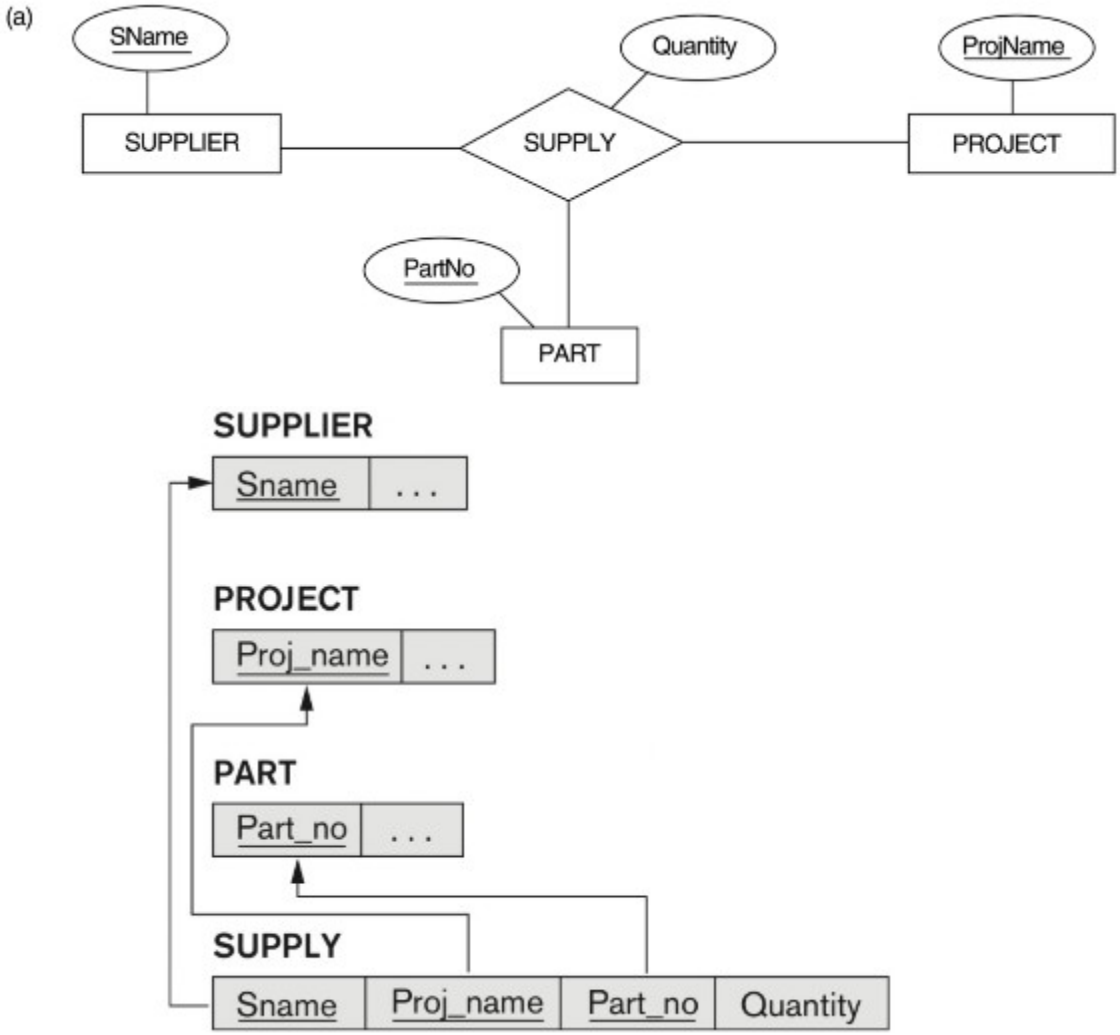
In S dobbiamo aggiungere le chiavi esterne, che si riferiscono alle chiavi primarie delle relazioni che rappresentano le entity type che partecipano alla relazione R.

La combinazione di queste chiavi esterne va a formare la **chiave primaria** di S

Ovviamente in S dobbiamo includere anche tutti gli attributi della relazione R.

Esempio:

- **Example:** The relationship type SUPPLY in the ER on the next slide.
  - This can be mapped to the relation SUPPLY shown in the relational schema, whose primary key is the combination of the three foreign keys {SNAME, PARTNO, PROJNAME}



Ecco un piccolo riassunto di come tradurre da ER a relazionale:

Table 9.1 Correspondence between ER and Relational Models

ER MODEL	RELATIONAL MODEL
Entity type	*Entity* relation
1:1 or 1:N relationship type	Foreign key (or *relationship* relation)
M:N relationship type	*Relationship* relation and *two* foreign keys
n-ary relationship type	*Relationship* relation and *n* foreign keys
Simple attribute	Attribute
Composite attribute	Set of simple component attributes
Multivalued attribute	Relation and foreign key
Value set	Domain
Key attribute	Primary (or secondary) key

Normalizzazione per database relazionali

Linee guida per la progettazione informale di database relazionali

La progettazione di un database relazionale consiste nel raggruppamento di attributi per formare schemi di relazione "buoni".
Ci sono due livelli per uno schema relazionale:

- Il livello logico di "visualizzazione utente".
- Il livello di "relazione di base" di archiviazione

Una **forma normale** è una proprietà di una base di dati relazionale che ne garantisce le **qualità**, ovvero l'assenza di eventuali difetti.
Come si misura la "qualità" di uno schema relazionale?

- **Livello logico**
  Ovvero in base al significato dello schema relazionale e degli attributi
  Questo livello aiuta l'utente a capire chiaramente il significato dei dati ed a formulare una query correttamente
- **Livello di memorizzazione e manipolazione**
  Riguarda come le tuple sono memorizzate e aggiornate (solo per le relazioni base)

Esistono delle misure informali di **qualità** per il disegno di schemi di relazione:

- **Semantica degli attributi**
  Il significato specifica come interpretare i valori degli attributi di una relazione.
  Bisogna disegnare uno schema di relazione del quale sia facile spiegarne il significato.
  Bisogna evitare di combinare attributi da tipi di entità e tipi di relationship differenti in una singola relazione.

Informalmente ogni tupla di una relazione dovrebbe rappresentare un'entità o una relationship

- **Riduzione dei valori ridondanti nelle tuple**
  Il primo obiettivo è quello di minimizzare lo spazio di memoria occupato dalle relazioni base
  Inoltre i valori ridondanti causano le anomalie negli aggiornamenti (bisogna tener aggiornati tutti i valori nelle operazioni di INSERT, DELETE e UPDATE che "toccano" questi attributi)
- **Non consentire insertion, deletion e modification anomalies**
  Queste anomalie accadono appunto quando abbiamo valori ridondanti nelle tuple o quando aggreghiamo in una relazione sola due entità diverse ad esempio.
  Se qualcuna è presente, annotare ciò chiaramente in modo che i programmi che aggiornano il database operino correttamente.
- **Riduzione dei valori** *null* **nelle tuple**
  Questo è frequente se raggruppiamo molti attributi in una relazione sola.
  Possono dare problemi nelle operazioni tipo COUNT e SUM oltre che a spreco di spazio.
  Il valore null può significare:
  - L'attributo non si applica a questa tupla
  - Il valore dell'attributo per questa tupla non è noto
  - Il valore dell'attributo è noto ma assente, cioè non è ancora stato registrato
  Gli attributi che assumono spesso un valore null possono essere trasferiti in una relazione separata. Se i valori null sono inevitabili bisogna assicurarsi che essi ricorrano solo in casi eccezionali e non per la maggioranza dellle tuple.
- **Non consentire tuple spurie**
  Progettare gli schemi di relazione in modo da poter effettuare JOIN con condizioni di uguaglianza su attributi che sono o chiave primaria o chiave esterna, in modo da non generare tuple spurie.
  Questo succede quando l'attributo che collega le due relazioni non è né chiave primaria né chiave esterna in nessuna delle due relazioni.
  Oppure quando applichiamo una normalizzazione senza rispettare bene le dipendenze funzionali.

Tali misure non sempre sono indipendenti (ovvero che in certe situazioni una è collegata all'altra)

Una relazione non è normalizzata se:

- presenta **ridondanze**
- si presta a **comportamenti poco desiderabili** durante le **operazioni di aggiornamento** (INSERT, UPDATE, DELETE)

Se uno schema non soddisfa la forma normale (ovvero non è in forma normale) è possibile applicare un processo per ottenerla, questo processo si chiama **normalizzazione**. Questo processo permette, appunto, di "trasformare" schemi non normalizzati in schemi che soddisfano la forma normale.

La normalizzazione si utilizza come tecnica di verifica dei risultati di una progettazione di basi di dati. La normalizzazione permette di individuare le anomalie e risolverle.

Per capire cosa sia una forma normale, vediamo subito un esempio di relazione che presenta anomalie:

Impiegato	Stipendio	Progetto	Bilancio	Funzione
Rossi	20	Marte	2	tecnico
Verdi	35	Giove	15	progettista
Verdi	35	Venere	15	progettista
Neri	55	Venere	15	direttore
Neri	55	Giove	15	consulente
Neri	55	Marte	2	consulente
Mori	48	Marte	2	direttore
Mori	48	Venere	15	progettista
Bianchi	48	Venere	15	progettista
Bianchi	48	Giove	15	direttore

Su questa relazione possiamo fare alcune osservazioni, che ci fanno capire che è possibile in qualche modo ottenere una normalizzazione:

- Lo stipendio per ciascun impiegato è unico ed è funzione del solo impiegato, ovvero è indipendente dal progetto in cui partecipa
- Il bilancio per ciascun progetto è unico ed è funzione del solo progetto, ovvero è indipendente dagli impiegati che vi partecipano
- Ogni impiegato, in ciascun progetto cui partecipa, svolge una sola funzione, eventualmente diversa da progetto a progetto

Queste osservazioni ci serviranno dopo.

Classifichiamo prima i tipi di anomalie, che possiamo notare in questa relazione non normalizzata:

- **Anomalia di cancellazione**
  Se un impiegato interrompe la partecipazione a tutti i progetti senza lasciare l'azienda, dobbiamo cancellarlo definitivamente, a meno di ammettere valori nulli sulla chiave *Progetto*, il che è inammissibile.
- **Anomalia di inserimento**
  Un nuovo impiegato senza progetto non può essere inserito.
- **Ridondanza**
  Lo stipendio di ciascun impiegato è ripetuto in tutte le tuple relative. (spreco di spazio)
- **Anomalia di aggiornamento**
  Se lo stipendio di un impiegato varia, è necessario andare a modificare il valore in diverse tuple.

Il motivo di tutte queste anomalie è che abbiamo usato **una unica relazione** per rappresentare **informazioni eterogenee** (gli impiegati con i relativi stipendi, i progetti con i relativi bilanci, le partecipazioni degli impiegati ai progetti con le relative funzioni).

Dipendenze funzionali

Per poter studiare in maniera sistematica questi vincoli è necessario introdurre un nuovo vincolo di integrità: la **dipendenza funzionale (DF)**. Questa dipendenza descrive i legami di tipo funzionale tra gli attributi di una relazione.
Le dipendenze funzionali si usano come **misure formali** della "bontà" della progettazione relazionale

Esempi di dipendenza funzionale sono:

- Lo **stipendio** di ciascun impiegato è unico (anche se partecipa a più progetti)
  Di conseguenza il valore dell'attributo $Impiegato$ **determina** il valore dell'attributo $Stipendio$ (più precisamente esiste una funzione che associa ad ogni elemento del dominio dell'attributo $Impiegato$ un solo elemento del dominio dell'attributo $Stipendio$)
- Il **bilancio** di ciascun progetto è unico e dipende solo dal progetto, indipendentemente dagli impiegati che vi partecipano.
  Di conseguenza il valore dell'attributo $Progetto$ **determina** il valore dell'attributo $Bilancio$

Il concetto di **dipendenza funzionale** è un vincolo tra due insiemi di attributi del database e può essere formalizzato nel seguente modo:
Data una relazione r su uno schema $R(X)$ e dati due sottoinsiemi non vuoti Y e Z di X, esiste in r una dipendenza funzionale da Y a Z se, per ogni coppia di tuple $t1$ e $t2$ di r con gli stessi valori su Y (cioè tali che $t1[Y]=t2[Y]$), risulta che $t1$ e $t2$ hanno gli stessi valori anche su Z (quindi $t1[Z]=t2[Z]$).
Una dipendenza funzionale tra Y e Z (come quella che abbiamo appena descritto) viene generalmente indicata con la seguente notazione:

$$Y \rightarrow Z$$

Che si interpreta come "i valori di Z dipendono/sono determinati da i valori di Y" oppure "i valori di Y determinano univocamente/funzionalmente i valori di Z".

Ritornando al nostro esempio sono individuabili facilmente due dipendenze funzionali (e come abbiamo visto è proprio su queste "relazioni" che si basano le anomalie):

$$Impiegato \rightarrow Stipendio$$
$$Progetto \rightarrow Bilancio$$

Notiamo che le dipendenze funzionali che abbiamo trovato non è detto che esistano… guardando una relazione popolata possiamo solamente essere in grado di dire dove **potrebbero** esistere delle dipendenze funzionali (scartando i casi dove non possono esistere sicuramente, trovando dei casi che violano il vincolo).
Questo perché le dipendenze funzionali sono derivate dalla semantica degli attributi.

Note that given the state of the TEACH relation, we can say that the FD: Text → Course may exist. However, the FDs Teacher → Course, Teacher → Text and Course → Text are ruled out.

TEACH

Teacher	Course	Text
Smith	Data Structures	Bartram
Smith	Data Management	Martin
Hall	Compilers	Hoffman
Brown	Data Structures	Horowitz

Ovviamente esistono delle dipendenze funzionali banali, ad esempio

$$Impiegato\ \ Stipendio \rightarrow Stipendio$$

Ma questa dipendenza è ovvia, in quanto due tuple con gli stessi valori su $Impiegato$ e $Stipendio$ è ovvio che abbiano lo stesso valore di $Stipendio$, essendo uno dei due attributi.
Si tratta quindi di una **dipendenza banale**, ovvero è **sempre soddisfatta**.

Possiamo formalizzare nella seguente maniera:

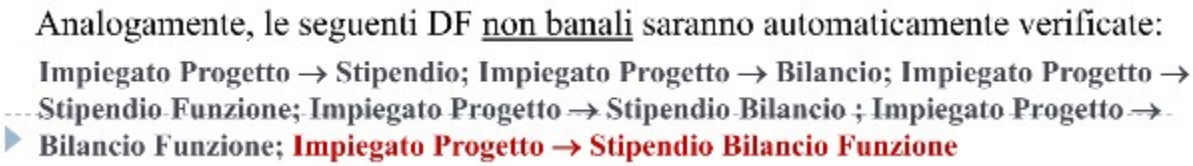
- Una DF $Y \rightarrow A$ è **banale** se A appartiene ad Y
- Una DF $Y \rightarrow Z$ è **non banale** se nessun attributo di Z appartiene ad Y

D'ora in poi con dipendenza funzionale ci riferiremo sempre a dipendenze funzionali non banali, visto che quelle banali sono inutili da considerare.

Se prendiamo una **chiave** K di una relazione R, si può facilmente verificare che esiste sempre una DF tra K e ogni altro attributo dello schema di R. Questo è dovuto al fatto che, per definizione di vincolo di chiave, **non possono esistere due tuple con gli stessi valori** su K, e quindi una DF che ha K al primo membro sarà sempre soddisfatta.
Di conseguenza possiamo affermare che **esisterà sempre** una DF tra una chiave candidata di una relazione e **tutti gli attributi dello schema della relazione** (esclusi quelli della chiave stessa, essendo una dipendenza funzionale banale).

Riprendendo il nostro esempio, dato che $Impiegato$ e $Progetto$ formano una chiave, allora

$$Impiegato\ \ Progetto \rightarrow Funzione$$



Possiamo quindi concludere dicendo che il vincolo di dipendenza funzionale **generalizza** il vincolo di chiave.
Più precisamente, una DF $Y \rightarrow Z$ su uno schema $R(X)$ **degenera in un vincolo di chiave** se $Y \cup Z = X$. In tal caso, infatti, Y è (super)chiave per lo schema $R(X)$.

Facciamo ora la seguente osservazione:

$$Impiegato \rightarrow Stipendio$$
$$Progetto \rightarrow Bilancio$$

In queste due DF, la prima componente **non corrisponde ad una chiave e causa anomalie**.

$$Impiegato \ \ Progetto \rightarrow Funzione$$

In questa DF invece, la prima componente **corrisponde ad una chiave e non causa anomalie**.

Questo perchè *Impiegato* e *Progetto* formano una superchiave, che (per definizione) non può contenere due tuple uguali. Perciò, la terza DF non genera mai ridondanze.
Le anomalie perciò sono causate dalle DF $Y \rightarrow Z$ tali che Y non contiene una chiave.

Normalizzazione

La **normalizzazione** è il processo di decomposizione di relazioni "cattive" insoddisfacenti scomponendo i loro attributi in relazioni più piccole.
Una **forma normale** è una forma raggiunta da una relazione che è sotto determinate condizioni che considerano chiavi e dipendenze funzionali.
Ci sono diversi versioni di forme normali:

- 2NF, 3NF, BCNF
  Condizioni basate su chiavi e dipendenze funzionali
- 4NF
  Condizioni basate su chiavi, dipendenze multivalued e MVD
- 5NF
  Condizioni basate su chiavi e join dependency

Tramite la normalizzazione si ottengono progettazioni di alta qualità e che soddisfino le proprietà desiderabili.
Il progettista del database non deve per forza raggiungere la più alta forma di normalizzazione possibile, le più comuni sono 3NF e BCNF (già la 4NF è raramente usata nella pratica)

Prima forma normale

La prima forma normale non permette:

- Attributi composti
- Attributi multivalore

- Relazioni annidate (ovvero che una relazione comprende una serie di attributi che rappresentano un altro oggetto, che non è atomico per quella relazione)

Quindi gli unici attributi permessi da 1NF sono valori **atomici** (o **indivisibili**)

DEPARTMENT

DNAME	DNUMBER	DMGRSSN	DLOCATIONS
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

- L'idea è di rimuovere DLOCATIONS che viola la 1NF e porlo in una relazione separata insieme con la chiave primaria.
  La chiave primaria di questa relazione è la combinazione di {DNUMBER, DLOCATION}.

DEPARTMENT

DNAME	DNUMBER	DMGRSSN
Research	5	333445555
Administration	4	987654321
Headquarters	1	888665555

DEPT_LOCATIONS

DNUMBER	DLOCATION
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

Seconda forma normale

Definiamo:

- **Attributo primo**
  Un attributo che è membro della primary key K
- **Dipendenza funzionale "piena"** (full functional dependency)
  Una dipendenza funzionale $Y \rightarrow Z$ è piena se rimuovendo qualsiasi attributo da Y la dipendenza funzionale non è più valida (altrimenti la dipendenza funzionale si dice **parziale**).

Uno schema relazionale R è in **seconda forma normale** se R è in prima forma normale e tutti gli attributi non primi sono pienamente funzionalmente dipendenti dalla primary key di R.
La definizione generale di seconda forma normale non riguarda solo la chiave primaria ma tutte le chiavi candidate, e di conseguenza tutti gli attributi non primi devono essere pienamente funzionalmente dipendenti da ognuna di queste chiavi.
Se R non è in seconda forma normale può essere diviso in più relazioni in seconda forma normale, in cui gli attributi non primi sono associati solo con la parte di chiave primaria da cui sono funzionalmente dipendenti.

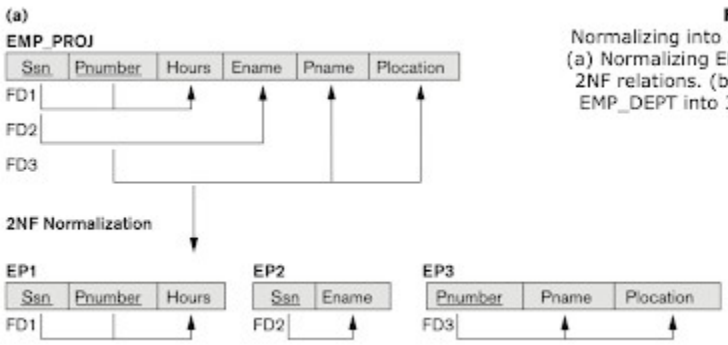


Figure 14.11
Normalizing into 2NF and 3NF. (a) Normalizing EMP_PROJ into 2NF relations. (b) Normalizing EMP_DEPT into 3NF relations.

Terza forma normale

Una dipendenza funzionale $X \rightarrow Z$ è detta **transitiva** se esiste un insieme di attributi Y che non è sottoinsieme di alcuna chiave in R e valgono $X \rightarrow Y$ e $Y \rightarrow Z$
Uno schema relazionale R è in **terza forma normale** se è in seconda forma normale e nessun attributo non primo di R è transitivamente dipendente dalla primary key di R.
La definizione generale di terza forma normale non riguarda solo la chiave primaria ma tutte le chiavi candidate, e di conseguenza nessun attributo non primo di R deve essere transitivamente dipendente da nessuna chiave di R.

Uno schema relazionale R è in terza forma normale se per ogni dipendenza funzionale $X \rightarrow A$ vale:

- X è superchiave di R, oppure
- A è un attributo primo di R

Oppure se ogni attributo non primo di R rispetta le seguenti condizioni:

- È pienamente funzionalmente dipendente da ogni chiave di R
- È dipendente non-transitivamente da ogni chiave di R

In sintesi:

- 1NF
  Tutti gli attributi dipendono dalla chiave
- 2NF
  Tutti gli attributi dipendono da tutta la chiave
- 3NF
  Tutti gli attributi dipendono solo dalla chiave

Forma normale	Test	Normalizzazione
1NF	La relazione non deve presentare attributi non atomici.	Forma una nuova relazione per ogni attributo non atomico.
2NF	La chiave primaria contiene attributi multipli e attributi non chiave sono funzionalmente dipendenti da parte della chiave primaria.	Crea una relazione per ogni chiave parziale con i suoi attributi dipendenti. (mantieni una relazione per la chiave primaria e gli attributi che dipendono funzionalmente da questa)
3NF	Non deve esistere una dipendenza transitiva tra un attributo non chiave e la chiave primaria.	Decomponi la relazione in un insieme di relazioni che includono gli attributi non chiave che funzionalmente determinano altri attributi non chiave.

Forma normale di Boyce e Codd

Vediamo ora la nozione di **forma normale di Boyce e Codd**.
Una relazione r è in forma normale di Boyce e Codd (**BCNF**) se, per ogni dipendenza funzionale (non banale) $X \to Y$ definita su di essa, X contiene una chiave K di r, ovvero X è una superchiave di r.
La BCNF richiede quindi che i concetti in una relazione siano omogenei (quindi che contenga solo proprietà/attributi direttamente associate alla chiave).
Se una relazione r non soddisfa la BCNF, la rimpiazziamo con altre relazioni che soddisfano la BCNF, **decomponendo** r **sulla base delle dipendenze funzionali**, al fine di separare i concetti.

Una procedura intuitiva per la normalizzazione (non valida in generale ma solo nei casi semplici, come vedremo dopo) è la seguente:
Per ogni dipendenza funzionale $X \to Y$ (con diverso primo membro) che viola la BCNF (quindi che X non è una superchiave), bisogna definire una relazione su XY ed eliminare Y dalla relazione originaria.
Purtroppo con questa tecnica non sempre le decomposizioni sono di buona qualità (ad esempio rischiamo di perdere qualche vincolo / qualche dipendenza funzionale).
Ecco un esempio di decomposizione corretta:

Un esempio di decomposizione

Le tre relazioni, ottenute decomponendo la relazione originaria secondo le DF, sono in BCNF.

Impiegato	Stipendio	Progetto	Bilancio	Funzione
Rossi	20	Marte	2	tecnico
Verdi	35	Giove	15	progettista
Verdi	35	Venere	15	progettista
Neri	55	Venere	15	direttore
Neri	55	Giove	15	consulente
Neri	55	Marte	2	consulente
Mori	48	Marte	2	direttore
Mori	48	Venere	15	progettista
Bianchi	48	Venere	15	progettista
Bianchi	48	Giove	15	direttore

Progetto → Bilancio

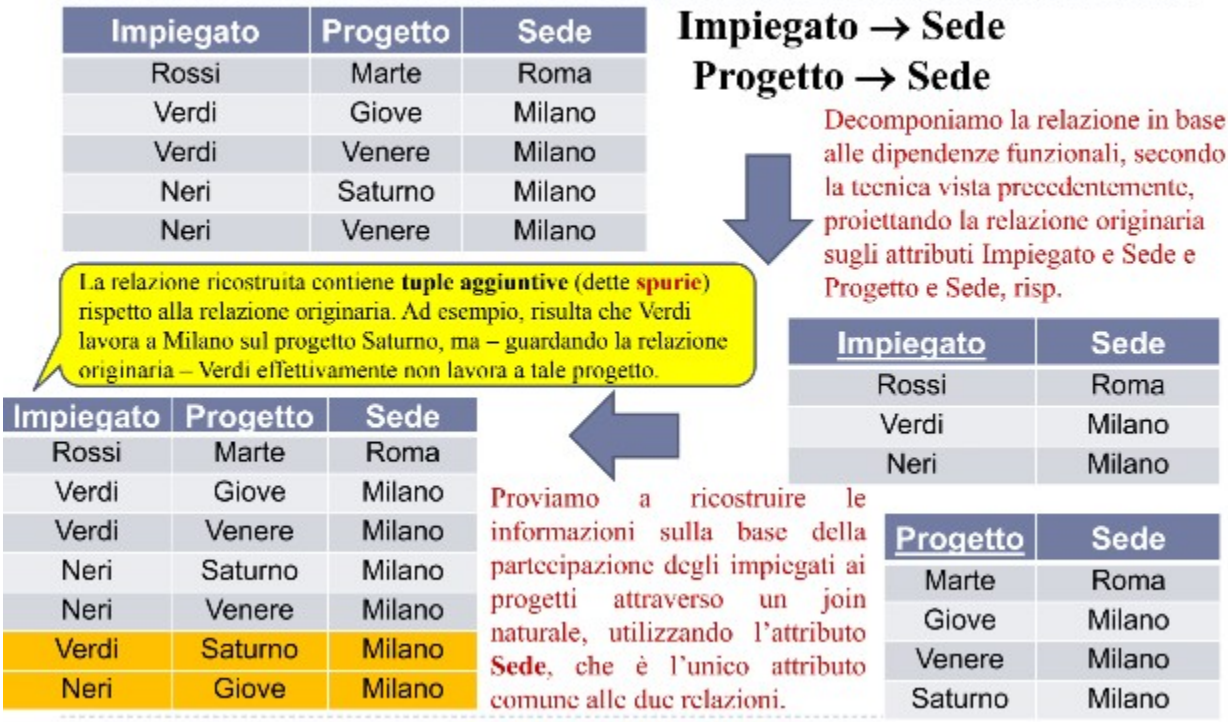
Progetto	Bilancio
Marte	2
Giove	15
Venere	15

Si noti che abbiamo decomposto la relazione originaria in modo che a ciascuna dipendenza corrisponda una diversa relazione la cui **chiave** è proprio il **primo membro della dipendenza stessa**.

Impiegato → Stipendio

Impiegato	Stipendio
Rossi	20
Verdi	35
Neri	55
Mori	48
Bianchi	48

Bisogna stare attenti a fare bene la decomposizione, perché se viene fatta male rischiamo di alterare le informazioni che abbiamo, chiamata **decomposizione con perdita**, ecco un esempio:



Una relazione *r* si decompone senza perdita su X_1 e X_2 se il JOIN delle proiezioni di *r* su X_1 e X_2 è uguale a *r* stessa (ovvero non contiene **tuple spurie**).
È un requisito **irrinunciabile** il fatto che una decomposizione sia effettuata senza perdita, altrimenti i nostri dati perdono validità.

È possibile individuare una condizione che garantisce la **decomposizione senza perdita** di una relazione:
Sia *r* una relazione su un insieme di attributi X e siano X_1 e X_2 due sottoinsiemi di X tali che $X = X_1 \cup X_2$. Sia inoltre $X_0 = X_1 \cap X_2$.
Allora *r* **si decompone senza perdita** su X_1 e X_2 se soddisfa la dipendenza funzionale $X_0 \rightarrow X_1$ oppure $X_0 \rightarrow X_2$ (questa condizione è sufficiente ma non necessaria).
In altre parole, la **decomposizione senza perdita** è garantita se gli attributi comuni nelle relazioni decomposte **contengono una chiave** per almeno una delle relazioni decomposte.
Ecco un esempio:

Decomposizione senza perdita

Impiegato	Progetto	Sede
Rossi	Marte	Roma
Verdi	Giove	Milano
Verdi	Venere	Milano
Neri	Saturno	Milano
Neri	Venere	Milano

Impiegato → Sede
Progetto → Sede

Decomponiamo la relazione originaria **r** sfruttando solamente la **DF Impiegato → Sede,** tenendo conto che nella relazione decomposta Impiegato è **chiave** per la proiezione di **r** sugli attributi Impiegato e Sede.

Impiegato	Sede
Rossi	Roma
Verdi	Milano
Neri	Milano

La relazione ricostruita è identica alla relazione originaria.

E' importante notare che la tecnica appena analizzata è **sufficiente** ma **non necessaria** per la decomposizione senza perdita. Esistono infatti istanze di relazioni che **non soddisfano nessuna delle due dipendenze**, ma al tempo stesso si decompongono senza perdita.

Proviamo a ricostruire le informazioni attraverso un join naturale, utilizzando l'attributo **Impiegato**, che è l'unico attributo comune alle due relazioni.

Impiegato	Progetto
Rossi	Marte
Verdi	Giove
Verdi	Venere
Neri	Saturno
Neri	Venere

Ora però abbiamo un nuovo problema, ovvero che separando le due tabelle dell'esempio precedente "perdiamo" un vincolo, ovvero quello della dipendenza *Progetto → Sede*:

Un ulteriore problema

Impiegato	Progetto	Sede
Rossi	Marte	Roma
Verdi	Giove	Milano
Verdi	Venere	Milano
Neri	Saturno	Milano
Neri	Venere	Milano



Impiegato → Sede
Progetto → Sede

Impiegato	Sede
Rossi	Roma
Verdi	Milano
Neri	Milano

Impiegato	Progetto
Rossi	Marte
Verdi	Giove
Verdi	Venere
Neri	Saturno
Neri	Venere

- Nonostante la decomposizione ottenuta sia senza perdita, essa presenta un nuovo incoveniente…
- Supponiamo di voler inserire una nuova tupla che specifica la partecipazione dell'impiegato Neri, che opera a Milano, al progetto Marte.

Impiegato	Progetto
Rossi	Marte
Verdi	Giove
Verdi	Venere
Neri	Saturno
Neri	Venere
Neri	**Marte**

- Sulla relazione originaria un tale aggiornamento verrebbe individuato come **illecito**, perché porterebbe ad una violazione della dipendenza **Progetto → Sede**.
- Sulla relazione decomposta **ImpiegatoProgetto**, invece, non è possibile definire alcuna dipendenza funzionale e perciò rilevare alcuna violazione di dipendenze.

Possiamo quindi dire che una scomposizione **conserva le dipendenze** se ciascuna delle dipendenze funzionali dello schema originario **coinvolge attributi che compaiono tutti insieme in uno degli schemi decomposti**. Solo in questa maniera si può garantire sullo schema composto il soddisfacimento degli stessi vincoli dello schema originario.

Una decomposizione quindi deve sempre soddisfare:

- La **decomposizione senza perdita**
  Ovvero che garantisca la ricostruzione delle informazioni originarie (cioè senza informazioni spurie) a partire da quelle rappresentate nelle relazioni decomposte
- La **conservazione delle dipendenze**
  Che garantisce il mantenimento dei vincoli di integrità originari, con le relazioni decomposte che hanno la stessa capacità della relazione originaria di rappresentare i vincoli di integrità e quindi di rilevare aggiornamenti illeciti.

Ovviamente sono possibili sulle relazioni decomposte ulteriori aggiornamenti, legati ai singoli concetti rappresentati in ciascuna di esse, che non hanno un corrispettivo sulla relazione originaria, senza però corrispondere a violazione dei vincoli.

Dato uno schema che viola una forma normale, l'attività di **normalizzazione** è quindi volta a ottenere una decomposizione che sia **senza perdita**, **conservi le dipendenze** e che **contenga relazioni in forma normale**.

Terza forma normale

Ora però notiamo che ci sono casi in cui è impossibile raggiungere la forma normale di Boyce e Codd rispettando sia la decomposizione senza perdita che la conservazione delle dipendenze (come nell'esempio appena visto), ecco un altro esempio:

- Talvolta non è possibile raggiungere una buona decomposizione in forma normale di Boyce e Codd.

Dirigente	Progetto	Sede
Rossi	Marte	Roma
Verdi	Giove	Milano
Verdi	Marte	Milano
Neri	Saturno	Milano
Neri	Venere	Milano

Supponiamo che siano definite le seguenti dipendenze:
Dirigente → Sede (ogni dirigente opera presso una sede)
Progetto Sede → Dirigente (per ogni sede, un progetto ha un solo responsabile).

- La relazione non è in BCNF, perché il primo membro della DF **Dirigente → Sede** non è superchiave.
- Allo stesso modo, si può notare come non sia possibile alcuna buona decomposizione di questa relazione: infatti, la DF **Progetto Sede → Dirigente** coinvolge tutti gli attributi e quindi nessuna decomposizione è in grado di conservarla.
- Perciò, talvolta la **BCNF non è raggiungibile.**

Introduciamo quindi una nuova forma normale, per poter trattare casi come quello precedente, che è meno restrittiva delle BCNF, ovvero le **terza forma normale**.
Una relazione *r* è in terza forma normale se, per ogni DF (non banale) $X \rightarrow Y$ definita su *r*, è verificata almeno una delle seguenti condizioni:

- X contiene una chiave K di *r*
- Ogni attributo in Y è contenuto in almeno una chiave di *r*

Di conseguenza tornando al nostro esempio:

Dirigente	Progetto	Sede
Rossi	Marte	Roma
Verdi	Giove	Milano
Verdi	Marte	Milano
Neri	Saturno	Milano
Neri	Venere	Milano

Dirigente → Sede
Progetto Sede → Dirigente

- La relazione non soddisfa la BCNF, ma soddisfa la 3FN. Infatti, **Progetto Sede → Dirigente** ha come primo membro una chiave della relazione, mentre **Dirigente → Sede**, pur non contenendo una chiave al primo membro, ha un unico attributo al secondo membro che fa parte della chiave **Progetto Sede.**
- Si osservi che la relazione **presenta una forma di ridondanza**. Ogni volta che un Dirigente compare in una tupla, viene ripetuta per esso la Sede in cui opera.
- Questa ridondanza viene però **tollerata** dalla 3FN,perché non sarebbe possibile una decomposizione in BCNF che elimini tale ridondanza e al tempo stesso conservi tutte le dipendenze.

La 3FN è meno restrittiva della forma normale di Boyce e Codd e quindi non offre le medesime garanzie di qualità per una relazione; ha però il vantaggio di essere **sempre ottenibile**.
È possibile infatti dimostrare che una qualunque relazione che non soddisfa la 3FN è certamente **decomponibile senza perdita e con conservazione delle dipendenze** in relazioni in 3FN.
Una relazione che non soddisfa la 3FN si decompone in tante relazioni ottenute per proiezione sugli attributi corrispondenti alle dipendenza funzionali, con l'unica accortezza che alla fine **almeno una delle relazione decomposte contenga una chiave della relazione originaria**.

Dirigente	Progetto	Stipendio
Rossi	Marte	30
Verdi	Giove	30
Verdi	Venere	30
Neri	Saturno	30
Neri	Venere	30

Dirigente → Stipendio

- La relazione non soddisfa la 3FN. Infatti, l'unica DF **Dirigente → Stipendio** ha come primo membro un attributo non chiave, e come secondo membro un attributo che non partecipa a nessuna chiave della relazione.
- Una decomposizione in una relazione negli attributi **Dirigente Stipendio** e in un'altra sull'attributo **Progetto** violerebbe la proprietà di decomposizione senza perdita, in quanto nessuna delle due relazioni contiene una chiave.
- Per garantire tale proprietà, dobbiamo invece definire la seconda relazione sugli attributi **Dirigente Progetto,** che formano una chiave della relazione originaria.
- **Le due relazioni ottenute sono in 3FN!**

Dirigente	Stipendio
Rossi	30
Verdi	30
Neri	30

Dirigente	Progetto
Rossi	Marte
Verdi	Giove
Verdi	Venere
Neri	Saturno
Neri	Venere

Chiusure

Chiusura di un insieme di dipendenze funzionali

Come abbiamo detto la dipendenza funzionale è una proprietà semantica degli attributi.
Di conseguenza una dipendenza funzionale non può essere inferita automaticamente da un'estensione di relazione, ma deve essere definita esplicitamente da chi conosce la semantica degli attributi.

Sia F l'insieme delle dipendenze funzionali (DF) di uno schema di relazione R. Il disegnatore dello schema specifica le dipendenze funzionali semanticamente ovvie F.
In tutte le istanze di relazione legali però, valgono numerose altre dipendenze funzionali, le quali possono essere **inferite** o **dedotte** da F.
È possibile calcolare queste altre DF che sono logicamente implicate da F. Ad esempio se $X \rightarrow Y$ e $Y \rightarrow Z$ allora $X \rightarrow Z$.

Un insieme F di DF **implica logicamente** una DF $X \rightarrow Y$ se ogni istanza valida che soddisfa F soddisfa anche $X \rightarrow Y$.
L'insieme di tutte le DF logicamente implicate da F si chiama la **chiusura** di F e viene indicata con la notazione F^{+}.

- F = {
  - SSN → { ENAME, BDATE, ADDRESS, DNUMBER},
  - DNUMBER → {DNAME,DMGRSSN}
    }
- Possiamo inferire:
  - SSN → {DNAME,DMGRSSN}
  - SSN → SSN
  - DNUMBER → DNAME
  - …
- F ⊨ X→Y denota che la FD X→Y è inferita da F.

Abbiamo le seguenti notazioni:

- $\{X, Y\} \rightarrow Z$ è abbreviato in $XY \rightarrow Z$
- $\{X, Y, Z\} \rightarrow \{U, V\}$ è abbreviato in $XYZ \rightarrow UV$

Vediamo ora come calcolare F^+.

F^+ può essere calcolato mediante gli **assiomi di Armstrong** (AA):

- **Assioma di riflessività**: Se $Y \subseteq X$, allora $X \rightarrow Y$
- **Assioma di aumento**: Se $X \rightarrow Y$, allora $XZ \rightarrow YZ$
- **Assioma di transitività**: Se $X \rightarrow Y$ e $Y \rightarrow Z$, allora $X \rightarrow Z$

Questi assiomi sono:

- **Corretti** (sound)
  Se gli AA generano una nuova DF f a partire da un insieme F di DF, allora f è logicamente implicata da F ed è vera in ogni stato che soddisfa F
- **Completi** (complete)
  Se una DF f è logicamente implicata da F, allora gli AA sono in grado di generarla.
  In generale la chiusura di F può essere determinata usando solo gli AA.

In altre parole, visto che sono corretti e completi, gli AA sono in grado di generare **TUTTE** e **SOLE** le dipendenze funzionali che sono logicamente implicate da F.

Tipicamente il progettista del db prima specifica le dipendenze funzionali F a partire dalla semantica degli attributi e poi usa le regole di inferenza di Armstrong per inferire dipendenze funzionali aggiuntive.

Dagli AA si possono derivare altre tre regole:

- **Unione**: Se $X \rightarrow Y$ e $X \rightarrow Z$, allora $X \rightarrow YZ$
- **Scomposizione**: Se $X \rightarrow YZ$, allora $X \rightarrow Y$ e $X \rightarrow Z$
- **Pseudotransitività**: Se $X \rightarrow Y$ e $WY \rightarrow Z$ allora $WX \rightarrow Z$

Vediamo un esempio di applicazione degli assiomi:

- Assumiamo di avere il seguente insieme di DF:
  - F = {**A -> BCDEFGH, CE -> A, BD -> E**}

- Procediamo con l'applicazione degli AA:
  - Transitività:

    CE -> A
    A -> BCDEFGH

    CE -> BCDEFGH

  - Aumento:

    BD -> E

    BDC -> CE

  - Transitività:

    BDC -> CE
    CE -> A

    BDC -> A

Un insieme F di dipendenze funzionali **copre** un altro insieme E di dipendenze funzionali se ogni DF di E è presente anche in F^+, cioè se ogni dipendenza in E è può essere inferita a partire da F.
Per determinare se F copre E bisogna calcolare X^+ (definizione prossimo capitoletto) rispetto ad F per ogni dipendenza funzionale $X \rightarrow Y$ in E, e quindi verificando se questo X^+ comprende gli attributi presenti in Y
Due insiemi E ed F di dipendenze funzionali sono **equivalenti** se $E^+ = F^+$, ossia E è equivalente ad F (e viceversa) se E copre F e F copre E.

Chiusura di un insieme di attributi

Per trovare le superchiavi di una relazione R, è necessario introdurre la nozione di **chiusura di un insieme di attributi**.
Dato un insieme di DF F e un insieme X di attributi, la **chiusura** di X, denotata da $(X)^+$, è definita come l'insieme degli attributi che sono funzionalmente dipendenti da X.

L'algoritmo per il calcolo di $(X)^+$ è il seguente:

1. Poniamo $(X)^+ = X$
2. Se esiste una DF $Z \rightarrow W$ in F dove $Z \subseteq (X)^+$, si aggiunga W a $(X)^+$
3. Ripetere la procedura finchè non è possibile aggiungere più nulla a $(X)^+$

O detta in un'altra maniera:

- determinare ogni insieme di attributi che appare come parte sinistra di qualche dipendenza funzionale in F.

- usare le regole di Armstrong per determinare l'insieme di tutti gli attributi dipendenti da X.

```
X+:= X;
repeat
   oldX+ := X+;
   for each functional dependency Y → Z in F do
       if X+ ⊇ Y
       then X+ := X+ ∪ Z;
until (oldX+ = X+);
```

Vediamo un esempio per capire meglio:

- Sia F = {**A -> B**, **B -> C**, **AC -> D** }
- Qual è la chiusura di A (ovvero $(A)^+$)?
- Applichiamo l'algoritmo di chiusura:
  - Poniamo $(X)^+$ = A
  - Deriviamo B (applicando la DF **A -> B**)
  - Poniamo $(X)^+$ = AB
  - Deriviamo C (applicando la DF **B -> C**)
  - Poniamo $(X)^+$ = ABC
  - Deriviamo D (applicando la DF **AC -> D**)
  - Poniamo $(X)^+$ = ABCD
- La chiusura di A sarà l'insieme di attributi ABCD

Vediamo ora come **verificare** se un insieme di attributi è superchiave di una relazione.
Usiamo la chiusura di un insieme di attributi X per verificare se X è una superchiave di una relazione R.
Il metodo è il seguente:

1. Partiamo da una relazione R con attributi U e un insieme di dipendenze funzionali F: $<R(U),\ F>$
2. Calcoliamo $(X)^+$ e verifichiamo se $(X)^+ = U$, ovvero se vale $X \rightarrow U$
3. Se vale, allora X è una superchiave per $<R(U),\ F>$, altrimenti non lo è

NB: L'insieme di attributi X potrebbe non essere una chiave minimale.

Vediamo un esempio pratico:

- Sia < R(A,B,C,D), F >, dove F = {**A -> B**, **B -> C**, **AC -> D** }
  - $(A)^+$= ABCD

    A è una superchiave (di fatto una chiave, visto che è costituito da un solo attributo)
  - $(AC)^+$ = ACBD

    AC è una superchiave, ma non è una chiave (non è minimale)
  - $(B)^+$ = BC

    B non è una superchiave (e *a fortiori* nemmeno una chiave)

Vediamo ora l'algoritmo che ci permette di **identificare** le superchiavi di una relazione R.

1. Determinare gli attributi di R che non sono né sul lato sinistro né sul lato destro di una DF (attributi *Isolati*)
2. Determinare gli attributi che sono solo sul lato sinistro di una qualsiasi DF (attributi *Left*)
3. Determinare gli attributi che sono solo sul lato destro di una qualsiasi DF (attributi *Right*)
4. Combinare gli attributi *Isolati* e *Left* (Step 1 e Step 2)
5. Calcolare la chiusura degli attributi ottenuti nello Step 4
   - se la chiusura degli attributi coincide con tutti gli attributi di R, allora gli attributi ottenuti nello Step 4 costituiscono la sola chiave candidata e l'algoritmo termina
   - altrimenti si continua con gli altri step
6. Individuare gli attributi che non sono stati considerati negli Step 3 e 4 (che quindi sono su entrambi i lati, *Both*)
7. Calcolare la chiusura degli attributi ottenuti nello Step 4 (se ce ne sono) e tutte le combinazioni di attributi ottenuti nello Step 6 e determinare le chiusure minimali di attributi che sono uguali a R (ovvero appena vedo che c'è una chiusura uguale a R di n attributi, cerco se ce ne sono altre di n attributi e poi mi fermo)

Ecco un esempio:

- Sia < R(A,B,C,D,E), F >, dove F = {**A -> B, BC -> E, ED ->A** }
- Step 1 (*Isolati*): { }
- Step 2 (*Left*): { C, D }
- Step 3 (*Right*): { }
- Step 4 (unione di *I* e *L*): { C, D }
- Step 5 (chiusura): $(CD)^+$= CD (non è una chiave)
- Step 6: { A, B, E }
- Step 7:

  $(ACD)^+$ = ACBDE (chiave candidata)

  $(BCD)^+$ = ACBDE) (chiave candidata)

  $(CDE)^+$ = ACBDE (chiave candidata)

- Aggiungere altri attributi produrrebbe solo superchiavi, per cui l'algoritmo termina
- ACD, BCD e CDE sono le sole chiavi candidate

Copertura minimale

Il calcolo di una chiusura di un insieme F di DF può essere estremamente dispendioso, dato che potrebbe richiedere di derivare un numero esponenziale di DF.
Per ovviare a questo problema, è stata introdotta una versione più semplice (ma logicamente equivalente) di un insieme F di DF chiamata **copertura minimale**

Una **copertura minimale** di F è un insieme di DF F_{min} tale che:

- $F_{min}{}^{+} = F^{+}$ (ovvero i due insiemi sono logicamente equivalenti, dato che la loro chiusura transitiva è identica)
- Tutte le DF in F_{min} sono della forma $X \rightarrow A$
- Se rimuoviamo una DF o un attributo dal lato sinistro di una DF in F_{min}, allora F_{min} non è più equivalente a F, ovvero $F_{min}{}^{+} \neq F^{+}$

In altri termini, F_{min} contiene una quantità minima di informazione per descrivere tutte le DF logicamente implicate da F (visto che le chiusure sono uguali)
È possibile dimostrare che uno schema $<R(U),\ F>$ è in BCNF se e solo se le condizioni per essere in BCNF sono soddisfatte da F_{min}. Per questa ragiona possiamo limitarci a considerare F_{min}.

Per calcolare la copertura minimale F_{min} è sufficiente applicare i seguenti passi:

1. Per ogni DF della forma $X \rightarrow ABC\ldots$
   Normalizziamola in un insieme di DF nella forma $X \rightarrow A$, $X \rightarrow B$, $X \rightarrow C$, …

2. Per ogni DF della forma $XA \rightarrow B$
   Se $A \in (X)^+$, allora A è ridondante e può essere rimossa, e quindi $X \rightarrow B$
3. Per ogni DF della forma $X \rightarrow A$
   Se $A \in (X)_{-X \rightarrow A}{}^+$, allora $X \rightarrow A$ è ridondante e può essere rimossa

Ecco un esempio:

Sia < R(A,B,C,D,E), F>, con F = { **A -> BCE**, **CDB -> A**, **CD -> E**, **E -> B** }

- Normalizzazione:
  - Da **A -> BCE** ricaviamo **A -> B**, **A -> C**, **A -> E**,
  - Otteniamo l'insieme di DF { **A -> B**, **A -> C**, **A -> E**, **CDB -> A**, **CD -> E**, **E -> B** }

- Rimuoviamo gli attributi ridondanti dal lato sinistro delle DF:
  - $(CD)^+$ = CDE**B**, per cui B è ridondante
  - $(C)^+ = C$ (niente da fare)
  - $(D)^+ = D$ (niente da fare)

- Rimuoviamo le DF ridondanti:
  - Da A è possibile derivare B senza usare **A -> B** e da CD posso ricavare E senza usare **CD -> E**. Quindi rimuoviamo **A -> B** e **CD -> E**

$$F_{min} = \boldsymbol{A \rightarrow C, A \rightarrow E, CD \rightarrow A, E \rightarrow B}$$

Vediamo come ultimo l'algoritmo per la scomposizione in BCNF, dato in input $< R(U),\ F >$ (dove F è una copertura minimale) viene dato in output $< R(U),\ F >$ in BCNF senza perdita di informazione.
L'algoritmo è il seguente:

1. Si scelga una DF $X \rightarrow A$ in F che violi le condizioni della BCNF
2. Si calcoli $Y = (X)^+$ ¿ e $Z = U$ ¿
3. Si costruiscano i due seguenti schemi di relazione: $< R_1(ZX),\ (\Pi_{ZX} F^+)_{min} >$ e $< R_1(XY),\ (\Pi_{XY} F^+)_{min} >$
4. Se uno dei due schemi non è ancora in BCNF, riapplicare questo algoritmo

Ecco un esempio:

- < R(A,B,C), F > dove F = { **A -> B**, **B -> C** }
- F è già una copertura minimale
- La sola chiave è A (è l'unico attributo che determina funzionalmente tutti gli altri)
- **A -> B** soddisfa la condizione della BCNF, ma **B -> C** no
- Calcoliamo tutti gli attributi che si possono derivare da B: $(B)^+ = BC$
- Separiamo quindi R in due relazioni:
  - una relazione ha come attributi $(B)^+$=BC,
  - l'altra contiene gli attributi rimanenti, cioè A
  - B deve rimanere in entrambe le relazioni per permettere il join delle due relazioni ottenute

$$< R_1(B,C), B \rightarrow C >, < R_2(A,B), A \rightarrow B >$$

- Entrambi gli schemi sono ora in BCNF e quindi la procedura termina

Modello di costo e Metodi di storage

Memoria centrale e secondaria

Per capire come i dati di un database relazionale vengano memorizzati, è necessario prima ricordare come i diversi tipi di memoria utilizzati dai moderni computer sono messi in relazione tra di loro.

Un computer ha fondamentalmente due tipi di memoria.
La **memoria centrale** (la RAM), che è solitamente dell'ordine dei GB, ed è estremamente veloce (Si possono leggere anche fino a 3.2 GB al secondo dalla RAM).
Poi abbiamo la **memoria secondaria**. Questa è solitamente costituita dai cosiddetti hard disk, che sono dispositivi meccanici (e quindi lenti), in cui i dati vengono memorizzati un bit alla volta su un piatto di metallo. Il piatto è suddiviso in cerchi concentrici, chiamate tracce, e su ogni cerchio, ciascun punto del cerchio viene magnetizzato con una certa polarità per memorizzare il valore 0 o il valore 1.
Questa operazione di magnetizzazione è effettuata da una punta che si può muovere dall'esterno verso l'interno (e viceversa) del piatto, mentre il piatto è in rotazione.
Chiaramente, questo meccanismo è molto più lento della RAM, ma permette di immagazzinare molte più informazioni. Le dimensioni sono dell'ordine dei TB (migliaia di GB), ma è possibile leggere fino a un massimo di 500 MB al secondo.

Poi c'è la **memoria terziaria** che sono i media rimovibili (flash memory).

Type	Capacity*	Access Time	Max Bandwidth	Commodity Prices (2014)**
Main Memory- RAM	4GB–1TB	30ns	35GB/sec	$100–$20K
Flash Memory- SSD	64 GB–1TB	50µs	750MB/sec	$50–$600
Flash Memory- USB stick	4GB–512GB	100µs	50MB/sec	$2–$200
Magnetic Disk	400 GB–8TB	10ms	200MB/sec	$70–$500
Optical Storage	50GB–100GB	180ms	72MB/sec	$100
Magnetic Tape	2.5TB–8.5TB	10s–80s	40–250MB/sec	$2.5K–$30K
Tape jukebox	25TB–2,100,000TB	10s–80s	250MB/sec–1.2PB/sec	$3K–$1M+

È chiaro quindi che i nostri database devono risiedere in memoria secondaria, viste le dimensioni che questi possono raggiungere e visto che ci serve una memoria persistente (e non volatile). Però questo porta ad avere che ogni volta che vogliamo accedere ai dati del nostro database incorreremo in costi elevati in termini di tempo richiesto. Ovviamente, leggere una volta tutto il database in RAM non è una soluzione, semplicemente perché non avremmo abbastanza spazio. Il DBMS dovrà quindi limitare al minimo gli accessi alla memoria secondaria, cercando di mantenere in memoria centrale i dati più rilevanti, e per tutto il tempo necessario, prima di eliminarli e fare spazio ad altri dati.

Analizziamo meglio questo aspetto. Per questioni di efficienza e di gestione degli errori, gli hard disk moderni, quando viene loro chiesto di leggere dei dati, non si limitano a leggere un bit alla volta, ma essi leggeranno sempre i dati in blocchi, solitamente grandi 512 byte. Ciascun blocco è chiamato **settore** del disco.
Il motivo per cui il disco è così strutturato è dovuto al fatto che ogni volta che chiediamo al disco di leggere qualcosa, esso dovrà spostare fisicamente la testina sulla traccia corretta.
Questa operazione richiede parecchio tempo, e sprecare tale movimento per leggere solo un bit non è una buona idea. Quindi, una volta che la testina si trova sul punto giusto, leggerà non un bit ma una sequenza di 512 byte a partire da quel punto. Questa operazione non costa molto, perchè si tratta di aspettare che il piatto giri sotto la testina per farle leggere i dati richiesti.
Visto che i settori sono disposti uno dopo l'altro su ogni traccia, possiamo immaginare il nostro disco come un grandissimo array dove ogni elemento è un settore, che è l'unità minima che possiamo leggere e scrivere.



Modelli di costo

Immaginiamo di avere una relazione R che memorizza persone nel seguente modo:

id	name	city	age
1	Marco	Trento	33
2	Andrea	Cosenza	24
3	Giuseppe	Milano	27
4	Federica	Trento	20
5	Rossana	Cosenza	18
6	Laura	Trento	25

La prima riga della tabella occupa 4 *byte* per l'id (un int), 5 *byte* per il name, 6 *byte* per la city e 4 *byte* per l'age, quindi in totale la riga occupa 19 *byte*.

Un **record** (o tupla) è una collezione di dati correlati tra loro, in questo caso ogni riga rappresenta un record.
Per semplificare l'analisi, si usa assumere che tutti i record abbiano la stessa dimensione (anche se non è così, campi di lunghezza variabile, campi opzionali, record di tipo differente).
La dimensione in byte di un record di una relazione R si indica con t_R.

È chiaro che la dimensione di un settore consente di memorizzare molti record tutti insieme.
Quindi il DBMS organizza logicamente il disco raggruppando più settori in una cosiddetta **pagina**. Per Postgres la dimensione di una pagina è 8KB (quindi 16 settori). Ciò viene fatto sempre seguendo il principio che una volta che facciamo fare lo sforzo al disco di muoversi su una determinata area, tanto vale leggere quanti più dati possibile, e solitamente possiamo permetterci di mantenere in memoria centrale blocchi di dati dell'ordine degli 8KB. In generale, usiamo P per indicare la dimensione in byte di una pagina.
Qualunque operazione il DBMS voglia fare sul disco (lettura o scrittura) sarà effettuata quindi accedendo ad una pagina per intero, e mai una sua parte.

Quando il DBMS deve eseguire delle interrogazioni, dovrà svolgere varie operazioni: sia di lettura/scrittura su disco, ma anche altre operazioni in memoria centrale. Ma come abbiamo detto, le operazioni in memoria centrale sono molto più veloci, e quindi possiamo ignorarle. Ciò che fa davvero perdere tempo al DBMS è l'accesso alla memoria secondaria.
Quindi misureremo il **costo** di effettuare determinate operazioni da parte del DBMS **contando il numero di pagine** che dovrà leggere e scrivere per effettuare quell'operazione.
Introduciamo quindi un modello di costo basato sul numero di pagine lette e scritte della memoria secondaria.

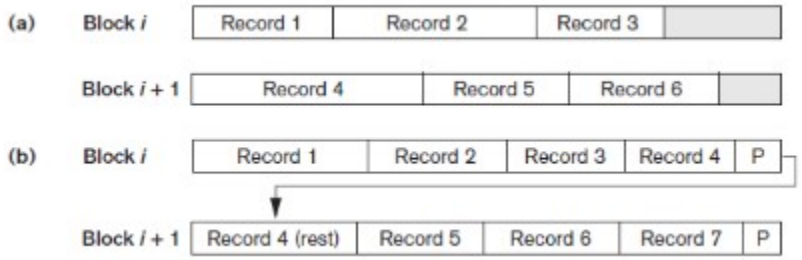
In base al tipo di storage fisico che scegliamo, ci sono certe operazioni che vengono effettuate più velocemente e altre che vengono effettuate più lentamente.
Se noi sappiamo quali saranno le operazioni più comuni sul database, possiamo scegliere adeguatamente il miglior metodo di storage per ottimizzare il costo delle operazioni.

I record possono venir salvati nei settori in due maniere:

- **Spanned**
  Se il record è più grande dello spazio rimasto in un settore, si salva quello che "ci sta" e si aggiunge un puntatore alla prossimo settore, dove sarà memorizzato la fine del record

- **Unspanned**
  Se il record è più grande dello spazio rimasto in un settore, lo si salva completamente in quella successiva.

Il **blocking factor** è il numero medio di record per blocco (settore) del file (della pagina).



Le pagine nel disco possono essere memorizzate:

- In locazioni contigue
- In locazioni “linked”
- In locazioni “indexed”

Il **file header** contiene informazioni che servono al sistema sugli indirizzi del disco e descrive i formati

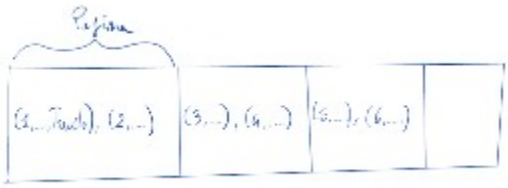
Metodi di allocazione dei dati

Com’è che il DBMS memorizza ciascuna relazione del database su disco?
Abbiamo tre metodi principali per memorizzare una relazione:

- **Heap file**
  Le tuple scritte una dopo l’altra in ordine di inserimento
- **Sorted file**
  Le tuple sono ordinate in base al valore di un attributo
- **Indexed file**
  Viene creato un indice separato rispetto al file che contiene le tuple vere e proprie della relazione

Heap file

Un heap file (anche chiamato unordered file) è un file che viene creato su disco per memorizzare una certa relazione (es. R), in cui le tuple vengono scritte una dopo l’altra man mano che arrivano, e quindi non sono disposte in un ordine particolare. In particolare, le tuple vengono scritte riempiendo per quanto possibile una pagina, prima di passare alla prossima.



Se una pagina non è sufficiente per memorizzare tutte le tuple (quasi sempre è così), allora il DBMS deve creare una nuova pagina, ma non scrive la tupla in eccedenza spezzandola tra due pagine (unspanned). Questo perché ci costringerebbe a leggere due pagine per ottenere quella tupla. Il DBMS preferisce quindi sprecare lo spazio in eccedenza nella pagina precedente, e scrivere la tupla in eccedenza per intero nella pagina successiva (la memoria secondaria è solitamente molto grande, e quindi non è un problema lo spreco di spazio).

In generale, per una relazione R, indichiamo con P_R il numero di pagine usate per immagazzinare tutta la relazione.

Vediamo ora i costi delle varie operazioni con questo metodo di memorizzazione:

SCAN
L'operazione consiste nell'ottenere tutte le tuple dalla relazione R, quindi $SELECT\ *\ ¿\ R;$.
Dobbiamo leggere tutte le pagine del file, perchè dobbiamo restituire tutte le tuple della relazione.
Il costo di uno Scan è quindi P_R

$$Costo_{Scan} = P_R$$

EQUALITY SEARCH
L'operazione consiste nella ricerca di tuple che hanno un certo attributo (o più attributi) uguale a un certo valore. L'attributo (o gli attributi) che vengono usati per effettuare la ricerca viene chiamato **chiave di ricerca**.
Consideriamo la query $SELECT\ *\ ¿\ R\ WHERE\ city = 'Trento';$
Si noti che è possibile vi siano più persone che vivono a Trento, e non abbiamo garanzia su dove queste tuple si trovino nel file. Quindi, per essere sicuri di trovarle tutte, non possiamo fare altro che leggere tutte le pagine della relazione di nuovo.
Quindi, l'Equality Scan su una relazione R ha un costo pari a P_R.

$$Costo_{EqSearch} = P_R$$

INSERT
L'operazione consiste nell'inserire una tupla nella relazione R, quindi consideriamo la query $INSERT\ INTO\ R\ VALUES\ (\ldots)$.
Basta leggere l'ultima pagina dell'Heap in memoria centrale, se c'è spazio per la nuova tupla, la inserisce, e poi sovrascrive la pagina vecchia su disco con la nuova. Se non c'è spazio, allora scrive su disco una pagina nuova con la nuova tupla dentro.
In entrambi i casi, il DBMS dovrà leggere e scrivere in totale due pagine. Il costo di una insert in una relazione R memorizzata tramite Heap è quindi 2.

$$Costo_{Insert} = 2$$

Nota: Qualcuno potrebbe obiettare: "ma come fa il DBMS a sapere dove si trova l'ultima pagina per poterla leggere? Non dovrebbe prima leggerle tutte prima di trovarla?"
La risposta è no, perchè dobbiamo sempre tenere a mente che tutte le pagine hanno la stessa dimensione P e queste stanno tutte in un file. Quindi, al DBMS basta sapere quanto è grande il

file in byte, diciamo S (questo è un valore che può tenere aggiornato man mano che la relazione cambia).

Quindi, $P_R = \frac{S}{P}$ sarà il numero di pagine del file. Quindi ora il DBMS sa che l'ultima pagina è la pagina in posizione $P_R - 1$, e può chiedere al disco di muoversi di conseguenza nel punto richiesto.

DELETE

L'operazione consiste nell'eliminare una tupla dalla relazione R, quindi consideriamo la query $DELETE \ ¿ \ R \ \ WHERE \ \ city = 'Trento';$

Per eliminare le persone con city Trento dobbiamo prima trovare le tuple corrispondenti e poi eliminarle. Per farlo, siamo di nuovo costretti a leggere una pagina per volta dall'Heap, una volta letta la pagina, dobbiamo verificare se la pagina contiene qualche tupla con city = 'Trento', dobbiamo eliminare quelle tuple dalla pagina, e riscrivere la pagina su disco.

Quante pagine dobbiamo leggere? P_R.

E quante pagine dobbiamo riscrivere? Dipende da quante delle pagine che abbiamo letto contengono tuple con city = 'Trento'.

Se sappiamo con esattezza che che in R ci sono 10 tuple con city Trento, possiamo considerare il caso peggiore (in cui ogni tupla è in una pagina diversa) e quindi dovremo scrivere 10 pagine nuove, quindi il costo sarebbe $P_R + 10$.

Il numero di tuple di R con un certo valore c in un attributo A si indica con $|R_{A=c}|$, quindi nel nostro caso abbiamo $|R_{city='Trento'}| = 10$.

Però in generale il DBMS non conosce il numero esatto di tuple con un certo valore in un attributo, ma può sfruttare altre informazioni per risalire a questa informazione (in modo approssimato).

Ad esempio, considerando la relazione R, abbiamo 6 tuple. Indichiamo il numero di tuple in una relazione R con $|R|$. Consideriamo anche il numero di valori distinti su un attributo A, denotato da $|R.A|$, quindi $|R.city| = 3$.

Possiamo quindi concludere che **in media**, ci dovrebbero essere $\frac{|R|}{|R.city|} = \frac{6}{3} = 2$ tuple in R con city pari a una delle 3 città (assumiamo quindi che i valori dell'attributo sono uniformemente distribuiti).

La frazione che abbiamo riportato non è altro che il valore atteso del numero di tuple nella relazione che hanno come valore di city una certa costante c. Quindi possiamo dire che

$$|R_{city='Trento'}| = |R_{city='Cosenza'}| = |R_{city='Milano'}| = 2$$

In generale abbiamo:

$$|R_{A=c}| = \left\lceil \frac{|R|}{|R.A|} \right\rceil .$$

Quindi il costo per fare la delete sarà approssimato a $P_R + |R_{A=c}|$, dove A è l'attributo di R su cui facciamo la ricerca.
Ecco un esempio:

Prendiamo la tabella iniziale:

- $|R.city| = 3$ (perché l'attributo *city* ha 3 valore distinti
- Quindi $\frac{|R|}{|R.city|} = \frac{6}{3} = 2$

[Ovviamente si assume una distribuzione uniforme di valori, che non sempre è garantita].

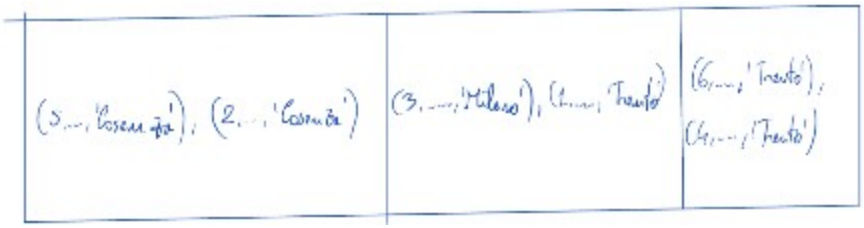
id	name	city	age
1	Marco	Trento	33
2	Andrea	Cosenza	24
3	Giuseppe	Milano	27
4	Federica	Trento	20
5	Rossana	Cosenza	18
6	Laura	Trento	25

$\frac{1}{|R.A|}$ è detto **selectivity factor** (denotato da f) per l'attributo A e indica in sostanza la percentuale di tuple di R che ci aspettiamo di avere con un certo valore sull'attributo A.
Nel nostro esempio il selectivity factor su city è $\frac{1}{3} = 33\,\%$.

$$\text{Costo}_{\text{Delete}} = P_R + |R_{A=c}| = P_R + \lceil f \cdot |R| \rceil.$$

Sorted file

Possiamo rendere alcune delle interrogazioni viste sopra più efficienti? Come possiamo riorganizzare le tuple della nostra relazione per abbassare il costo?
Un metodo funzionante è disporre le tuple in modo ordinato, rispetto a qualche attributo (o insieme di attributi).
Ad esempio, possiamo immaginare di ordinarle per città, usando l'ordine alfabetico. Il file della relazione così ottenuto si chiama **Sorted file** (file ordinato).



SCAN
Per la scan (cioè ottenere tutte le tuple della relazione), non cambia nulla, dobbiamo per forza

leggere tutte le pagine, per prendere tutte le tuple.

$$Costo_{Scan} = P_R$$

EQUALITY SEARCH

Riconsideriamo la query $SELECT \ * \ ¿ \ R \ WHERE \ city = 'Trento';$

Possiamo usare la ricerca binaria. Cioè, iniziamo leggendo la pagina in mezzo al file.

Se il valore 'Trento' è successivo alfabeticamente ai valori delle città delle tuple nella pagina allora le tuple che stiamo cercando stanno nel lato sinistro, escludendo anche la pagina appena letta. Altrimenti, le persone con city 'Trento', se ci sono, devono stare nella parte di sinistra, includendo anche la pagina che abbiamo letto.

Quindi possiamo escludere metà del file. Allora andiamo nella metà della parte rimanente, e facciamo lo stesso. Se le tuple nella pagina hanno tutte una city minore di 'Trento', allora dobbiamo guardare alla parte destra, escludendo anche la pagina letta. E così via, finchè la porzione che ci interessa del file non viene ridotta a una sola pagina. Qui, se la pagina non ha tuple con city = 'Trento' allora vuol dire che non ce ne sono, altrimenti cominciamo a leggere le pagine successive, fino a che le tuple con city = 'Trento' non finiscono.

Se indichiamo con $L(P_R)$ il numero di pagine che dobbiamo leggere con ricerca binaria su un sorted file con P_R pagine, allora

$$L(P_R) = 1 + L\left(\frac{P_R}{2}\right)$$

Ovvero, leggiamo la pagina centrale (1) e poi lavoriamo su metà del file.

Espanendo la formula otteniamo

$$L(P_R) = 1 + 1 + 1 + 1 + \ldots + L\left(\frac{P_R}{2^l}\right)$$

dove l è essenzialmente il numero di passi necessari a rendere la porzione del file che consideriamo pari ad una sola pagina, quindi $\frac{P_R}{2^l} = 1$ e quindi $L(P_R) = \log_2(P_R)$.

Sono quindi necessarie $\log_2(P_R)$ letture (se il risultato del logaritmo non è un intero, allora prendiamo il valore intero superiore più vicino come caso peggiore)

Questo numero di letture ci permette di leggere la primissima pagina del file che potrebbe contenere tuple con city = 'Trento'. Se tutte le tuple con city = 'Trento' sono qui, abbiamo finito, ma in genere, se ne abbiamo più di una, e queste stanno in più pagine consecutive, dobbiamo leggere tutte le pagine che contengono tuple con city = 'Trento' più una pagina aggiuntiva per capire quando fermarci. Quante sono queste pagine?

Dipende quante tuple entrano in una pagina. Se una tupla occupa t_R byte e una pagina è grande P byte, allora il numero di tuple per pagina è

$$\left\lfloor \frac{P}{t_R} \right\rfloor$$

Abbiamo in totale $\left|R_{city='Trento}\right|$ tuple in R con city = 'Trento', quindi il numero di pagine che conterrà queste tuple sarà

$$\left\lceil \frac{|R_{\text{city='Trento'}}|}{\left\lfloor \frac{P}{t_R} \right\rfloor} \right\rceil .$$

Una di queste l'abbiamo già letta (è quella che abbiamo trovato con ricerca binaria), ma dovremo leggere una pagina in più per capire quando fermarci, quindi in totale, il numero di pagine da leggere rimane lo stesso:

$$\text{Costo}_{\text{EqSearch}} = \lceil \log_2(P_R) \rceil + \left\lceil \frac{|R_{\text{city='Trento'}}|}{\left\lfloor \frac{P}{t_R} \right\rfloor} \right\rceil .$$

Confrontando con il costo di una Equality Search in un heap file, che è P_R, questo è molto più conveniente.
Quindi la equality search è molto più efficiente su un sorted file (se la ricerca è fatta sull'attributo secondo cui le tuple sono ordinate). Ovviamente non è possibile ordinare le tuple rispetto a diversi attributi contemporaneamente. Dobbiamo scegliere quale vogliamo usare.

Il costo resta lo stesso anche se eseguiamo una ricerca per intervallo, in quanto il criterio è lo stesso per trovare la prima pagina, basta cercare la prima pagina col valore minimo dell'intervallo e da lì leggere tutte le pagine fino a che non si supera il valore massimo dell'intervallo.

DELETE
Per la delete facciamo praticamente le stesse operazioni della ricerca. L'unica differenza è che mentre leggiamo le pagine contigue che contengono le tuple con, ad esempio, city = 'Trento', dobbiamo riscriverle su disco senza queste tuple. Quindi il costo complessivo è

$$\text{Costo}_{\text{Delete}} = \lceil \log_2(P_R) \rceil + 2 \cdot \left\lceil \frac{|R_{\text{city='Trento'}}|}{\left\lfloor \frac{P}{t_R} \right\rfloor} \right\rceil .$$

Per mantenere i vantaggi del sorted file dobbiamo mantenere tutte le tuple in maniera contigua, quindi dobbiamo anche spostare tutte le tuple sulla destra di quelle eliminate di vari posti a sinistra, questo può richiedere tanti accessi al disco (alla peggio P_R). Però per il corso, non consideriamo questi costi, ma è importante sapere che esistono, in modo da apprezzare i vantaggi e gli svantaggi delle varie soluzioni.

INSERT
Vediamo ora la insert. Immaginiamo di voler aggiungere una tupla che riguarda una persona che vive a 'Verona'. Ora non possiamo più inserirla semplicemente in fondo al file, ma dobbiamo mantenere l'ordine. Quindi dobbiamo prima cercare nel file in che punto va inserita la tupla. Per questo serve una ricerca binaria, con costo $\log_2(P_R)$, e poi dobbiamo scrivere la pagina con la nuova tupla, quindi $+1$.
Questo ovviamente nel caso fortuito che la pagina abbia spazio disponibile per inserire la nuova tupla. In questo caso il costo sarà

$$\text{Costo}_{\text{Insert}} = \lceil \log_2(P_R) \rceil + 1.$$

Ma se non c'è spazio dobbiamo anche "spostare" in avanti di una pagina tutte le pagine e fare spazio per una pagina nuova, e questo è molto costoso (alla peggio serve leggere P_R pagine e scriverne altre P_R). Se vengono fatte tante insert, ciò accadrà quasi sicuramente prima o poi. Ma anche ignorando questo fattore, vediamo che le insert sono più costose in un sorted file, visto che in un Heap l'insert ha costo costante 2.

Indexed file

Gli indici vengono utilizzati per velocizzare l'accesso ai record sotto determinate condizioni di ricerca.
Le strutture a indice forniscono percorsi di accesso secondari.
Per "creare" un indice può essere usato qualsiasi campo. Si possono costruire più indici

La maggior parte degli indici sono basati su file ordinati e l'indice è organizzato in una struttura ad albero.

L'**indice ordinato** è simile all'indice in un libro di testo, dove c'è un **indexing field/attribute** (ovvero il campo/attributo di indicizzazione).
L'indice salva ogni valore dell'index field con una lista di puntatori a tutti i blocchi del disco che contengono un record con quel valore.
I valori nell'indice sono **ordinati**.

Esistono vari tipi di indice:
Il **primary index**, dove l'indexing field è una chiave e di conseguenza il file è ordinato tramite questa chiave (tipicamente si usa la chiave primaria).
Il **clustering index** viene usato se più record possono avere lo stesso valore per l'indexing field (ovvero non è una chiave).
Un file può avere al massimo un primary index o al massimo un clustering index, ma non entrambi, visto che gli indici si basano sull'ordinamento fisico dei dati nel file.

Ci sono poi i **secondary index**, che possono essere specificati su qualsiasi campo non ordinativo. Un file può avere uno o più secondary index.

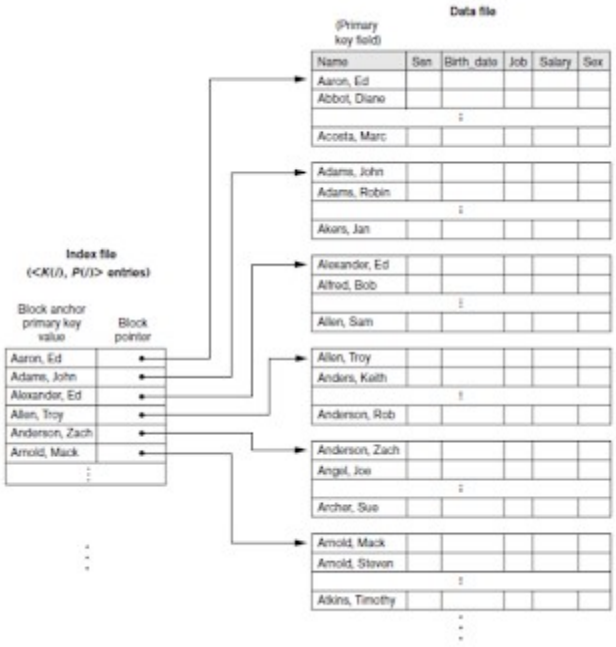
Gli indici possono essere:

- **Densi**
  Se l'indice ha una voce per ogni valore della chiave di ricerca del file dei dati
- **Sparsi**
  Se l'indice ha delle voci solo per alcuni valori della chiave di ricerca

I **primary index** sono file ordinati con due campi:

- La chiave primaria $K(i)$
- Il puntatore a un blocco del disco $P(i)$

Ci sarà una voce dell'indice nel file dell'indice per ogni blocco del file dei dati.
Il primary index è un indice **sparso**.



Vediamo ora i costi sul primary index.
Osserviamo che il file contentente l'indice primario occupa meno pagine dei dati stessi in quanto:

- Ci sono meno tuple rispetto al file dei dati
- Ogni tupla occupa meno spazio (solo due campi a dimensione fissa)

Per fare una ricerca, devo prima trovare la pagina (block) in cui ci sono i dati, quindi devo eseguire una ricerca binaria sul file dell'indice, che costa $\log_2 P_i$ (dove P_i è il numero di pagine occupate dall'indice).
Poi devo leggere la pagina trovata, quindi il costo diventa $\log_2 P_i + 1$

Prendiamo in considerazione una relazione R con $|R| = 300.000$ tuple.
Consideriamo pagine di dimensione $P = 4\,KB = 4.096\ \ byte$.
La dimensione di una tupla è $t_R = 100\ \ byte$.
Di conseguenza abbiamo $\frac{4.096}{100} = 40$ tuple per pagina, e quindi abbiamo $P_R = \frac{300.000}{40} = 7.500$ pagine.
Se facessimo una ricerca binaria direttamente sul file dei dati (sorted file), avremmo un costo pari a $\log_2 7.500 = 13$ accessi.

Nel file dell'indice prendiamo in considerazione il campo chiave da $9\ \ byte$ e il puntatore alla pagina con i dati da $6\ \ byte$, di conseguenza ogni tupla avrà dimensione $t_i = 15\ \ byte$.

Per ogni pagina abbiamo $\frac{4.096}{15} = 273$ indici.

Quindi il numero di pagine nel file dell'indice è $P_i = \frac{7.500}{273} = 28$ (perchè vogliamo un indice per ogni pagina del file dei dati)
Il costo della ricerca della pagina dei dati sul file dell'indice è di $\log_2 28 = 5$ accessi, di conseguenza il costo totale è di $5 + 1$ accessi (molto più conveniente)

Notiamo che visto le dimensioni ridotte del file di indice ($7.500 * 15 \ byte = 112{,}5 \ KB$) la ricerca binaria può di fatto essere eseguita nella memoria principale a costi ridottissimi.

Il **problema** dei primary index sono gli **inserimenti** e le **eliminazioni**, perchè bisogna spostare i record attorno e cambiare di conseguenza gli indici.
Per risolvere questi problemi si può:

- Usare un file per i record in eccesso non ordinato
- Usare una linked list per i record in eccesso

I **secondary index** forniscono un secondo "significato" agli accessi al file dei dati, dove esiste già esiste un accesso primario.
I secondary index sono dei file ordinati con due campi:

- L'indexing field $K(i)$
- Il puntatore a un blocco del disco (o al record) $P(i)$

Di solito un secondary index ha bisogno di più spazio e di un tempo di ricerca più lungo rispetto a un primary index, però migliora il tempo di ricerca per record arbitrari

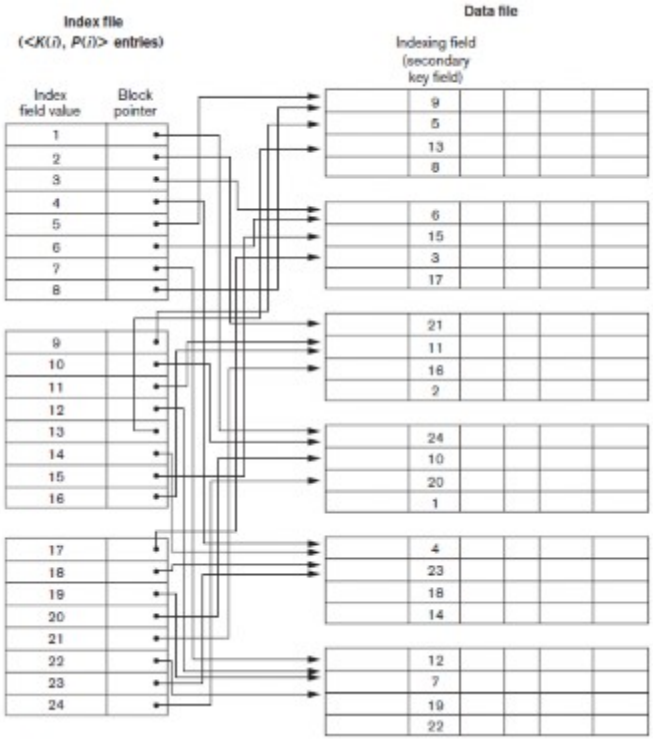


Figure 17.4 Dense secondary index (with block pointers) on a nonordering key field of a file.

	Index Field Used for Physical Ordering of the File	Index Field Not Used for Physical Ordering of the File
Indexing field is key	Primary index	Secondary index (Key)
Indexing field is nonkey	Clustering index	Secondary index (NonKey)

Table 17.1 Types of indexes based on the properties of the indexing field

Type of Index	Number of (First-Level) Index Entries	Dense or Nondense (Sparse)	Block Anchoring on the Data File
Primary	Number of blocks in data file	Nondense	Yes
Clustering	Number of distinct index field values	Nondense	Yes/no[a]
Secondary (key)	Number of records in data file	Dense	No
Secondary (nonkey)	Number of records[b] or number of distinct index field values[c]	Dense or Nondense	No

[a]Yes if every distinct value of the ordering field starts a new block; no otherwise.
[b]For option 1.
[c]For options 2 and 3.

Table 17.2 Properties of index types

Gli **indici multilivello** sono stati progettati per ridurre notevolmente lo spazio di ricerca rimanente durante la ricerca.
La struttura è la seguente:

- **Index file**
  Il primo livello (o livello base) di un indice multilivello
- **Second level**
  Il primary index per il primo livello
- **Third level**
  Il primary index per il secondo livello

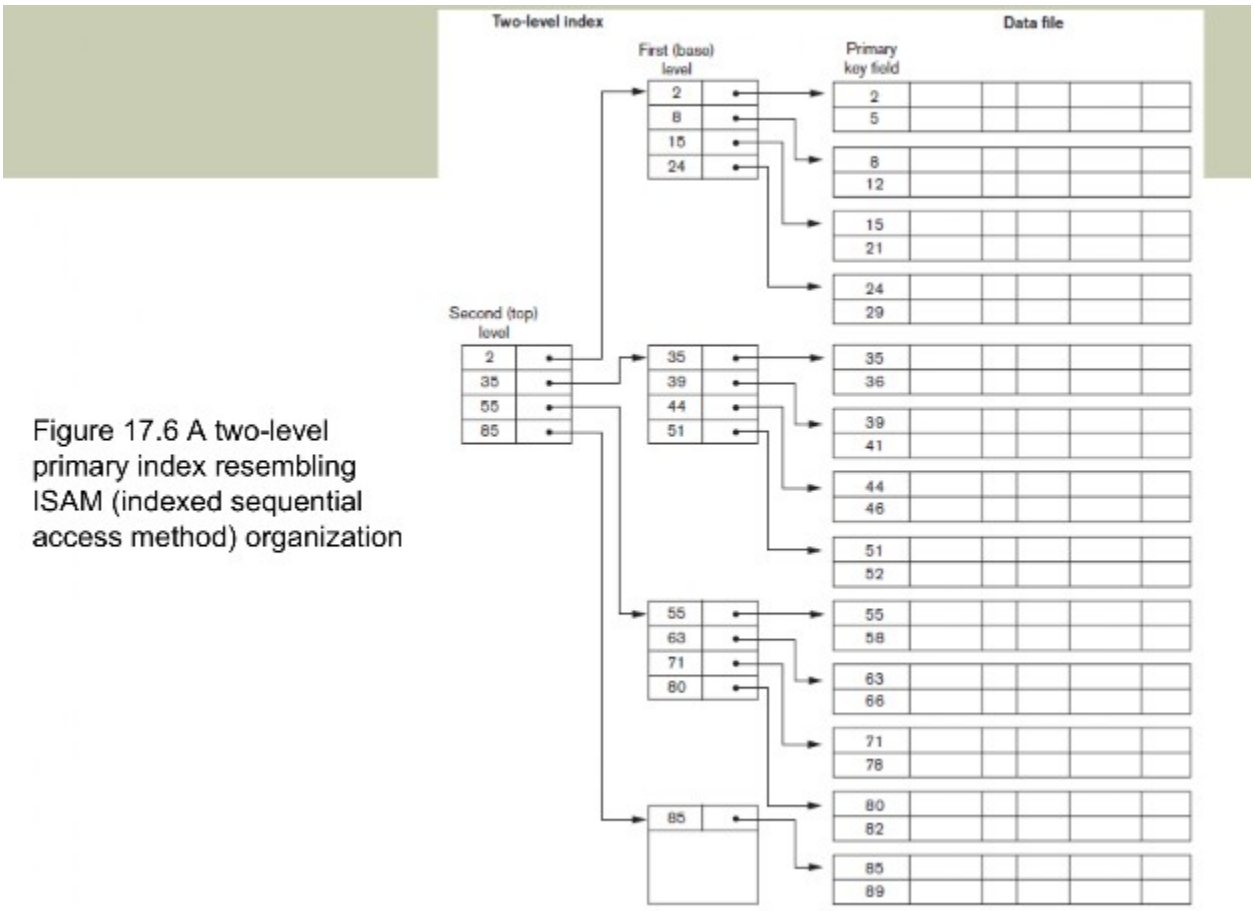


Figure 17.6 A two-level primary index resembling ISAM (indexed sequential access method) organization

B+Tree

Vediamo ora gli **indici dinamici multilivello** usando **B+Trees**
La terminologia delle strutture dati ad albero è la seguente:

- L’albero è formato da nodi
- Ogni nodo (tranne la radice) ha un nodo genitore e zero o più nodi figli
- I nodi foglie non hanno nodi figli
- L’albero si dice sbilanciato se i nodi foglie appaiono in livelli differenti
- I nodi non foglia sono chiamati nodi interni
- Il sottoalbero di un nodo consiste nel nodo e tutti i suoi discendenti

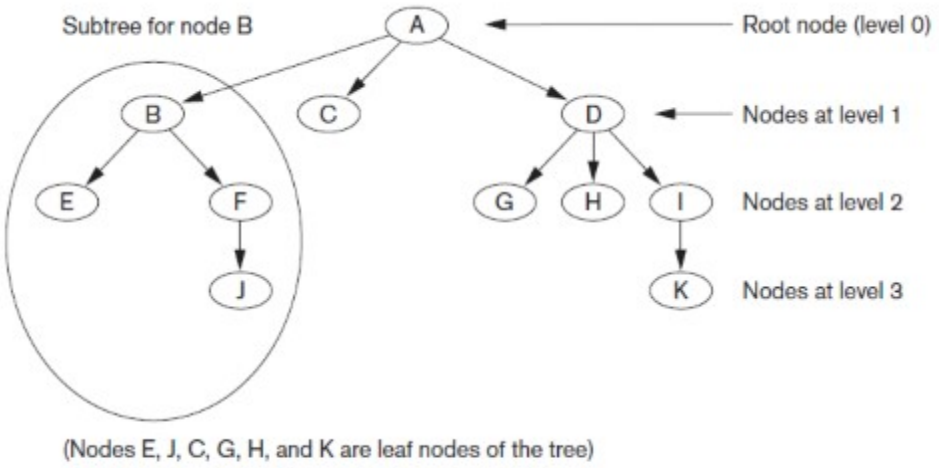


Figure 17.7 A tree data structure that shows an unbalanced tree

Nei B+Trees i puntatori ai dati vengono salvati solo nei nodi foglia. Le foglie hanno una voce per ogni valore del campo di ricerca, e un puntatore al record se il campo di ricerca è chiave.
Se il campo di ricerca non è chiave, il puntatore punta a un blocco contenente i puntatori ai record nel file dei dati.

Nei nodi interni abbiamo qualche valore del campo di ricerca per “guidare” la ricerca.

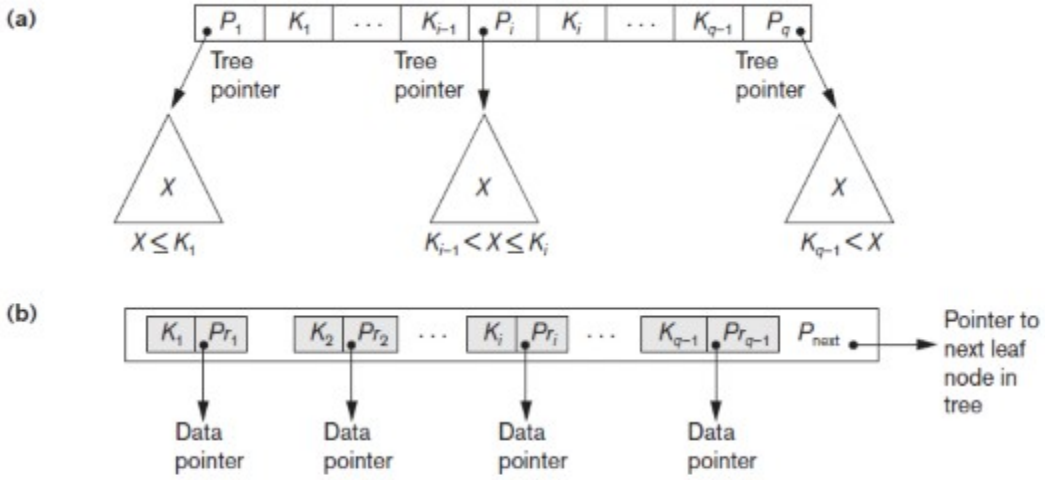
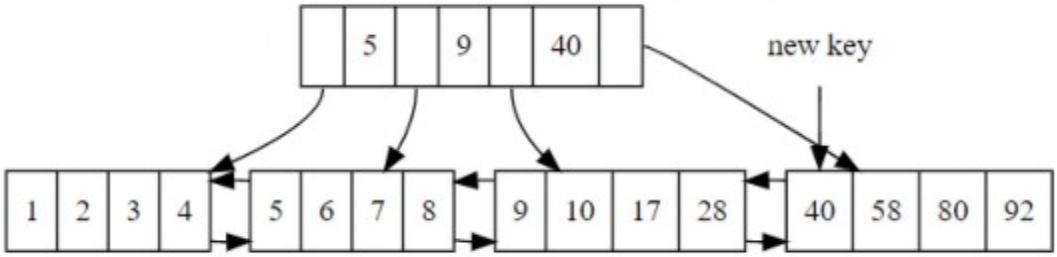


Figure 17.11 The nodes of a B+-tree (a) Internal node of a B+-tree with $q-1$ search values (b) Leaf node of a B+-tree with $q-1$ search values and $q-1$ data pointers



Analizziamo ora i costi dei B+Tree

LOOKUP

Per costo di lookup si intende il numero di pagine che dobbiamo aprire per trovare il nodo foglia che contiene i puntatori ai dati che stiamo cercando.

Sia B il numero massimo di figli che può avere un nodo interno di un B+Tree e sia A la chiave di ricerca.

Il costo sarà $\lceil \log_B(|R.A|) \rceil$.

EQUALITY SEARCH

Sia la relazione R memorizzata su un file non ordinato (unclustered).

Immaginiamo di voler trovare tutte le tuple della relazione R che corrispondono ad un filtro, tipo:

$$SELECT \ * \ ¿ \ R \ WHERE \ age = 30;$$

Dato B il numero massimo di figli che può avere un nodo abbiamo:

$$\text{Costo}_{\text{EqSearch}} = \lceil \log_B(|R.\text{age}|) \rceil + |R_{\text{age}=30}|$$

Se invece la relazione R è memorizzata su un file ordinato:

$$\text{Costo}_{\text{EqSearch}} = \lceil \log_B(|R.\text{age}|) \rceil + \left\lceil \frac{|R_{\text{age}=30}|}{\left\lfloor \frac{P}{t_R} \right\rfloor} \right\rceil$$

Se volessimo effettuare una ricerca per intervalli come la seguente

$$SELECT \ * \ ¿ \ R \ WHERE \ age > 18 \ \wedge \ age < 30;$$

Dobbiamo leggere la foglia che contiene le tuple con $age = 18$, e poi possiamo usare i puntatori "orizzontali" tra le foglie per trovare tutte le pagine del file dei dati che ci servono.

La ricerca su B+Tree è in generale più efficiente di quella su sorted file, perchè:

- Non ci sono costi per mantenere ordinato il file dei dati
- La ricerca è logaritmica in base B (e non in base 2)

INSERT

L'inserimento di una nuova tupla ha lo stesso costo dell'Equality Search, più il costo di creare (eventualmente) una nuova pagina e di salvare la pagina modificata su disco:

$$\text{Costo}_{\text{Insert}} = \lceil \log_B(|R.\text{age}|) \rceil + 1 + 1$$

Non consideriamo il costo di aggiornare l'indice dopo una Insert.

SCAN

Lo scan ha il solito costo di aprire tutte le pagine in cui è memorizzata la relazione R

$$\text{Costo}_{\text{Scan}} = P_R$$

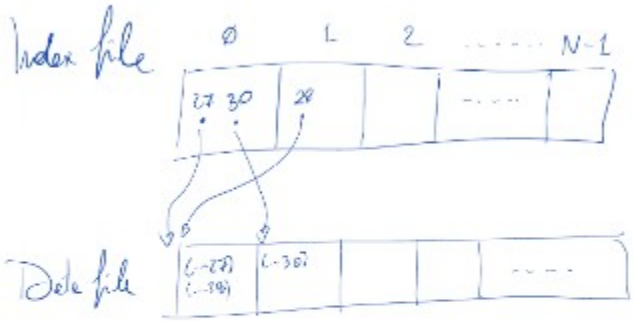
Hash indexes

È una struttura secondaria per accedere ai file
Si usa l'hashing per una chiave di ricerca diversa da quella usata per l'ordinamento primario.
Le voci dell'indice sono nella forma (K, P_r) o (K, P):

- P_r: puntatore al record che contiene la chiave
- P: puntatore al blocco che contiene il record per quella chiave

L'idea è di scegliere una **funzione Hash** (ad esempio per i numeri interi possiamo usare la funzione resto $h(x) = x \,\%\, N$ dove N è l'operatore resto della divisione).
In base al valore della funzione Hash su un certo attributo, si allocano i diversi puntatori in un **bucket**.
Se la pagina del bucket contiene troppi puntatori, il DBMS dovrà creare una pagina di overflow, che sarà connessa alla pagina principale del bucket e che potrà contenere i nuovi puntatori al file di dati.

Hash Indexes: esempio



Esempio per l'attributo *age*

- Per trovare una tupla con un certo valore di *age x* basterà applicare la funzione di hash *h* a *x*
- Questo ci darà il bucket che contiene il puntatore alla pagina del data file con quella tupla (al costo di leggere la o le pagine che contengono il bucket nell'index file)

Hash Indexes (cont'd.)

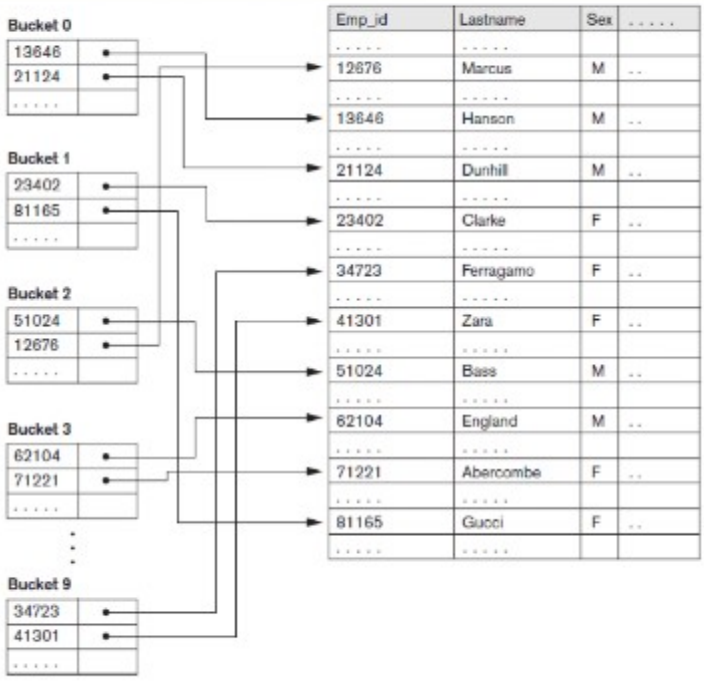


Figure 17.15 Hash-based indexing

Vediamo ora i costi delle principali operazioni:

EQUALITY SEARCH

Immaginiamo di voler trovare tutte le tuple di una relazione R che corrispondono al filtro:

$$SELECT \ * \ ¿ \ R \ WHERE \ age = 30;$$

Il costo sarà:

$$\text{Costo}_{\text{EqSearch}} = L_h + |R_{A=c}|$$

dove

- L_h è il costo del lookup nel file di indice
- $|R_{A=c}|$ è il numero di tuple che hanno valore c per l'attributo A nella relazione R

Il costo del lookup nel file di indice L_h è dato dal calcolo della funzione h sul valore cercato (che si fa in memoria centrale a costo 0) e dalla lettura delle pagine che contiene il bucket.

Se ci sono pagine di overflow, è necessario leggerle tutte, anche se in generale, se la funzione di hash è scelta bene, non ce ne saranno molte (per default si assume un valore medio di 1,2). L'hash index è quindi **MOLTO efficiente** in caso di Equality Search.

Per la ricerca su intervalli invece non è molto efficiente, perché non è detto che tuple con valori vicini di un attributo siano nello stesso bucket dell'index file.
Per ogni valore dell'intervallo dobbiamo rifare la lookup, trovare il bucket giusto nell'index file e poi leggere tutte le pagine puntate da quel bucket.

INSERT

L’inserimento di una nuova tupla richiede:

- di aprire l’ultima pagina del file di dati e appendere la nuova tupla in fondo (costo: 1+1)
- eseguire il lookup per trovare il bucket in cui inserire il puntatore al nuovo valore (costo L_h)

Il costo quindi sarà

$$\text{Costo}_{\text{Insert}} = L_h + 1 + 1$$

Anche in questo caso ignoreremo il costo di aggiornare l’indice dopo una INSERT.

DELETE

La cancellazione delle tuple in base al valore c di un attributo A richiede:

- eseguire il lookup per trovare il bucket con i puntatori alle tuple con valore c (costo L_h)
- aprire una alla volta le pagine del file di dati che contengono tuple con valore c per l’attributo A, rimuovere le tuple e riscrivere la pagina modificata (costo $2 * \left| R_{A=c} \right|$)

Il costo totale quindi è

$$Costo_{Delete} = L_h + 2 * \left| R_{A=c} \right|$$

SCAN

Lo SCAN ha il solito costo di aprire tutte le pagine in cui è memorizzata la relazione R:

$$\text{Costo}_{\text{Scan}} = P_R$$

Possiamo quindi ignorare l’index file e accedere direttamente al file dei dati.

Quindi l’hash index è molto efficiente, tranne che per la ricerca dell’intervallo.
Tuttavia osserviamo che, prima o poi, i bucket richiederanno sempre più pagine di overflow e quindi il costo di *lookup* aumenterà. Il DBMS dovrà periodicamente riorganizzare l’index file, utilizzando una funzione di hash con più bucket. Questo però richiederà di spostare i puntatori nei nuovi bucket, con relativo costo. Questo approccio si chiama **Extendible Hash Index**.

Creazione indice su DBMS

A simple version of `CREATE INDEX` statement is as follows:

```
CREATE INDEX index_name ON table_name [USING method]
(
    column_name [ASC | DESC] [NULLS {FIRST | LAST }],
    ...
);
```

In this syntax:

- First, specify the index name after the `CREATE INDEX` clause. The index name should be meaningful and easy to remember.
- Second, specify the name of the table to which the index belongs.
- Third, specify the index method such as `btree`, `hash`, `gist`, `spgist`, `gin`, and `brin`. PostgreSQL uses `btree` by default.
- Fourth, list one or more columns that to be stored in the index. The `ASC` and `DESC` specify the sort order. ASC is the default. `NULLS FIRST` or `NULLS LAST` specifies nulls sort before or after non-nulls. The `NULLS FIRST` is the default when `DESC` is specified and `NULLS LAST` is the default when `DESC` is not specified.

Slide 17-

Esiste la pratica di aggiornamento degli indici (tuning indexes) che dinamicamente rivaluta i requisiti per riorganizzare gli indici e avere performance migliori, perchè:

- Alcune query ci mettono troppo tempo per un problema con gli indici
- Alcuni indici possono non essere utilizzati
- Alcuni indici possono subire troppi aggiornamenti perchè sono basati su un attributo che è sottoposto a frequenti cambiamenti.

Le decisioni da prendere quando si progetta un database sono le seguenti:

- Se indicizzare un attributo
  L'attributo è una chiave o è usato da una query
- Quali attributi indicizzare
  Singolo o multiplo
- Se usare un clustered index
  Uno per tabella
- Se usare un hash index o un tree index
  Gli hash index non supportano le query sugli intervalli
- Se usare hashing dinamico
  Appropriato per file molto volatili

Operazioni di Join

Fino ad ora abbiamo visto le diverse modalità di eseguire operazioni su **singole relazioni** (operazioni come scan, equality search, etc).
Però bisogna ricordare che una delle operazioni più utilizzate (e dispendiose) nelle interrogazioni è l'operazione di **JOIN**. Analizziamo quindi diversi algoritmi per calcolare in modo più o meno efficiente la join fra due relazioni.

Prendiamo come riferimento il problema di fare la JOIN tra due relazioni $R(A,B)$ e $S(C,D)$ con la condizione $B=C$, cioè la JOIN

$$R \underset{R.B=S.C}{} S$$

Come sempre per "costo" intendiamo il numero di pagine che dovrà leggere e/o scrivere l'operazione che prendiamo in considerazione per ottenere il risultato completo.
Assumiamo che man mano che le tuple del risultato vengono prodotte dall'algoritmo di join, esse non vengano riscritte su disco, ma solamente mostrate a video all'utente, quindi non abbiamo nessun costo di scrittura.

Prendiamo in considerazione le seguenti tuple per le due relazioni (non ci interessa sapere i valori degli attributi A e D):

A	B
...	7
...	9
...	6
...	15
...	20

R

C	D
15	...
10	...
7	...
9	...
20	...

S

Nested loop join

L'algoritmo più semplice per il calcolo della join è il Nested loop join. Come il nome suggerisce, l'idea è di implementare la join con due cicli for:

1. Per ogni tupla t di R fai
    a. Per ogni tupla t' di S fai
        i. Se $t.B=t'.C$ allora stampa a video la tupla t,t'

Vediamo quante tuple l'algoritmo dovrà leggere dal disco:
Inizierò con la prima tupla di R, e per questa tupla dovrà leggere tutte le tuple di S, quindi $1+|S|$.
Questo dovrà farlo per ogni tupla di R, quindi moltiplichiamo per il numero di tuple di R e otteniamo $|R|\cdot(1+|S|)=|R|+|R|\cdot|S|$.

Ricordiamo però che il DBMS non legge mai una tupla per volta, ma legge sempre blocchi di tuple, ovvero **pagine**. Di conseguenza l'algoritmo corretto di nested join loop è:

1. Per ogni pagina p di R fai
    a. Per ogni pagina p' di S fai

i. Per ogni coppia di tuple $t \in p$, $t' \in p'$ tali che $t.B = t'.C$, stampa a video t, t'

In sostanza, l'algoritmo legge un gruppo di tuple di R e un gruppo di tuple di S, e una volta caricate in memoria centrale, farà una "piccola join" fra queste tuple. Quest'ultima join è effettuata senza leggere pagine dal disco, e quindi non ha costo.

Usando un'analisi identica a quella usata per la versione basata su tuple, otteniamo che il costo sarà:

$$\text{Costo}_{\text{NestedLoopJoin}} = P_R + P_R \cdot P_S$$

Notiamo che se la JOIN fosse stata $S \underset{S.C=R.B}{} R$ il costo sarebbe stato $P_S + P_S \cdot P_R$.
I due costi hanno un termine in comune ($P_S \cdot P_R$) e quindi differiscono solo per il primo termine. Infatti il DBMS quando decide di usare questo algoritmo, predilige di fare la JOIN partendo dalla relazione più piccola, così da avere un costo finale inferiore. (Esistono ovviamente metodi più efficienti di fare una join che rendono indipendente la scelta della relazione più piccola, però questo è l'algoritmo più semplice da implementare ed è applicabile ovunque, indipendentemente dal fatto che le tuple siano ordinate o meno o abbiano degli indici).

Sort-merge join

Qui diamo per assunto che le tuple non siano memorizzate in ordine sparso, bensì siano ordinate rispetto al valore dell'attributo che partecipa alla join, riprendendo le tuple viste prima avremo quindi:

A	B
...	6
...	7
...	9
...	15
...	20

R

C	D
7	...
9	...
10	...
15	...
20	...

S

Sotto questa assunzione possiamo effettuare la join in modo più efficiente.
Assumiamo per semplicità che gli attributi $R.B$ e $S.C$ non contengano duplicati (con la presenza
di duplicati il costo di questo algoritmo sale leggermente). E immaginiamo di focalizzarci sulle tuple.

Procediamo in questa maniera:

1. Prendiamo la prima tupla di R, diciamo t, e la prima tupla di S, diciamo t'
2. Confrontiamo i valori $t.B$ e $t'.C$
   - Se $t.B$ è minore di $t'.C$ (ad esempio $t.B = 6$ e $t'.C = 7$) lasciamo in memoria t' e leggiamo la tupla successiva di R e la inseriamo in t.
   - Se $t.B$ è uguale a $t'.C$ (ad esempio $t.B = 7$ e $t'.C = 7$) le tuple vengono messe in join e mostrate a schermo, e leggiamo e aggiorniamo la tupla successiva di una delle due relazioni (indifferente quale).

- Se $t.B$ è maggiore di $t'.C$ (ad esempio $t.B = 9$ e $t'.C = 7$) lasciamo in memoria t e leggiamo la tupla successiva di S e la inseriamo in t'.

3. Ricominciamo dal punto 2

Notiamo che con questo algoritmo abbiamo letto tutte le tuple di R e di S, ma soprattutto **ogni tupla non è mai stata letta più di una volta**. Quindi abbiamo letto $|R|+|S|$ tuple.
Per lo stesso discorso di prima il DBMS non accede ad una tupla per volta ma ad una pagina per volta, di conseguenza il costo sarà $P_R + P_S$. Questo è un netto miglioramento rispetto al Nested join loop.
Però a tutto c'è un prezzo. Infatti questo algoritmo può essere applicato solo se le due relazioni sono ordinate rispetto all'attributo di join. Se non è così l'algoritmo dovrà prima **riordinare** le tuple delle due relazioni. Per questo l'algoritmo si chiama sort-merge, perchè abbiamo prima una fase di ordinamento e poi una fase di fusione (quella per calcolare il join).

External sorting

Se le tuple di R ed S non sono ordinate, quanto costa riordinarle?

Un modo per ordinare le tuple è quello di leggere tutte le pagine di R in memoria centrale (a costo P_R), ordinare le tuple (a costo 0), e riscriverle sul disco (a costo P_R). Questo è impraticabile in quanto lo spazio della RAM potrebbe non riuscir a contenere tutte le tuple di R.
Quindi ci serve un algoritmo più intelligente che legga alcune pagine alla volta della relazione in RAM, effettui delle operazioni, riscriva le pagine modificate su disco, legga altre pagine di R, e così via.
Algoritmi di questo tipo vengono chiamati di **external sorting**. Ne esistono diversi ma non li vedremo in dettaglio. Accenneremo solo al costo dei due più importanti.

Il primo algoritmo (il più semplice) richiede che in RAM sia presente spazio sufficiente per memorizzare solo 3 pagine, e il suo costo è:

$$2 \cdot P_R \cdot (\lceil \log_2 P_R \rceil + 1)$$

L'altro algoritmo (più complesso), può sfruttare tante pagine in RAM quante il DBMS ne ha a disposizione. Se B è il numero di pagine che è possibile caricare in RAM, allora questo algoritmo ha costo:

$$2 \cdot P_R \cdot (\left\lceil \log_{B-1} \left\lceil \frac{P_R}{B} \right\rceil \right\rceil + 1)$$

Solitamente B è molto grande (sulle centinaia) e quindi il costo di questo algoritmo è molto più basso.
Di fatto è più basso di quello dell'algoritmo precedente anche quando $B = 3$, ma per questioni storiche, è importante menzionare il primo. Quindi, quando considereremo il caso di sole 3 pagine disponibili in RAM, assumeremo di usare il primo algoritmo.

Quindi nel complesso, se R ed S non sono già ordinate, il costo dell'algoritmo Sort-merge è di:

$$\text{Costo}_{\text{SortMergeJoin}} = \text{Costo per ordinare R} + \text{Costo per ordinare S} + P_R + P_S$$

Quindi Sort-merge join è sempre meglio del nested loop join? Se ci concentriamo sugli specifici algoritmi visti, allora si, è praticamente sempre da preferire, soprattutto quando le relazioni sono già ordinate.
Tuttavia, nested loop join è implementato con varie ottimizzazioni nei DBMS reali. Ad esempio, se R contiene l'elenco delle persone con associato un language id che indica la lingua che la persona parla e S contiene le lingue, e volessimo fare la join $R \underset{R.lan\text{-}id = S.lan\text{-}id}{} S$, allora molto probabilmente S è molto piccola, e il DBMS deciderà di leggerla completamente in RAM una volta sola, essenzialmente portando il costo della join a $P_R + P_S$.

Hash join

Un altro modo per rendere la join più efficiente è quello di utilizzare una hash table. L'idea è di utilizzare una funzione di hash h che possa essere applicata ai valori dell'attributo $R.B$ e ai valori dell'attributo $S.C$.
L'osservazione chiave è che se due tuple t e t' (una di R, e l'altra di S) possono essere messe in join, cioè $t.B = t'.C$, allora i due valori $t.B$ e $t'.C$ devono per forza avere stesso valore di hash. Ovviamente, se i due valori hanno stesso hash, non è detto che essi coincidano.
Consideriamo quinidi le nostre relazioni R ed S viste in precedenza:

A	B
...	7
...	9
...	6
...	15
...	20

R

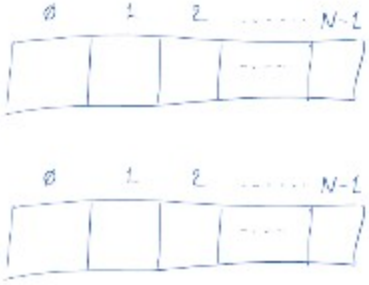
C	D
15	...
10	...
7	...
9	...
20	...

S

Assumiamo che la funziona hash che scegliamo sia:

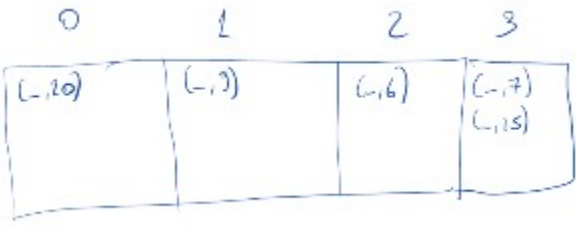
$$h(x) = x \,\%\, N$$

dove N è il numero di bucket scelto dal DBMS.
Il DBMS creerà allora una copia di R e una di S su disco, dove però le tuple sono organizzate secondo la funzione di hash. (In particolare il DBMS utilizzerà N pagine su disco per la relazione R e N pagine per la relazione S).

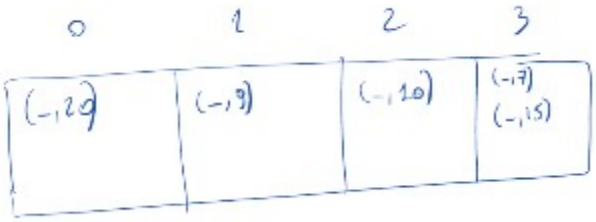


Come prima cosa quindi bisogna riscrivere tutte le tuple di R e di S nei bucket giusti.
Il DBMS inizierà con R. Leggerà una pagina per volta di R, e per ciascuna tupla t in quella pagina, calcolerà il valore di hash $h(t.B)$ per capire a quale bucket assegnare la tupla. Ad esempio, se $t.B = 6$, allora $h(t.B) = h(6) = 6 \,\%\, N$. Se $N = 4$, allora $h(6) = 2$. Quindi la tupla verrà

scritta nel bucket 2. Quando le tuple della pagina di R letta saranno esaurite, si passerà alla pagina successiva.
Il costo di questa operazione è pari al costo di leggere tutte le pagine di R, quindi P_R, più il costo per **ogni tupla** di leggere il bucket corretto in RAM (1), modificarlo con la tupla da inserire (0), e riscrivere il bucket su disco (1).
Quindi il costo sarà $P_R + |R| \cdot (1+1)$.
Così otteniamo una copia della relazione R su disco, dove però le tuple sono organizzate per bucket.
Se ad esempio consideriamo la relazione R dell'esempio e la funzione di hash $h(x) = x \,\%\, 4$, avremo:



Successivamente il DBMS farà lo stesso per la relazione S (con costo $P_S + |S| \cdot (1+1)$.
Considerando l'esempio otteniamo:



Adesso che le relazioni sono copiate sul disco e divise in bucket, è chiaro che se due tuple t e t' possono andare in join, ovvero $t.B = t'.C$, allora il bucket in cui si trova t e il bucket in cui si trova t' devono avere lo stesso numero (ovvero stesso hash, essendo lo stesso valore).
Però non è detto che due tuple che finiscono nello stesso bucket possano andare in join (per le proprietà delle funzioni hash).
Quindi ora per fare il join tra R ed S sarà sufficiente leggere gli stessi bucket dalle copie di R ed S e calcolare tutte le join possibili in memoria centrale tra le tuple del primo bucket con le tuple del secondo bucket. Siamo certi a questo punto che nessuna tupla del bucket letto di R dovrà andare in join con tuple di S che si trovano in un bucket diverso, e viceversa.
Assumendo ciascun bucket sia sufficientemente grande da contenere tutte le tuple con quel valore di hash della relazione R (e della relazione S), e ricordando che la copia di R (e la copia di S) contiene di fatto le medesime tuple di R (e di S), il numero totale di pagine lette sarà $P_R + P_S$.
Quindi il costo totale sarà pari al costo di creare le due copie, più il costo di fare la join:

$$\text{Costo}_{\text{HashJoin}} = (P_R + 2 \cdot |R|) + (P_S + 2 \cdot |S|) + (P_R + P_S)$$

Se le tuple delle due relazioni sono già distribuite nei bucket secondo una certa funzione di hash, allora il costo della join sarà semplicemente $P_R + P_S$ (di gran lunga migliore del costo di una nested loop join). Però se così non fosse ci sarà un costo da pagare per ridistribuire prima le tuple.
Quindi l'utilizzare o meno questo metodo, come anche il sort-merge join, dipende dal valore effettivo dei costi dei vari metodi per specifici valori di P_R, P_S, $|R|$, $|S|$, ecc.
Scegliere quale dei metodi è il più indicato è compito del DBMS.

Index Nested loop join

Come indica il nome si tratta di una variante del nested loop join, che però sfrutta la presenza di un **indice**.
Consideriamo di nuovo le nostre relazioni

A	B
...	7
...	9
...	6
...	15
...	20

R

C	D
15	...
10	...
7	...
9	...
20	...

S

Assumiamo che adesso sull'attributo $S.C$ sia stato creato un indice (es., un B+-tree o un hash index). Allora possiamo sfruttare questo indice per rendere molto più efficiente il nested loop join classico, evitandoci di utilizzare due cicli for che accedono entrambi al disco. In particolare, il nuovo algoritmo diventerebbe:

1. Per ogni pagina p di R fai
    a. Per ogni tupla $t \in p$, trova tutte le tuple t' in S con $t'.C = t.B$ (tramite l'indice), e stampa t, t'.

La differenza principale con il nested loop join è che ora, per trovare tutte le tuple di S che vanno in join con una certa tupla di R, non sarà necessario fare uno scan completo di S (e quindi costo
P_S), ma sarà sufficiente fare una equality search su S, usando l'indice, per trovare le tuple con valore nell'attributo $S.C$ pari al valore dell'attributo B della tupla di R.

Quindi il costo dell'algoritmo dipenderà dal tipo di indice che abbiamo e sarà:

$$\text{Costo}_{\text{IndexNestedLoopJoin}} = P_R + |R| \cdot \text{Costo}_{\text{EqSearchS}}$$

Il fattore quadratico si è ridotto notevolmente visto che il costo di un equality search indicizzata è generalmente più basso del costo di fare uno scan completo della relazione S.
Notiamo che l'indice deve essere obbligatoriamente presente su $S.C$ e invece in questa situazione l'indice su $R.B$ non servirebbe a nulla (a meno di invertire le relazioni nel join).

Tabelle riassuntive costi

Di seguito vengono riportate due tabelle riassuntive con i costi delle operazioni su singola relazione e i costi degli algoritmi di join, e una tabella che riassume la notazione utilizzata. I valori L_b e L_h indicano rispettivamente i costi di lookup in un B$^+$-tree e in un Hash index. Se conosciamo il numero di figli di un nodo intermedio del B$^+$-tree e il valore $|R.A|$, allora $L_b = \lceil \log_B |R.A| \rceil$, altrimenti si assume $L_b = 3$. Mentre per L_h, se il suo valore esatto non ci viene dato, assumiamo $L_h = 1.2$.

Tabella 1: Costi Operazioni su singola relazione

	Scan SELECT * FROM R	EqSearch SELECT * FROM R WHERE R.A = c	Insert INSERT INTO R VALUES (...)	Delete DELETE FROM R WHERE R.A = c
Heap file	P_R	P_R	$1 + 1$	$P_R + 2 \cdot \lvert R_{A=c} \rvert$
Sorted file	P_R	$\lceil \log_2 P_R \rceil + \left\lceil \frac{\lvert R_{A=c} \rvert}{\lfloor \frac{P}{t_R} \rfloor} \right\rceil$	$\lceil \log_2 P_R \rceil + 1$	$\lceil \log_2 P_R \rceil + 2 \cdot \left\lceil \frac{\lvert R_{A=c} \rvert}{\lfloor \frac{P}{t_R} \rfloor} \right\rceil$
Unclustered B$^+$-tree	P_R	$L_b + \lvert R_{A=c} \rvert$	$L_b + 1 + 1$	$L_b + 2 \cdot \lvert R_{A=c} \rvert$
Clustered B$^+$-tree	P_R	$L_b + \left\lceil \frac{\lvert R_{A=c} \rvert}{\lfloor \frac{P}{t_R} \rfloor} \right\rceil$	$L_b + 1 + 1$	$L_b + 2 \cdot \left\lceil \frac{\lvert R_{A=c} \rvert}{\lfloor \frac{P}{t_R} \rfloor} \right\rceil$
Hash index	P_R	$L_h + \lvert R_{A=c} \rvert$	$L_h + 1 + 1$	$L_h + 2 \cdot \lvert R_{A=c} \rvert$

Tabella 2: Costi algoritmi di join. Relazioni R(A,B) e S(C,D).

	Costo di $R \underset{R.B=S.C}{\ltimes} S$
Nested loop join	$P_R + P_R \cdot P_S$
Sort-merge join	$\text{Costo}_{\text{OrdR}} + \text{Costo}_{\text{OrdS}} + P_R + P_S$
Hash join	$(P_R + 2 \cdot \lvert R \rvert) + (P_S + 2 \cdot \lvert S \rvert) + (P_R + P_S)$
Index Nested Loop (indice su S.C)	$P_R + \lvert R \rvert \cdot \text{Costo}_{\text{EqSearchS}}$

Tabella 3: Notazione

P	Dimensione di una pagina in byte
t_R	Dimensione di una tupla di R in byte
P_R	Numero di pagine occupate dalla relazione R
$\lvert R.A \rvert$	Numero di valori distinti nell'attributo A di R
$\lvert R_{A=c} \rvert$	Numero di tuple di R con R.A=c
$\left\lfloor \frac{P}{t_R} \right\rfloor$	Numero di tuple che entrano per intero in una pagina
$\left\lceil \frac{\lvert R_{A=c} \rvert}{\lfloor \frac{P}{t_R} \rfloor} \right\rceil$	Numero di pagine necessarie a contenere tutte le tuple di R con R.A=c
$\lvert R_{A=c} \rvert = \left\lceil \frac{\lvert R \rvert}{\lvert R.A \rvert} \right\rceil$	Se $\lvert R_{A=c} \rvert$ non è noto ma $\lvert R.A \rvert$ lo è
$\lvert R_{A=c} \rvert = \lceil f \cdot \lvert R \rvert \rceil$	Se $\lvert R_{A=c} \rvert$ non è noto ma il selectivity factor f dell'attributo A lo è